



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Vibe Coding for React Development

Course Code: C1143

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 4.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	10 July 2026	Initial release — Learner Guide covering all 24 hands-on labs across six topics, with key-concept explanations, the AI React Bug Checklist, troubleshooting and a glossary.	Dr. Alfred Ang
2.0	11 July 2026	Added a JavaScript prerequisites chapter and per-topic Concept deep-dive sections with worked code and analogies.	Dr. Alfred Ang
3.0	11 July 2026	Restructured the whole course around ONE progressively-built full-stack project on Neon + Vercel: removed the standalone side projects, expanded the props / refs / hooks / components / state explanations, and added the mini-capstone (add a course-reviews feature, mostly on your own).	Dr. Alfred Ang
4.0	11 July 2026	Replaced the running project with Cook & Bake Academy (a cooking & bakery school website, deployed live at cookbake-academy.vercel.app). Re-architected Topic 5 from the browser-side Neon Data API + Row Level Security to a three-tier serverless API (/api routes over Neon, parameterised SQL, bcrypt + JWT). Added a foundations chapter on the real DOM vs the Virtual DOM and Babel/JSX compilation, and deepened useRef; removed the	Dr. Alfred Ang

		JavaScript-refresher chapter (modern JavaScript is now assumed).	
--	--	---	--

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

Introduction

This Learner Guide accompanies the course AI Vibe Coding for React Development (C1143), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 25 hands-on labs, organised by the six course topics, together with an explanation of the key concept behind every lab.

Across the course you build one application: Cook & Bake Academy, a full-stack React course catalogue with a Neon Postgres database, user accounts, a private dashboard and a live public URL. Each topic adds a layer to the same app, so nothing you build is thrown away.

You will build it the way software is increasingly built: by prompting an AI coding agent, then reading, understanding and correcting what it produces. That second half is the whole point. An agent will hand you working React in seconds. It will also hand you a `key={index}` that corrupts your list on delete, an effect that leaks a timer, and a database call that leaves the UI spinning forever on a typo. If you cannot read the code, you cannot ship it.

Note: Use this guide alongside the course slides and the lab folders in `labs/` of the course repository. Every lab folder is a checkpoint containing a README and a drop-in `src/` snapshot: if an AI edit breaks your app beyond repair, copy that lab's `src/` over your own and carry on.

The Vibe Coding Loop

Every lab in this course follows the same five steps. Learn the loop, not just the syntax.

- **Prompt** — Name the file, the exports, the props and the traps you already know about. A prompt that says 'handle the error branch and clear loading in a finally' gets you code that does.
- **Generate** — Let the agent write it. Speed is the point — this step should be seconds, not minutes.
- **Read** — Audit the output against a checklist before you run it. Working is not the same as correct.
- **Understand** — Learn the React concept underneath. This is the part that transfers to the next project.
- **Correct** — Fix what the agent got wrong. Commit first, so git diff shows you exactly what changed.

The agent is a very fast junior developer with an excellent memory and no judgement. You supply the judgement.

Course Learning Outcomes

- LO1: Build a React web app using vibe coding — scaffold a Vite + React project, prompt an AI coding agent, and read, understand and correct the code it generates.

- LO2: Deploy a React web app to the cloud — version the project with Git and GitHub, produce a production build, and publish it to Vercel or GitHub Pages.
- LO3: Apply core React concepts — components, JSX and how Babel compiles it, the real DOM versus the Virtual DOM, props, composition, lists and keys, conditional rendering, events and controlled forms.
- LO4: Apply React Hooks — useState, useEffect with cleanup, useRef for direct DOM access, useContext, useReducer, and custom hooks that share logic.
- LO5: Integrate a backend API and database — build serverless API routes over Neon Postgres, fetch them from React with loading and error states, and add password hashing and JWT authentication.
- LO6: Implement real application navigation with React Router — routes, nested layouts, dynamic parameters, URL state and protected routes — and deploy the full-stack app.

Before You Start — Environment Setup

What you need

- Node.js 20 or newer (22 recommended) — check with `node --version`.
- A code editor; VS Code is recommended.
- Git, and a GitHub account.
- An AI coding agent: Claude Code, Cursor or GitHub Copilot.
- A free Neon account (the Postgres database and authentication, from Topic 5).
- A free Vercel account (deployment, from Topic 2).

Scaffold the project

```
npm create vite@latest cookbake -- --template react
cd cookbake
npm install
npm install react-router-dom
npm run dev
```

Set up the backend (needed from Topic 5 onward)

- Create a free Neon project and copy its connection string (DATABASE_URL).
- Run neon/schema.sql once against that database. It creates the users, courses, enrolments and reviews tables and seeds the 20-course catalogue.
- The database is reached only from the serverless functions in api/ — never from the browser. Install the API-side packages: `npm install @neondatabase/serverless bcryptjs jsonwebtoken`.
- Copy .env.example to .env.local and set DATABASE_URL and JWT_SECRET. Neither is prefixed VITE_.

- For local API development, run the functions with ``vercel dev`` (or Vite's proxy to them); in production, set the same two variables as encrypted Vercel environment variables.

The one security rule

The connection string (`DATABASE_URL`) and the signing secret (`JWT_SECRET`) live only on the server — inside the `api/` functions — and must never appear in client code or in a `VITE_`-prefixed variable, because everything prefixed `VITE_` is compiled into the JavaScript bundle that anyone can download and read. The browser only ever talks to your own `/api/*` routes. A user's identity comes from the signed JWT the API verifies, never from the request body, and ownership of a row is enforced in the SQL itself.

Conventions used in every lab

- Shared files live only in the app: `src/index.css` (design tokens), `src/data/courses.js` (the 20-course catalogue used in Topics 1-4) and `src/lib/api.js` (the fetch wrapper that calls `/api`, created in Topic 5).
- The backend lives in `api/` — serverless functions that own the database connection.
- Each lab folder holds a `README.md` and, where code changes, a drop-in `src/` snapshot.
- Restore a checkpoint with: `cp -R labs/<topic>/<lab>/src/ .` `cookbake/src/`
- Commit before you prompt. Inspect with ``git diff``. Undo with ``git restore .``

How React Really Renders: the DOM, the Virtual DOM and Babel

This course assumes you already write modern JavaScript — arrow functions, destructuring, spread, map and filter, template literals and `async/await`. What makes React click is one layer down: what the DOM actually is, why updating it by hand hurts, how React's Virtual DOM avoids that, and how Babel turns the JSX you write into the plain JavaScript objects React works with.

The DOM is the browser's object tree

Think of it like... The DOM is not a picture of the page — it IS the page, a live model you can reach in and edit.

- When the browser parses HTML it builds the DOM: a live tree of objects, one per element.
- The page you see on screen is a picture of that tree. Change the tree and the screen changes.
- JavaScript can reach into it: `document.getElementById` returns the real object for one node.
- Every element node has properties you can write — `textContent`, `className`, `style`, `value`.

- React does not replace the DOM. It manages it for you, so you stop writing the code below.

```
// The DOM is a real, live object you can hold:

const el = document.getElementById('count')

el.textContent = '19'           // the screen updates
el.className = 'badge is-on'   // so does this
```

Note: The DOM is the browser's live object tree — the page itself, not a copy.

Updating the page without React

Think of it like... Manual DOM work is giving turn-by-turn directions from memory — miss one turn and nobody notices until you are lost.

- Without React you must say HOW to update: find the node, then overwrite each property by hand.
- You write the change twice — once in the markup, once in the patch code — and they drift apart.
- The data now lives in a DOM string. Reading '19' back out and adding to it is your problem.
- Add a filter, a search box and a shortlist and every one of them must patch every affected node.
- Twenty course cards, four fields each: eighty little patches you have to remember to keep in step.

```
// Vanilla JS: filter the catalogue by hand.

function showBakery() {
  const cards = document.querySelectorAll('.card')
  cards.forEach((card) => {
    const isBakery = card.dataset.category === 'Bakery'
    card.style.display = isBakery ? 'block' : 'none'
  })
  document.getElementById('count').textContent = '10 courses'
```

```
// ...and now update the chip, the heading, the empty
// state, the URL... every one, every time, by hand.
}
```

Note: Hand-patching the DOM means saying HOW, everywhere, forever — and it does not scale.

The Virtual DOM is React's plan

Think of it like... The Virtual DOM is an architect's drawing — React compares the new plan to the old before knocking down a single real wall.

- The Virtual DOM is a lightweight tree of plain JavaScript objects describing what the UI should be.
- It is cheap: making one is just allocating objects. Touching the REAL DOM is what costs.
- On every render React builds a fresh virtual tree for the current state.
- Reconciliation is the diff: it compares the new tree with the previous one, node by node.
- It then patches ONLY the real DOM nodes that actually changed, and leaves the rest alone.

```
// State changes: 20 courses -> 10 (Bakery only).
// React builds a NEW virtual tree, diffs it, and
// concludes: remove 10 <article> nodes, and set one
// text node from '20 courses' to '10 courses'.
//
// It does NOT rebuild the navbar, the hero, the
// footer or the surviving cards. They did not change.
```

Note: React diffs a cheap JS tree, then patches only the real nodes that changed.

Real DOM vs Virtual DOM, side by side

Think of it like... Imperative is reciting the recipe step by step; declarative is ordering the dish and letting the kitchen work out the steps.

- Imperative (no React): you locate nodes and mutate them. You own every step of HOW.
- Declarative (React): you describe WHAT the UI is for the current state. React works out the steps.
- The React version has no `getElementById`, no `textContent` and no `className` assignment anywhere.
- Because the UI is recomputed from state every time, it can never silently drift out of sync.
- Your job shrinks to one thing: keep the state correct, and describe the UI as a function of it.

// WITHOUT React – you patch the real DOM yourself:

```
const el = document.getElementById('seats')
el.textContent = String(seats)
el.className = seats > 0 ? 'badge' : 'badge is-full'
```

// WITH React – you describe the result, once:

```
<span className={seats > 0 ? 'badge' : 'badge is-full'}>
  {seats}
</span>
```

Note: Stop describing HOW to change the page; describe WHAT it should be.

Babel: the browser never sees JSX

Think of it like... Babel is the translator in the booth — you speak JSX, the browser only ever hears JavaScript.

- JSX is not valid JavaScript. Paste it into a browser console and you get a syntax error.
- Babel is a compiler that rewrites your JSX into ordinary function calls before it ships.
- Vite runs Babel for you through `@vitejs/plugin-react` — it is why the plugin is in `vite.config.js`.
- `<CourseCard fee={680}/>` becomes `React.createElement(CourseCard, { fee: 680 })`.
- That call is not markup and it is not a DOM node. It returns a plain JavaScript OBJECT.

```
// YOU WRITE (JSX):
```

```
<CourseCard course={course} fee={680} />
```

```
// BABEL COMPILES IT TO (plain JavaScript):
```

```
React.createElement(CourseCard, { course: course, fee: 680 })
```

```
// WHICH RETURNS (a plain object – this is a React element):
```

```
{ type: CourseCard, props: { course: {...}, fee: 680 } }
```

Note: JSX is sugar. Babel turns every tag into a call that returns a plain object.

Why that one fact explains everything

Think of it like... Once you know JSX is JavaScript in a costume, its 'weird' rules stop being rules and become consequences.

- You write `className`, not `class`, because `class` is a reserved word in JavaScript.
- Attributes are camelCase (`onClick`, `htmlFor`) because they are really keys in a props object.
- `{ }` is a window for an EXPRESSION — a value. An `if` or a `for` produces no value, so it cannot fit.
- One root element per return, because a JavaScript function returns exactly one value.
- Capitalised names, because `createElement('div')` means a tag but `createElement(CourseCard)` means YOUR function.

```
<h3 className="card__title">{course.title}</h3>
```

```
// -> createElement('h3', {className:'card__title'}, ...)
```

```
// lowercase string 'h3' = a real HTML tag
```

```
<CourseCard course={course} />
```

```
// -> createElement(CourseCard, {course})
```

```
// capitalised IDENTIFIER = your function
```

Note: Every JSX rule falls out of one fact: it compiles to createElement calls.

Topic 01 — Build a React Web App Using Vibe Coding

Vite · JSX · Components · Props · The Prompt → Read → Correct loop

Vibe coding means building software by prompting an AI agent — but the skill this course teaches is not prompting. It is reading. An agent will hand you working React in seconds, and it will just as cheerfully hand you code that is subtly, expensively wrong. Every lab in this course therefore runs the same five-step loop: Prompt, Generate, Read, Understand, Correct.

In this topic you scaffold the project, generate your first component, and vibe-code a whole landing page. Then you stop and audit what the agent produced. It works — and it contains two deliberate design flaws that Topic 3 will teach you to repair. Recognising them is the point.

Key Concepts — Topic 01

- **Vibe coding is a loop, not a shortcut** — Prompt → Generate → Read → Understand → Correct. The agent writes; you supply the judgement.
- **A component is just a function** — It takes props (its arguments) and returns JSX describing what the UI should look like.
- **JSX is not HTML** — It compiles to function calls returning plain JavaScript objects. Hence className, camelCase, one root element, and {} for expressions.
- **Props flow one way** — Parent to child, and they are read-only. A child never writes to a prop.
- **The entry point** — index.html holds <div id="root">; main.jsx calls createRoot(...).render(<App/>) to mount React into it.
- **Never ship code you cannot read** — AI output that works is not the same as AI output that is correct. Audit it before you run it.

Concepts Explained — Topic 01

Vibe coding is a loop, not magic

Think of it like... Vibe coding is flying with autopilot on — you still watch the instruments and take the controls the moment something drifts.

- The loop is Prompt, Generate, Read, Understand, Correct — the agent writes, you supply the judgement.
- An AI agent produces working React in seconds, but 'working' and 'correct' are not the same thing.
- The value you add is reading the output against a checklist and knowing the concept underneath it.
- Every lab in this course repeats the same five steps so the habit becomes automatic.
- By the end you should build fast and be able to say exactly why every line is there.

```
// The five-step loop, every lab:
//   1 Prompt   2 Generate   3 Read
//   4 Understand       5 Correct
//
// The agent writes. You supply the judgement.
```

Note: Speed comes from the agent; correctness comes from you reading it.

A component is just a function

Think of it like... A component is a cookie cutter: define the shape once, stamp out as many cookies as you like.

- A React component is a JavaScript function that returns JSX describing a piece of the UI.
- Its name must start with a capital letter, or Babel compiles it to a literal HTML tag instead.
- You use it by writing it like a tag: `<CourseCard />` calls the function and renders what it returns.
- Because it is a function, you reuse it by calling it many times, each with different props.
- This course uses function components and hooks only — never class components.

```
function CourseCard() {
  return <h3>Artisan Sourdough Bread Baking</h3>
}
```

```
<CourseCard />
```

Note: Capitalised function in, JSX out — that is the whole idea.

What a component really is

Think of it like... A component is a vending machine: press the same buttons (props) and the same snack (UI) always drops out.

- A component is a plain function: it takes props in and returns one JSX tree describing the UI.
- It must return a single root — one tree — because a JavaScript function returns exactly one value.
- A component should be pure: given the same props, it must always return the same output.
- Purity means no surprises in render — no fetching, timers or mutation; those belong in effects.
- Purity is what lets React render it as often as it likes, and trust the result every time.

```
function CourseCard({ title }) {
  return <h3>{title}</h3> // one tree, no side effects
}
```

```
// same title in -> same markup out, every time
```

Note: Same props in, same tree out — a capitalised, pure function.

JSX is a blueprint, not HTML

Think of it like... JSX is a blueprint, not the building — React reads it and constructs the real DOM from the instructions.

- JSX looks like HTML but is not — Babel compiles it to createElement calls that return objects.
- Those objects describe what the UI should look like; React turns them into real DOM nodes for you.
- Because it is really JavaScript, HTML's class becomes className and attributes become camelCase.
- Understanding that JSX is code, not markup, explains every rule that follows in this topic.
- You never call createElement by hand — Vite runs Babel — but knowing it is there demystifies JSX.

```
const el = <h1 className="brand">Cook & Bake</h1>
```

```
// Babel compiles it to:
```

```
React.createElement('h1', { className: 'brand' },  
    'Cook & Bake')
```

Note: JSX compiles to createElement calls returning plain JS objects, not HTML.

One root element and Fragments

Think of it like... One root is a single moving box React hands back — not an armful of loose items; a Fragment is a box with no cardboard.

- A component must return a single root element, because a function returns exactly one value.
- Wrapping siblings in an extra <div> works but litters the page with meaningless containers.
- A Fragment (<>...</>) groups siblings under one root while adding no node to the real DOM.
- Reach for a Fragment whenever a wrapper div would only exist to satisfy this one-root rule.
- Returning two adjacent tags with no wrapper is a syntax error the agent will sometimes produce.

```
// RootLayout.jsx – the real app shell  
  
return (  
    <>  
        <Navbar />  
        <main><Outlet /></main>  
        <Footer />  
    </>  
    // one root; <> adds no extra div  
)
```

Note: Return one root; use a Fragment to group without an extra div.

Curly braces embed expressions

Think of it like... Curly braces are a window cut into your markup — JavaScript shows through, but only values, never statements.

- Inside JSX, {} drops the result of a JavaScript expression into the output.

- An expression produces a value: a variable, a call, a ternary, a template literal all qualify.
- A statement such as if, for or a variable declaration does not — it produces no value and errors.
- This is why you use a ternary or && inside JSX rather than an if statement.
- The braces run the code every render, so what you see always reflects the current data.

```
<h1>Welcome to {academy}</h1>           // expression: ok
<span>$${course.fee}</span>               // expression: ok
<span>{isBakery ? '🍰 Bakery' : '👨🍳 Cooking'}</span> // ok

<h1>{if (x) {}}</h1>                     // statement: error
```

Note: {} takes expressions (values), never statements like if or for.

Props are a component's arguments

Think of it like... Props are a recipe card handed to a component: it reads the ingredients and cooks, but never edits the card.

- A prop is a value passed into a component, exactly as you pass an argument to a function.
- You read props by destructuring them in the parameter list: function CourseCard({ course }).
- Give a default with = so a missing prop has a sensible value: ({ fee = 0 }).
- Props flow one way, parent to child, and they are read-only — a child must never reassign a prop.
- String props use quotes; any other type uses braces: title="Macaron Masterclass" fee={420}.

```
function CourseCard({ title, fee = 0 }) {
  return <h3>{title} – $${fee}</h3>
}
```

```
<CourseCard title="Macaron Masterclass" fee={420} />
```

Note: Props are read-only arguments that flow down from parent to child.

One object prop beats nine loose ones

Think of it like... Passing one course object is handing over the whole recipe card, not reading out nine ingredients down the phone.

- The real CourseCard takes ONE prop — course — and destructures the fields it needs inside.
- That object's shape is the database row you will fetch in Topic 5, so nothing downstream changes.
- Nine separate props means nine things to keep in step at every call site. One object means one.
- Destructure inside the body when the object is the prop: `const { slug, title, fee } = course`.
- Collect leftover props with `...rest` and spread them onward: `<article {...rest}>`.

```
// src/components/CourseCard.jsx (real code)

export default function CourseCard({ course }) {

  const { slug, title, category, level, fee } = course

  return <Link to={` /courses/${slug}`}>{title}</Link>

}
```

```
<CourseCard course={course} />
```

Note: Pass the whole object; its shape is the DB row, so the card never changes.

The entry point: #root and main.jsx

Think of it like... index.html is an empty stage with one marked spot; main.jsx is the stagehand who mounts the whole React play onto it.

- index.html is almost empty — its job is to provide one `<div id="root"></div>`.
- main.jsx is the entry file: it imports App and mounts it into that root div.
- `createRoot(...).render(<App/>)` is the single call that hands the whole app over to React.
- This is the ONE place React touches the real DOM by hand. Everything below it is declarative.

- Knowing this chain explains where the app 'starts' when you read AI-scaffolded code.

```
// index.html

<div id="root"></div>

// src/main.jsx (real code)

createRoot(document.getElementById('root')).render(
  <StrictMode><App /></StrictMode>,
)
```

Note: main.jsx mounts <App/> into the single #root div in index.html.

Never ship code you cannot read

Think of it like... AI code that runs is a contract you have not read — it may work today and cost you everything on the clause you skipped.

- Code that compiles and renders can still hide bugs the agent has no way to notice.
- Run every generated file past a short checklist before you trust it in your project.
- Common landmines: `key={index}`, a `useEffect` with no cleanup, a `fetch` with no error branch.
- Also check that no secret has been placed in a `VITE_` variable, where it becomes public.
- If you cannot explain why a line is there, you are not ready to ship it — that is the whole skill.

```
// Audit checklist before you run AI code:

//   - key={index} on a list?

//   - useEffect missing a cleanup return?

//   - fetch with no error state?

//   - a secret sitting in a VITE_ variable?

//   - SQL built by string concatenation?
```

Note: Audit generated code against a checklist before you trust it.

Deep Dive — What vibe coding actually is

Vibe coding is a way of building software where you describe what you want in plain language and an AI coding agent writes the code. It is genuinely fast, and it is genuinely the direction the industry is moving. But the word 'vibe' hides a trap: it makes the process sound like you can switch your brain off and let the machine drive. You cannot. The right mental model is an aircraft on autopilot. The autopilot flies the plane and does the tedious work, but the pilot never stops watching the instruments, and the instant the aircraft drifts, the pilot takes the controls. In vibe coding you are the pilot, and the code is the aircraft.

That is why this course frames every task as a five-step loop: Prompt, Generate, Read, Understand, Correct. You write a clear prompt. The agent generates code. Then comes the half the course actually teaches: you read the output line by line, you understand the React concept underneath each line, and you correct whatever is wrong before it ever runs in your project. The agent supplies speed; you supply judgement. Remove your judgement from the loop and you are not a fast developer, you are a fast producer of bugs you cannot explain.

This matters because an AI agent will hand you code that runs but is subtly wrong far more often than code that obviously breaks. It will give you a list keyed by array index that corrupts on delete, an effect with no cleanup that leaks a timer, a fetch with no error branch that spins forever on a typo, or an `/api/` route that trusts a user id out of the request body. None of these throw an error at you. Every one of them is a bug you can only catch by reading. The goal of Topic 1 is to make you someone who can build the Cook & Bake Academy site in minutes and say exactly why every single line of it is there.

- Prompt: describe the outcome you want, with enough context to be unambiguous.
- Generate: let the agent produce the code.
- Read: go through it line by line, not just glance at the result in the browser.
- Understand: name the React concept behind each part.
- Correct: fix the subtle bugs before the code enters your project.

Deep Dive — Why JSX is not HTML — and what Babel does to it

The single most useful thing you can internalise in Topic 1 is that JSX is not HTML — it is JavaScript wearing an HTML-shaped costume. The browser has never seen JSX and cannot parse it; paste `<CourseCard fee={680}/>` into a console and you get a syntax error. Something has to translate it into real JavaScript before it ships, and that something is Babel, a compiler. Vite wires Babel in for you through the `@vitejs/plugin-react` plugin listed in `vite.config.js`. When Babel meets `<CourseCard fee={680}/>`, it rewrites it into a plain function call, `React.createElement(CourseCard, { fee: 680 })`, and that call returns an ordinary JavaScript object — roughly `{ type: CourseCard, props: { fee: 680 } }`. That object is not markup and it is not a DOM node; it is a lightweight

description React can hold in memory. A whole tree of those objects is what we later call the Virtual DOM. You are not writing markup the browser parses; you are writing a blueprint that Babel turns into function calls and React reads to construct the building.

Once you see JSX as code that compiles to createElement calls, every rule that trips people up stops being arbitrary. You write className instead of class because class is a reserved word in JavaScript. Attributes are camelCase (onClick, htmlFor) because they are really keys in the props object of that createElement call. You wrap expressions in curly braces because {} is the window through which live JavaScript values flow into the output — and only expressions, things that produce a value, fit through that window; an if statement or a for loop produces no value and so is a syntax error inside JSX. And a component must return a single root element because a function returns exactly one thing; when you do not want an extra wrapper div, a Fragment (<>...</>) groups the siblings under one root while adding nothing to the real DOM.

This is also why component names must be capitalised. Babel compiles a lowercase tag like <div> to createElement('div', ...) — a string, meaning a built-in HTML element — but a capitalised tag like <CourseCard> to createElement(CourseCard, ...) — an identifier, meaning your function. The capital letter is the single signal the compiler uses to tell your components apart from the browser's own elements. None of this is memorisation for its own sake; every rule falls straight out of the one fact that JSX compiles to createElement calls.

- The browser never sees JSX; Babel (run by Vite via @vitejs/plugin-react) compiles it away.
- <CourseCard fee={680}/> becomes React.createElement(CourseCard, {fee:680}), which returns a plain object.
- class becomes className, attributes are camelCase, because they are keys in a props object.
- {} embeds expressions (values) only; one root per return; capitalised names mean YOUR function.

```
// YOU WRITE (JSX):
```

```
<CourseCard course={course} fee={680} />
```

```
// BABEL COMPILES IT TO (plain JavaScript):
```

```
React.createElement(CourseCard, { course: course, fee: 680 })
```

```
// WHICH RETURNS (a plain object – a React element):
```

```
{ type: CourseCard, props: { course: {...}, fee: 680 } }
```

Deep Dive — Components and props: functions and their arguments

A React component is nothing more exotic than a JavaScript function that returns JSX. That is the entire definition. If you can write a function, you can write a component. The power comes from the fact that, like any function, a component can take inputs — and in React those inputs are called props. Think of a component as a cookie cutter: you define the shape once, then stamp out as many cookies as you like, and props are what let each cookie come out different. One CourseCard definition renders all twenty Cook & Bake courses, each fed a different course object.

Props are best understood as a recipe card handed to a chef. The chef reads the card, cooks exactly what it says, and hands back the dish — but the chef never scribbles on the card. That is the one rule that matters most: props flow one way, from parent down to child, and they are read-only. A child component must never reassign a prop it was given. Data flows down. If a child needs to change something, the parent passes down a function for the child to call — the way HomePage passes Hero an onBrowse function so the hero's button can trigger a scroll it knows nothing about — but the value itself stays owned by whoever passed it. This one-way flow is what makes a React app predictable: you can always trace where a value came from by walking up the tree.

In practice you read props by destructuring them straight out of the parameter list. The real CourseCard takes a single course prop — function CourseCard({ course }) — and then pulls the fields it needs out of that object with `const { slug, title, category, fee } = course`. One object prop beats nine loose ones, and there is a deeper payoff: the shape of that object is exactly the database row you will fetch in Topic 5, so when the data stops being a hard-coded array and starts arriving over the network, not a single line of CourseCard has to change. You give a prop a default with an equals sign, (`{ fee = 0 }`), so a missing prop degrades gracefully instead of rendering undefined, and you pass props at the call site like attributes: strings in quotes, everything else in braces, so `title="Macaron Masterclass" fee={420}`. When you read AI-generated components, the first thing to check is that the props flowing in match the props being read out.

- A component is a capitalised function that returns JSX.
- Props are its arguments, read by destructuring: `function CourseCard({ course })`.
- Props are read-only and flow one way, parent to child.
- Pass one object prop whose shape is the DB row, so nothing downstream changes in Topic 5.

```
// src/components/CourseCard.jsx (real code)

export default function CourseCard({ course }) {

  const { slug, title, category, fee } = course

  return <Link to={`/${category}/${slug}`}>{title} – S${fee}</Link>

}
```

`<CourseCard course={course} />`

Deep Dive — A component is a pure function

You already know a component is a JavaScript function that returns JSX, but there is a deeper property that makes React work at all, and it is worth stating plainly: a component should be a pure function of its props. Purity has a precise meaning here. Given the same props, a component must always return the same tree, and while it renders it must not change anything outside itself — no writing to variables declared elsewhere, no fetching, no timers, no direct DOM edits. It simply takes props in and returns a description of UI out, like a vending machine that drops the same snack every time the same button is pressed. The name is capitalised so React can tell your component apart from a built-in HTML tag, and it returns a single root element because a function returns exactly one value.

Why does React demand this? Because purity is what lets React call your component whenever it likes, as often as it likes, and trust the result. React may render a component to build the Virtual DOM and then throw that render away; it may re-render it many times as state changes; in development StrictMode deliberately renders it twice to flush out impurity. If rendering had side effects — incrementing a counter, firing off a request — those would happen unpredictably and multiply. So React draws a hard line: rendering is for computing UI, and everything else, the side effects, happens elsewhere. Work triggered by the user goes in event handlers; work that must synchronise with an external system goes in `useEffect` (Topic 4). Keep render pure and your component becomes something React — and you — can reason about; break purity and you get the class of bugs that appear only sometimes, which are the hardest of all to find.

Purity is also what makes composition safe. React builds complex interfaces by nesting small, pure components, and the mechanism that keeps this flexible is the `children` prop: whatever JSX you place between a component's opening and closing tags arrives inside it as `props.children`. The real `Section` component is exactly this — a picture frame that renders `{children}` inside a centred container and does not care whether you slid a `CourseGrid`, a form, or a paragraph into it. The design rule that follows is to favour composition over configuration: rather than piling boolean props onto one component to cover every possible variation, let it accept children and stay generic. A handful of small pure components you can slot together will take you much further than one enormous component with thirty configuration props.

- A component should be pure: same props in, same tree out, with no side effects during render.
- Purity lets React render freely — repeatedly, speculatively, twice in StrictMode — and trust the output.
- Side effects belong in event handlers and `useEffect`, never in the body of a render.
- Compose small pure components; the `children` prop lets a wrapper like `Section` hold any content.

```
// src/components/Section.jsx (real code)

function Section({ title, children }) {
  return (
    <section className='section'>
      {title} && <h2>{title}</h2>
      {children}
    </section>
  )
}
```

```
<Section title='Popular courses'><CourseGrid courses={popular} /></Section>
```

Lab 1.1 — Set Up Your Vibe Coding Workspace

Objective: scaffold the Cook & Bake Academy project and run the vibe-coding loop once.

Goal: The client wants a website for their cooking & bakery school. The learner installs Node, VS Code and an AI coding agent, scaffolds the cookbake Vite + React app, traces how index.html, main.jsx and App.jsx put React on the page, and runs the Prompt to Generate to Read to Understand to Correct loop for the first time.

What you'll build

A running Vite + React dev server serving your own cookbake app at <http://localhost:5173> (Tech & files: Node 22, Vite, React 19, VS Code, Claude Code / Cursor / Copilot.)

Step-by-step

1. Confirm Node.js 20 or newer is installed

```
node --version && npm --version
```

2. Scaffold the project. The folder is cookbake — this is the app you build for the next two days

```
npm create vite@latest cookbake -- --template react
```

3. Install the dependencies and start the dev server

```
cd cookbake && npm install && npm run dev
```

4. Open `http://localhost:5173`, edit the `<h1>` in `src/App.jsx` and watch it hot-reload without a refresh
5. Trace the entry point by hand: `index.html` holds an empty `<div id="root">`, and `main.jsx` is the one place React ever touches the real DOM

```
createRoot(document.getElementById('root')).render(<App />)
```

6. Launch your AI agent in the project folder and give it a READING task first — you must be able to audit it before you trust it

Prompt: "Read `index.html`, `src/main.jsx` and `src/App.jsx` in this Vite project. Explain in plain English how a `<div id=\"root\">` in the HTML ends up showing the App component. Do not change any code."

7. Run the loop once end to end: Prompt, Generate, Read, Understand, Correct — replace the Vite boilerplate in `App.jsx` with a Cook & Bake Academy heading

Prompt: "Replace the contents of `src/App.jsx` with a single `<h1>` reading 'Cook & Bake Academy'. Delete the Vite logo, the counter and the unused imports. Keep it a default-exported function component."

Test it

The dev server runs at `http://localhost:5173` showing your Cook & Bake Academy heading, edits hot-reload instantly, and you can explain out loud what `index.html`, `main.jsx` and `App.jsx` each do.

Watch out for

If ``npm create vite`` fails, check ``node --version`` is 20 or newer. Vite will not run on Node 18.

Lab 1.2 — Your First Vibe-Coded Component: CourseCard

Objective: write a React component that receives data through props.

Goal: The catalogue is 20 bakery and cooking courses, so the card that shows one course is the component the whole site is built from. The learner prompts the agent for `CourseCard`, then reads the output line by line: a component is a function, JSX is not HTML, and props are read-only arguments flowing one way from parent to child.

What you'll build

A reusable `<CourseCard />` showing one course's emoji, title, level, fee, duration and campus (Tech & files: React function components, JSX, props, destructuring, `src/components/CourseCard.jsx`.)

Step-by-step

8. Create the folder that will hold every component you write

```
mkdir -p src/components
```

9. Prompt the agent for the card. Be specific about the props — vague prompts get invented APIs

Prompt: "Create `src/components/CourseCard.jsx`. A React function component named `CourseCard` taking flat props: `emoji`, `title`, `level`, `fee` (a number), `weeks` (a number), `campus`. Render the `emoji` large, then the `title`, a level chip, `'$' + fee`, the `weeks` and the `campus`. Use `className`, not `class`. Default-export it. No state, no hooks."

10. Read the output before you run it. Is it a function? Does it destructure its props in the signature?

```
export default function CourseCard({ emoji, title, level, fee, weeks, campus }) { ... }
```

11. Check the JSX rules the agent had to obey: `className` not `class`, camelCase attributes, one root element, and `{}` around every JavaScript expression

```
<span className="card__price">${fee}</span>
```

12. Render one card from `App.jsx` with literal props — this is a real course from the catalogue, BAK-101

```
<CourseCard emoji="🍞" title="Artisan Sourdough Bread Baking" level="Beginner" fee={680} weeks={4} campus="Bakehouse" />
```

13. Note the two kinds of braces: `fee={680}` passes the NUMBER 680, `fee="680"` would pass a string. Change one and see the difference in ``${fee}``

14. Correct the AI: try to reassign a prop inside `CourseCard` (`title = 'Hacked'`) and read the error. Props are the component's arguments and they are read-only — the child never writes to them

Test it

One course card renders the sourdough course with its emoji, title, level, \$680 fee, 4 weeks and campus; changing a prop in `App.jsx` changes what the card shows; and you can say why a component may never assign to its own props.

Watch out for

JSX is not HTML: it is ``className``, not ``class``, and a component must return a single root element (use a ``<>...</>`` Fragment if you need two).

Lab 1.3 — Vibe-Code the Cook & Bake Academy Landing Page

Objective: compose a page from components and audit the code the AI produced.

Goal: The learner vibe-codes the whole landing page — navbar, hero, a six-card course grid and a footer — from the client's mockup, then stops and audits it. The page works, but the AI left two smells behind: every component crammed into one file, and six copy-pasted cards. Naming them is the point of the lab.

What you'll build

A full Cook & Bake Academy landing page: navbar, hero, six course cards and a footer (Tech & files: JSX composition, props, src/App.jsx, src/components/CourseCard.jsx, src/index.css.)

Step-by-step

15. Give the agent the brand brief, then the page. Feeding it the mockup's look is what makes the output usable instead of generic

Prompt: "In src/index.css define the Cook & Bake Academy design tokens: a warm bakery palette (cream background, deep brown text, amber accent), Playfair Display for headings and Inter for body text. Then in src/App.jsx build a landing page with: a sticky navbar (🍪 Cook & Bake Academy brand, Courses / About links, Sign in button), a hero with the headline 'Master the art of cooking & baking' and a Browse courses button, a section titled 'Popular courses' holding a responsive grid of six <CourseCard/> elements, and a footer. Reuse the existing CourseCard component – do not rewrite it."

16. Run it. It works — now stop. Working is not the same as correct, and you are the one who has to maintain this

```
npm run dev
```

17. Fill the six cards with real courses from the client's catalogue, not lorem ipsum. Two are shown here; add BAK-104 Macaron Masterclass, CUL-201 Italian Cuisine Mastery, CUL-203 Japanese Sushi & Sashimi and CUL-210 Knife Skills & Kitchen Essentials

```
<CourseCard emoji="🥐" title="French Pastry & Viennoiserie"
level="Intermediate" fee={1480} weeks={8} campus="Bakehouse" />
```

18. Audit — smell 1: Navbar, Hero and Footer are all defined inside App.jsx. One file, four components, no reuse
19. Audit — smell 2: the grid hardcodes six <CourseCard> elements with copy-pasted props. The client has 20 courses and adds more every term
20. Write both smells down. Lab 3.2 fixes the first by extracting real component files; Lab 3.3 fixes the second with .map() over the data
21. Correct the AI once yourself: ask it to make the hero button scroll to the course grid, then read what it wrote before accepting

Prompt: "Add an `onClick` to the hero's 'Browse courses' button that smooth-scrolls to the courses section. Explain your approach in a comment before you write it."

Test it

The landing page renders the Cook & Bake navbar, hero, six real course cards and a footer; the grid is responsive; and you can name the two design flaws the AI left behind and say which later lab repairs each one.

Watch out for

The AI's code works, but it is one giant file with six copy-pasted cards. Do not 'fix' it yet — Labs 3.2 and 3.3 exist precisely to repair those two smells.

Topic 02 — Deploy Your React Web App to the Cloud

Git & GitHub · Production builds · Vercel · GitHub Actions · Env vars

Code that only runs on your laptop is not software yet. This topic puts your app on the internet, and it introduces the habit that makes fast AI editing safe: commit before you prompt. Git is what lets you read exactly what the agent changed (`git diff`) and throw it away when it is wrong (`git restore`).

You will also meet the security rule that governs the rest of the course. Vite compiles every `VITE_`-prefixed environment variable straight into the JavaScript bundle the browser downloads. A `VITE_` variable is therefore public by definition, and can never hold a secret.

Key Concepts — Topic 02

- **Git is the vibe coder's safety net** — Commit before you prompt. `git diff` shows what the agent changed; `git restore` undoes it.
- **React compiles to static files** — `npm run build` produces `dist/` — plain HTML, CSS and JS that any static host can serve.
- **Never commit secrets** — `node_modules`, `dist` and `.env.local` stay out of Git. A leaked credential in history is permanent.
- **VITE_ variables are public** — Anything prefixed `VITE_` is baked into the browser bundle at build time. Change one and you must rebuild.
- **Deploy on every push** — Vercel rebuilds from GitHub automatically and gives every branch its own preview URL.
- **SPA deep links need a rewrite** — A static host looks for `/courses` on disk. Rewrite every path to `/index.html` or refreshing a route 404s.

Concepts Explained — Topic 02

Git is the vibe coder's safety net

Think of it like... Git is a video-game save point: commit before you prompt, and any bad edit is one 'load' away from undone.

- Commit before you ask the agent to change anything, so you always have a clean point to return to.
- `git diff` shows exactly which lines the agent touched, line by line, before you accept them.
- `git restore .` throws away the working changes and puts you back at your last commit.
- Small, frequent commits make it obvious which prompt introduced which change.
- Without this habit, one confident-but-wrong AI edit can quietly break code you cannot recover.

```
git add -A && git commit -m 'before AI edit'
```

```
# prompt the agent, then review:
```

```
git diff          # what did it change?
```

```
git restore .     # undo if it went wrong
```

Note: Commit before you prompt; diff to review; restore to undo.

React compiles to static files

Think of it like... `npm run build` is packing for a trip — your sprawling workshop becomes one sealed suitcase of plain files any host can carry.

- `npm run build` bundles your source into a `dist/` folder of plain HTML, CSS and JavaScript.
- Babel has already compiled every piece of JSX away — the shipped bundle contains no JSX at all.
- Those files are static: no Node server runs them, so any file host on earth can serve them.
- `npm run preview` serves the built `dist/` locally so you can test the real production output.
- The API routes in `api/` are NOT part of this bundle — Vercel deploys them as separate functions.

```
npm run build     # -> dist/ (html, css, js – no JSX)
```

```
npm run preview   # serve dist/ locally to test
```

```
# dist/assets/index-*.js is what every visitor
# downloads. Open it. Anything in there is public.
```

Note: A React build is a folder of static files any host can serve.

Never commit secrets

Think of it like... A secret in Git history is a postcard, not a letter — once written on the back, everyone down the line can read it forever.

- A .gitignore lists paths Git must never track, keeping junk and secrets out of your history.
- Always ignore node_modules (huge, reinstallable), dist (rebuildable), and .env.local (secret).
- A credential committed even once lives in the history forever, even after you delete the line.
- If a secret is ever pushed, treat it as leaked: rotate it immediately, do not just remove it.
- Commit .env.example instead — the variable NAMES, with placeholder values and no real secrets.

```
# .gitignore
node_modules
dist
.env.local          # holds DATABASE_URL and JWT_SECRET

# .env.example    <- this one IS committed
DATABASE_URL=postgresql://USER:PASSWORD@...
JWT_SECRET=dev-secret-change-me
```

Note: node_modules, dist and .env.local never belong in Git history.

VITE_ variables are public. Always.

Think of it like... A `VITE_` variable is printed on the flyer, not whispered — baked into the bundle for anyone to read.

- Vite exposes only variables prefixed `VITE_` to browser code, via `import.meta.env.VITE_SOMETHING`.
- It SUBSTITUTES the value straight into the `.js` file it ships. It is not looked up at runtime.
- So a `VITE_` variable is visible to every visitor: open devtools, or just read `dist/assets/index-*.js`.
- A `VITE_` variable may hold a public URL. It may never hold a password, a key or a token.
- A variable with no `VITE_` prefix is never touched by Vite and never reaches the browser at all.

```
# .env.local
```

```
VITE_SITE_NAME=Cook & Bake Academy # public. fine.
```

```
DATABASE_URL=postgresql://user:PASSWORD@... # SERVER ONLY
```

```
// Browser code can only ever see the first one:
```

```
const name = import.meta.env.VITE_SITE_NAME
```

Note: `VITE_` vars are compiled into the public bundle — no secrets, ever.

Deploy on every push

Think of it like... Connecting Vercel to GitHub is a standing order to a baker — every push, a fresh loaf is baked and set out, no phone call needed.

- Connect your GitHub repo to Vercel once, and every push to main triggers a fresh build and deploy.
- This is continuous deployment: your live site always reflects the latest committed code.
- Every branch and pull request gets its own preview URL, so you can test before merging.
- Vercel also builds everything in `api/` into serverless functions, on the same domain as the site.
- Secrets like `DATABASE_URL` are set in the Vercel dashboard, encrypted — never in the repo.

```
git push origin main

# Vercel rebuilds and deploys automatically:

#   dist/    -> the static site
#   api/     -> serverless functions

# Same domain, so the browser calls /api/courses

# with no CORS and no separate backend URL.
```

Note: Push to GitHub; Vercel rebuilds the site AND the API functions.

SPA deep links need a rewrite

Think of it like... A static host is a receptionist with a filing cabinet — ask for /courses, find no such file, and you must be told to always hand back index.html.

- A single-page app has only one real file, index.html; the router fakes the other paths in the browser.
- Type /courses directly or refresh, and the static host looks for that file on disk and 404s.
- The fix is a rewrite rule: send every path to /index.html and let React Router match it.
- But you must EXCLUDE /api/*, or your API calls get rewritten to index.html and return a web page.
- That is what "Unexpected token '<'" means: you fetched a URL and res.json() got HTML back.

```
// vercel.json (real) – note the negative lookahead
{
  "rewrites": [
    { "source": "/((?!api/).*)", "destination": "/index.html" }
  ]
}

// Everything EXCEPT /api/* falls back to index.html.
```

Note: Rewrite every path to /index.html — except /api/*, or the API breaks.

Deep Dive — Git is the vibe coder's safety net

Version control matters for every developer, but for a vibe coder it is not optional — it is the safety net without which the whole approach is reckless. When you let an AI agent edit your code, you are handing the keys to something confident, fast, and occasionally very wrong. Git is the save point that lets you undo any move it makes. The discipline is simple: commit your working code before you prompt the agent to change anything. That commit is a clean state you can always return to, no matter how tangled the next few minutes get.

After the agent has worked, git diff is your review tool. It shows you, line by line, exactly what changed — which is precisely the 'Read' step of the loop made concrete. You are not trusting the agent's summary of what it did; you are looking at the actual edits. If you like them, you commit. If the agent has gone off the rails, git restore . throws the changes away and drops you back at your last commit as if the detour never happened. Small, frequent commits make this even more powerful, because each one isolates the effect of a single prompt, so when something breaks you know exactly which instruction caused it.

- Commit before every prompt so you always have a clean point to return to.
- git diff is the 'Read' step made concrete — the actual line-by-line changes.
- git restore . discards the working changes and returns you to the last commit.
- Small, frequent commits isolate which prompt caused which change.

```
git add -A && git commit -m 'before AI edit'
```

```
# prompt the agent, then review:
```

```
git diff          # what changed?
```

```
git restore .     # undo if wrong
```

Deep Dive — From build to deployment, and the secrets rule

A React app in development runs through Vite's dev server, but that is not what you ship. When you run npm run build, Vite bundles and minifies everything into a dist/ folder containing nothing but plain HTML, CSS and JavaScript — Babel has already compiled every piece of JSX away, so the shipped bundle contains no JSX at all. Those files are static — no Node process runs them — which is why deploying the front end is really just uploading a folder of files that any host on earth can serve. The api/ folder is different: Vercel builds each file there into a serverless function and serves it from the same domain, so the browser can call /api/courses with no CORS and no separate backend URL. Connect your GitHub repository to Vercel and all of this becomes automatic: every push to your main branch triggers a fresh build and deploy of both the static site and the functions, and every branch gets its own preview URL. This is continuous deployment, and it means your live site always reflects your latest committed code.

Two rules keep this safe, and both are about secrets. First, some things must never enter Git at all: `node_modules` (enormous and reinstallable), `dist` (rebuildable), and above all `.env.local` (your `DATABASE_URL` and `JWT_SECRET`). A `.gitignore` file lists these so Git never tracks them; you commit `.env.example` instead, which carries the variable names and placeholder values but no real secrets. A credential committed even once lives in the history permanently, so a leak means rotating the key, not just deleting the line. Second, understand exactly what 'public' means for environment variables, because this is the rule that returns with force in Topic 5. Vite exposes only variables prefixed `VITE_` to your browser code, and it does not look them up at runtime — it substitutes their values straight into the JavaScript bundle at build time. That means a `VITE_` variable is completely visible to anyone who opens dev tools or simply reads `dist/assets/index-*.js`. It is printed on the flyer, not whispered. So a `VITE_` variable may hold a public site name or a public URL, but it may never hold a password, a key or a token — and `DATABASE_URL`, which carries the database password, must therefore never carry the `VITE_` prefix. If you ever 'fix' a connection error by renaming `DATABASE_URL` to `VITE_DATABASE_URL`, you have just published full read-and-write access to your database to every visitor of your site.

- `npm run build` produces a static `dist/` folder; `api/` becomes serverless functions on the same domain.
- Vercel rebuilds and redeploys both on every push; each branch gets a preview URL.
- Never commit `node_modules`, `dist` or `.env.local` — a leaked secret is permanent; commit `.env.example`.
- `VITE_` variables are compiled into the browser bundle, so `DATABASE_URL` must never carry that prefix.

```
# .gitignore
```

```
node_modules
```

```
dist
```

```
.env.local          # DATABASE_URL, JWT_SECRET
```

```
# .env.example <- committed; names only, no secrets
```

```
DATABASE_URL=postgresql://USER:PASSWORD@ep-...neon.tech/db
```

```
JWT_SECRET=dev-secret-change-me
```

Deep Dive — Why refreshing a route can 404

This is the single most common deployment surprise, and it catches almost everyone once. Your single-page app has exactly one real HTML file: `index.html`. When a user clicks a `<Link>` to `/courses`, no new file is fetched — React Router simply swaps the view in the browser and rewrites the URL. It all works beautifully. Then the user bookmarks `/courses`, or just presses refresh, and the app 404s. Why? Because a refresh asks the

server directly for /courses, and a static host is a receptionist with a filing cabinet: it looks for a file literally named courses, finds nothing, and returns 'not found'.

The fix is a rewrite rule that tells the host to hand back /index.html for every path it does not recognise as a real file. Once index.html loads, React Router reads the URL and renders the right component, exactly as if you had navigated there in-app. On Vercel this is a couple of lines in vercel.json, but with one crucial subtlety: the rule must EXCLUDE /api/*. If you rewrite everything to index.html, your API calls get sent the HTML of your home page instead of JSON, and res.json() throws the tell-tale "Unexpected token '<' — you asked for data and got a web page. The real vercel.json uses a negative lookahead, "/((?!api/).*)", so every path except /api/* falls back to index.html. The symptom to watch for is an app that works perfectly while you click around but breaks the instant anyone refreshes on a deep link — so test a refresh on /courses before you call a deployment done.

- An SPA has one real file; the router fakes the other paths in the browser.
- A refresh asks the server for the real path, which does not exist on disk.
- Rewrite every unmatched path to /index.html — but EXCLUDE /api/*, or the API returns HTML.
- Always test a refresh on a deep link before declaring a deploy finished.

```
// vercel.json (real)

{
  "rewrites": [
    { "source": "/((?!api/).*)", "destination": "/index.html" }
  ]
}

// Everything EXCEPT /api/* falls back to index.html.
```

Lab 2.1 — Version Control with Git and GitHub

Objective: version the cookbake project and use Git as a safety net for AI edits.

Goal: The landing page only exists on one laptop, and the next lab hands it to an AI agent to rewrite. The learner initialises a repository, writes a .gitignore that keeps dependencies and secrets out of history, pushes cookbake to GitHub, and adopts the vibe coder's habit: commit before you prompt, diff after.

What you'll build

A GitHub repository holding the Cook & Bake Academy app, with node_modules, dist and .env.local excluded (Tech & files: git, GitHub, gh CLI, .gitignore.)

Step-by-step

22. Initialise the repository inside the cookbake folder

```
git init && git branch -M main
```

23. Write .gitignore — node_modules is reinstallable, dist is rebuildable, .env.local is secret. A credential committed once lives in the history forever

```
printf 'node_modules\ndist\n.env.local\n' > .gitignore
```

24. Stage everything and make the first commit

```
git add -A && git commit -m 'Cook & Bake Academy landing page'
```

25. Create the GitHub repository and push in one command

```
gh repo create cookbake --public --source=. --push
```

26. Now the safety net. Commit BEFORE you let an agent touch the code — this is the checkpoint you can fall back to

```
git commit -am 'checkpoint before AI refactor'
```

27. Let the agent make a real change, then read exactly what it did. git diff, not vibes, is how you audit an AI edit

```
Prompt: "Restyle the course grid to 3 columns on desktop and 1 on mobile, using CSS grid in src/index.css. Change nothing else." → git diff
```

28. If the agent broke it, throw the change away without losing your commit — then reprompt more precisely

```
git restore .
```

Test it

The cookbake repository exists on GitHub, git status is clean, node_modules and .env.local are untracked, and you can demonstrate reverting an AI edit with git diff followed by git restore.

Watch out for

Never commit `.env.local`. A secret pushed to GitHub stays in the history forever — rotating the credential is the only real fix.

Lab 2.2 — Deploy Cook & Bake Academy to Vercel

Objective: produce a production build and publish the app on a public URL.

Goal: The client wants to see their website. The learner inspects what npm run build actually emits, deploys the GitHub repo to Vercel so every push redeploys, adds the

SPA rewrite that Topic 6's routing will need, and learns the VITE_ rule: anything with that prefix is baked into the bundle and is therefore public.

What you'll build

Cook & Bake Academy live on a public Vercel URL, redeployed automatically on every git push (Tech & files: Vite production build, Vercel, vercel.json, environment variables.)

Step-by-step

29. Build the app and look at what came out. React is not running on a server — dist/ is plain static HTML, CSS and JS

```
npm run build && ls -R dist
```

30. Serve the production build locally before you ship it. This is the exact bundle the client will get

```
npm run preview
```

31. Import the cookbake repo at vercel.com/new. The Vite preset fills in build = npm run build and output directory = dist

32. Push a commit and watch Vercel rebuild by itself. Every branch and pull request gets its own preview URL

```
git push
```

33. Add vercel.json so deep links do not 404 once Topic 6 adds routing. The negative lookahead keeps /api/* out of the rewrite, because Topic 5 puts a real backend there

```
{ "rewrites": [{ "source": "/((?!api/).*)", "destination": "/index.html" }] }
```

34. Prove the VITE_ rule instead of taking it on trust: add VITE_SITE_NAME to .env.local, read it in a component, rebuild, and grep the shipped bundle for the value

```
echo 'VITE_SITE_NAME=Cook & Bake Academy' > .env.local && npm run build && grep -r 'Cook & Bake Academy' dist/assets/*.js
```

35. Draw the corollary now, before Topic 5: DATABASE_URL must NEVER carry the VITE_ prefix. It holds a password, and a VITE_ variable ships to every visitor

Test it

The app is live on a public Vercel URL, a fresh git push triggers an automatic redeploy, the VITE_ value is visibly present inside dist/assets/*.js, and you can explain why a VITE_ variable can never hold a secret.

Watch out for

A `VITE\_`-prefixed variable is compiled into the browser bundle. It is public. Your Neon `DATABASE\_URL` must never be one.

Lab 2.3 — Deploy to GitHub Pages with GitHub Actions

Objective: build a CI/CD pipeline and handle the two static-host routing gotchas.

Goal: Not every client uses Vercel. The learner vibe-codes a GitHub Actions workflow that builds and publishes the app on every push to main, then debugs the two classic static-host failures for themselves: a blank page caused by the wrong base path, and a 404 on refresh caused by the missing SPA fallback.

What you'll build

A GitHub Actions workflow deploying Cook & Bake Academy to GitHub Pages on every push to main (Tech & files: GitHub Actions, actions/deploy-pages, vite.config.js base path.)

Step-by-step

36. Prompt the agent for the workflow, then read every line of the YAML before you push it. CI you cannot read is CI you cannot debug

```
Prompt: "Create .github/workflows/deploy.yml for this Vite + React app. On push to main: checkout, setup-node 22 with npm cache, npm ci, npm run build, upload dist/ with actions/upload-pages-artifact@v3, and deploy with actions/deploy-pages@v4. Include the permissions block Pages requires."
```

37. Check the agent granted the exact permissions Pages needs — this is the block it most often omits

```
permissions: { contents: read, pages: write, id-token: write }
```

38. In the repo settings, set Pages → Build and deployment → Source to GitHub Actions, then push and watch the Actions tab

```
git add -A && git commit -m 'ci: deploy to GitHub Pages' && git push
```

39. Gotcha 1 — the site loads blank and the console shows 404s for the JS. Pages serves at /cookbake/, not /, so every asset URL is wrong

40. Fix it with the Vite base path, then rebuild. Keep it an env var so the Vercel deploy, which serves at /, is unaffected

```
export default defineConfig({ base: process.env.VITE_BASE || '/', plugins: [react()] })
```

41. Gotcha 2 — a static host looks for /courses on disk and 404s. Copy index.html to 404.html in the build step so Pages falls back to the app

```
cp dist/index.html dist/404.html
```

42. Confirm the pipeline: push, the workflow goes green, and the same commit is now live on TWO hosts — Vercel and Pages — from one repository

```
git push
```

Test it

The Actions workflow completes green, the site loads at `https://<user>.github.io/cookbake/` with its CSS and images intact, refreshing a sub-path does not 404, and you can explain what the base path and the 404.html fallback each fix.

Watch out for

GitHub Pages serves from `"/<repo>/"`, so a default `base` of `/'` gives a blank page with 404s on every asset.

Topic 03 — Improving Your App by Learning Core React Concepts

DOM vs Virtual DOM · Babel & JSX · Components & props · Lists and keys · Events & forms

Topic 1 left you with a page that works and code that does not scale: every component crammed into one file, and six copy-pasted course cards. This topic repairs both, and in doing so teaches the ideas the whole library rests on.

The central insight is that React is declarative. You never tell the browser how to change the page. You describe what the page should look like for the current state, and React compares that description against the previous one and applies the smallest possible set of real DOM edits. Everything else — JSX, props, keys, controlled inputs — follows from that one idea.

Key Concepts — Topic 03

- **The DOM is the browser's object tree** — The page you see IS the DOM. Touching it directly is slow and easy to get wrong — `document.getElementById`, `innerHTML`, manual patching.
- **The Virtual DOM is React's plan** — React builds a lightweight JS object tree, diffs it against the previous one, and patches ONLY the real DOM nodes that actually changed.
- **Babel compiles JSX away** — The browser has never seen JSX. Babel rewrites `<CourseCard fee={680}/>` into `React.createElement(...)` — a plain function call returning a plain object. Vite runs Babel for you.

- **Declarative beats imperative** — You describe what the UI should be for the current state; React works out which DOM operations get there.
- **Composition over configuration** — The children prop lets a component wrap arbitrary content instead of growing endless options.
- **key is identity, not decoration** — It tells React which course card is which across renders. `key={index}` corrupts state on reorder or delete.
- **The 0 && footgun** — 0 is falsy but not false, so `{seats && <Badge/>}` renders a literal 0. Write `seats > 0 && ...`
- **Derived state must not be stored** — If a value can be computed from existing state during render, compute it. Do not mirror it into `useState`.

Concepts Explained — Topic 03

The real DOM, and life without React

Think of it like... The DOM is the page itself — a live object tree. Editing it by hand is rewiring a running machine.

- The DOM is the browser's live object tree. Change a node's property and the screen changes.
- Without React you say HOW: `getElementById`, then overwrite `textContent`, `className`, `style`, `value`.
- The truth now lives in a DOM string, so you read '10 courses' back out and parse it to add to it.
- Every new feature multiplies the patches: filter the grid and you must also fix the count and the chip.
- One missed patch and the screen quietly disagrees with your data. Nothing throws. Nobody notices.

```
// Vanilla: filtering the Cook & Bake catalogue.

document.querySelectorAll('.card').forEach((card) => {

  const show = card.dataset.category === 'Bakery'

  card.style.display = show ? 'block' : 'none'

})

document.getElementById('count').textContent = '10 courses'

// ...plus the chip, the heading, the empty state...
```

Note: Hand-patching the DOM means saying HOW, in every place, every time.

The Virtual DOM and reconciliation

Think of it like... The Virtual DOM is an architect's blueprint — React compares the new drawing to the old before knocking down a single real wall.

- React keeps a lightweight JavaScript tree — the Virtual DOM — describing what the UI should be.
- Building it is cheap: it is just plain objects. Touching the real DOM is the expensive part.
- On each render React builds a NEW virtual tree for the current state.
- Reconciliation is the diff: it compares the new tree to the previous one to find what changed.
- It patches only those differences into the real DOM. Unchanged nodes are never touched.

```
// You describe the UI for this state:
```

```
<p>{visible.length} courses</p>
```

```
// Filter to Bakery: React builds a new virtual tree,
```

```
// diffs it, and patches ONE text node ('20' -> '10')
```

```
// plus removing 10 cards. The navbar, hero and footer
```

```
// were identical in both trees, so they are untouched.
```

Note: React diffs a virtual tree and patches only what actually changed.

Babel compiles JSX into function calls

Think of it like... Babel is a translator in a booth — you speak JSX, and the browser only ever hears plain JavaScript.

- The browser has NEVER seen JSX. It is not JavaScript and no browser can parse it.
- Babel compiles it away at build time. Vite wires Babel in through `@vitejs/plugin-react`.
- `<CourseCard fee={680}/>` becomes `React.createElement(CourseCard, { fee: 680 })`.
- That call returns a plain JavaScript object — `{ type, props }` — not HTML and not a DOM node.
- A tree of those objects IS the Virtual DOM. This is the join between the last slide and this one.

```
// YOU WRITE (JSX):
```

```
<CourseCard course={course} fee={680} />
```

```
// BABEL COMPILES IT TO (plain JavaScript):
```

```
React.createElement(CourseCard, { course: course, fee: 680 })
```

```
// WHICH RETURNS (a plain object – a React element):
```

```
{ type: CourseCard, props: { course: {...}, fee: 680 } }
```

Note: Babel rewrites every JSX tag into a call that returns a plain object.

Declarative beats imperative

Think of it like... Declarative code is ordering a dish, not reciting the recipe — you state the result you want and let React cook.

- Imperative code lists the steps: find the element, set its text, toggle a class, and so on.
- Declarative code describes the end result for the current state and lets React reach it.
- In React you never write 'update this node' — you write what the UI is, as a function of state.
- This means the UI can never drift out of sync with your data the way manual DOM edits do.
- Change the state, and the correct UI follows automatically; that is React's core promise.

```
// Imperative (vanilla): you do each step
```

```
el.textContent = String(visible.length)
```

```
// Declarative (React): you state the result
```

```
<p>{visible.length} courses</p>
```

Note: Describe the UI for the current state; React figures out the how.

State vs props

Think of it like... Props are the mail a component receives; state is the note it keeps on its own desk — one comes from outside, one it writes itself.

- Props are passed in from the parent and are read-only — the component cannot change them.
- State is owned by the component itself and can change over time in response to events.
- Changing state triggers a re-render; receiving new props from a parent also triggers a re-render.
- Ask 'does this component own this value, or is it given?' to decide between state and props.
- CourseCard is all props and no state. CoursesPage owns the query state and passes it down.

```
// SearchBar owns nothing – pure props.
function SearchBar({ value, onChange }) {
  return <input value={value}
    onChange={(e) => onChange(e.target.value)} />
}

// CoursesPage owns the state:
const [query, setQuery] = useState('')
<SearchBar value={query} onChange={setQuery} />
```

Note: Props come from outside and are read-only; state is owned and changeable.

State: a component's memory

Think of it like... State is a component's memory — a notepad it keeps between renders, not scratch paper binned each time.

- State is data a component owns and remembers across renders, unlike a variable reset every render.
- A plain let is recomputed and lost each render, so changing it can never update what is on screen.
- `useState` gives a remembered value plus a setter: `const [query, setQuery] = useState('')`.

- Calling the setter does two things: it stores the new value and schedules a re-render to show it.
- That re-render is the point — the UI redraws from the new state, so screen and data stay in step.

```
let q = ''          // reset every render – useless
q = 'sushi'        // nothing on screen changes

const [q, setQ] = useState('')
setQ('sushi')      // stored, AND schedules a re-render
```

Note: State is remembered data; its setter stores the value and re-renders.

Composition and the children prop

Think of it like... The children prop is a picture frame — it does not care what photo you slide in, it just wraps whatever you hand it.

- Composition means building complex UI by nesting simple components inside one another.
- The special children prop holds whatever JSX sits between a component's opening and closing tags.
- This lets Section, a Card or a Layout wrap arbitrary content without knowing what that content is.
- Favour composition over configuration — do not pile endless boolean props onto one component.
- The real Section takes id, title, eyebrow, alt and children — and renders anything you nest in it.

```
// src/components/Section.jsx (real code)
export default function Section({ title, children }) {
  return (
    <section className="section">
      {title} && <h2>{title}</h2>
      {children}
    </section>
  )
}
```

```

    </section>
  )
}

<Section title="Popular courses"><CourseGrid /></Section>

```

Note: children lets a component wrap any content instead of endless props.

Rendering lists with map()

Think of it like... map is a name-tag printer: feed it a list of students and it hands back one printed tag per person, in order.

- You render a list by calling map() to turn an array of data into an array of JSX elements.
- React accepts an array of elements directly inside JSX and renders them in order.
- This is what kills the hand-typed cards from Topic 1: 20 courses, one CourseGrid, one map().
- Adding a course is now a data edit, not a markup edit — and the grid updates itself.
- Every element in the returned list needs a key, which the next slide explains.

```

// src/components/CourseGrid.jsx (real code)

<div className="grid">

  {courses.map((course) => (
    <CourseCard key={course.id} course={course} />
  ))}

</div>

```

Note: map() turns a data array into an array of elements React renders.

key is identity, not decoration

Think of it like... A key is a student's name tag, not their seat number — reshuffle the room and React still knows exactly who is who.

- A key is a stable id React uses to track which item is which across renders.

- It lets React move, keep or remove the right cards instead of rebuilding the whole grid.
- Use a stable id from your data (course.id), never the array index, as the key.
- key={index} looks fine until you filter, reorder or delete — then state attaches to the wrong card.
- This is one of the single most common bugs in AI-generated list code, so always check it.

```
{courses.map((course) => (
  <CourseCard key={course.id} course={course} />
))}
```

```
// key={index} corrupts state when you filter to Bakery:
// the card at index 0 is now a different course, but
// React thinks it is the same one and keeps its state.
```

Note: Key each card by a stable id; key={index} corrupts state on change.

Conditional rendering and the 0 && trap

Think of it like... 0 && is a trapdoor — zero is falsy but not nothing, so React renders a bare 0 unless you gate on seats > 0.

- Use a ternary in JSX to choose between two elements based on a condition.
- Use && to render an element only when a condition is true, and nothing otherwise.
- The footgun: {seats && <Badge/>} renders a literal 0 on screen when seats is 0, because 0 is falsy.
- Fix it by making the left side a real boolean: {seats > 0 && <Badge/>}.
- This is subtle and the agent produces it often, so scan every && for a number on its left.

```
// From CourseCard: a ternary picks one of two.
<span>{isBakery ? '🍰 Bakery' : '🍳 Cooking'}</span>
```

```
// && renders one thing or nothing:
```

```
{seats > 0 && <span>{seats} places left</span>}
```

```
// {seats && ...} prints a literal 0 when seats is 0.
```

Note: Gate && on a real boolean (seats > 0), never a bare number.

Derived state must not be stored

Think of it like... Storing a derived value is photocopying a number already on the page — keep the copy and it soon disagrees with the original.

- If a value can be computed from existing state during render, compute it. Do not store it.
- The filtered course list is derived from courses + category + query. It is not its own state.
- Mirroring it into useState creates two sources of truth, and they will drift apart.
- Computing on render is cheap and always correct; a stored copy needs syncing you will forget.
- The agent loves to add a useState for filtered lists and totals. Delete it and compute inline.

```
// src/pages/CoursesPage.jsx (real code) – DERIVED.
```

```
const visible = courses.filter((c) => {  
  const matchesCat = category === 'All'  
  || c.category === category  
  const matchesQ = !needle  
  || c.title.toLowerCase().includes(needle)  
  return matchesCat && matchesQ  
})
```

Note: Compute derived values on render; never mirror them into useState.

Synthetic events

Think of it like... A synthetic event is a universal remote — React wraps every browser's quirks in one consistent set of buttons.

- You attach handlers with camelCase props like `onClick` and `onChange`, passing a function.
- Pass the function, do not call it: `onClick={handleEnroll}`, not `onClick={handleEnroll()}`.
- To pass an argument, wrap it in an arrow: `onClick={() => onChange(cat.value)}`.
- React hands your handler a synthetic event, a cross-browser wrapper over the native event.
- Read the input's value from `e.target.value`; call `e.preventDefault()` to stop a form reloading.

```
// src/components/CategoryFilter.jsx (real code)

{categories.map((cat) => (
  <button key={cat.value}
    onClick={() => onChange(cat.value)}>
    {cat.label}
  </button>
))}

// onClick={onChange(cat.value)} would CALL it on render.
```

Note: Pass a function to `onClick`; wrap in an arrow to pass an argument.

Controlled inputs

Think of it like... A controlled input is a puppet on React's strings — state moves the field, and every keystroke reports back to pull the string.

- In a controlled input, React state is the single source of truth for the field's value.
- You bind `value={state}` and update it in `onChange`, so state and the input never disagree.
- This unlocks live search, validation, formatting and disabling a submit button as the user types.
- A value with no `onChange` makes the field read-only — React warns you in the console.
- `SearchBar` is controlled and owns nothing: the value comes down, every keystroke goes back up.

```
// src/components/SearchBar.jsx (real code)
```

```
<input
  type="search"
  placeholder="Search sourdough, sushi, macaron..."
  value={value}
  onChange={(e) => onChange(e.target.value)}
/>
```

Note: Bind value to state and update it in onChange — React owns the field.

Deep Dive — The real DOM, and updating a page without React

Before you can appreciate what React does, you have to see what it saves you from. When the browser loads a page it parses the HTML into the DOM — the Document Object Model — which is a live tree of objects, one for each element. This is the crucial point that beginners miss: the DOM is not a picture of the page or a copy of it. The DOM IS the page. The pixels on screen are a rendering of that object tree, so when you change an object in the tree, the screen changes with it. JavaScript can reach straight into this tree: `document.getElementById('count')` hands you the real object for one node, and you can write to its properties — `textContent`, `className`, `style`, `value` — and watch the display update.

The problem is not that this is impossible; it is that YOU have to describe every step of HOW. Suppose the Cook & Bake catalogue has twenty course cards and you add a Bakery/Cooking filter. In vanilla JavaScript you select every card, loop over them, and toggle each one's display by hand. But that is only the beginning, because the number of visible courses is shown in a heading, so you must also find that node and overwrite its text; the active filter chip needs a class toggled; the empty-state message must appear when nothing matches; the URL might need updating so the filter is shareable. Every one of these is a separate manual patch, and every new feature multiplies them. Worse, the truth now lives inside a DOM string: to increment a count you read '10 courses' back out of a text node, parse the number, add one, and write it back. Miss a single one of these patches and the screen quietly disagrees with your data — the heading says twenty while ten cards show. Nothing throws an error. Nobody notices until a user does. This is exactly the tangle React was built to eliminate, and seeing it first is what makes the next two ideas land.

- The DOM is the browser's live object tree — the page itself, not a copy of it.
- Without React you say HOW: `getElementById`, then overwrite `textContent`, `className`, `style` by hand.
- The truth ends up living in DOM strings, which you must read back out and parse to update.

- Every feature multiplies the manual patches, and one missed patch silently desyncs the screen.

```
// Vanilla JS: filter the catalogue BY HAND.

document.querySelectorAll('.card').forEach((card) => {

  const show = card.dataset.category === 'Bakery'

  card.style.display = show ? 'block' : 'none'

})

// ...now keep the count in step, by hand:

const el = document.getElementById('count')

el.textContent = '10 courses'

// ...plus the chip, the empty state, the URL...
```

Deep Dive — The Virtual DOM, and declarative UI

React's answer to all that manual patching is the Virtual DOM. Instead of mutating the real DOM directly, React keeps a lightweight copy of the UI tree made of plain JavaScript objects — exactly the objects that Babel's `createElement` calls return, from Topic 1. Building this virtual tree is cheap, because it is just allocating objects in memory; it is touching the real DOM that is comparatively slow. Every time your state changes, React builds a brand-new virtual tree describing what the UI should be right now. Then it does something clever called reconciliation: it compares that new tree against the previous one, works out the minimal set of differences, and patches only those into the real DOM. Picture an architect comparing a new blueprint against the old one before knocking down a single real wall — nothing physical changes until the cheap paper comparison has found exactly what needs to move. When the Cook & Bake filter narrows twenty courses to ten, React does not rebuild the navbar, the hero or the footer; the diff shows they are identical in both trees, so it leaves them completely alone and only removes ten cards and updates one number.

This is what makes React declarative, and the distinction is the heart of the whole library. Imperative code is a list of steps to mutate the page — the vanilla filter from the previous dive. Declarative code describes the end result for the current state and lets React figure out how to get there. You never write 'update this text node' or 'toggle that display'; you write 'the visible courses are these, given this filter', and when the filter changes the correct UI simply follows. It is the difference between reciting a recipe and ordering the finished dish. And because the UI is always computed fresh from state, it can never silently fall out of step with your data the way hand-written DOM manipulation does — the desync bug from the previous dive becomes impossible by construction. Your whole job shrinks to two things: keep the state correct, and describe the UI as a function of it.

- The Virtual DOM is a cheap JavaScript tree (the objects `createElement` returns) React diffs first.
- Reconciliation finds the minimal changes and patches only those real nodes; the rest are untouched.
- Declarative means you describe the UI for the current state, not the steps to mutate the page.
- Because the UI is recomputed from state, it can never silently drift out of sync with your data.

// WITHOUT React – you patch the real DOM yourself:

```
el.textContent = String(visible.length)
```

```
el.className = visible.length ? 'grid' : 'grid is-empty'
```

// WITH React – you describe the result, once:

```
<p>{visible.length} courses</p>
```

```
<CourseGrid courses={visible} />
```

Deep Dive — Keys are identity, and why `key={index}` bites

When you render a list, you map an array of data into an array of JSX elements, and React asks you to give each element a key. It is tempting to treat the key as a formality and reach for the array index, and the agent will often do exactly that. This works right up until the list changes order, and then it produces one of the most confusing bugs in React. The reason is what a key actually means. A key is not a position — it is an identity. It is the name tag a student wears, not the seat they happen to be sitting in. Across renders, React uses keys to answer the question 'is this the same item as before, or a different one?' so it can move, keep, or remove the right rows rather than rebuilding the whole list.

Now picture the Cook & Bake grid keyed by index 0, 1, 2, and imagine each card holds some internal state — a hover flourish, or an input. Filter the catalogue to Bakery only, and the course that used to sit at index 3 is now at index 0. React sees 'key 0 still exists' and keeps the state that belonged to the old index-0 card, attaching it to a completely different course. The visible symptom is state jumping to the wrong cards on filter, reorder or delete. The fix is to key by something stable and unique from your data — `course.id`, which the real `CourseGrid` uses — so the name tag travels with the item no matter where it moves in the list. Whenever you review AI-generated list code, checking the key is one of the highest-value things you can do, because this bug renders perfectly and only surfaces on interaction.

- A key is a stable identity, not a position — a name tag, not a seat number.
- React uses keys to move, keep or remove the right cards across renders.

- `key={index}` attaches state to the wrong course when the grid filters, reorders or shrinks.
- Key by a stable id (`course.id`) from your data; auditing keys is high-value on AI code.

```
// src/components/CourseGrid.jsx (real code)

{courses.map((course) => (
  <CourseCard key={course.id} course={course} />
))}
```

```
// key={index} corrupts state when you filter to Bakery
```

Deep Dive — Composition, events and controlled inputs

React builds complex interfaces by composing simple components, and the mechanism that makes this elegant is the `children` prop. Whatever JSX you place between a component's opening and closing tags arrives inside that component as `props.children`. The real `Section` component uses exactly this: it renders `{children}` inside a centred container, so it can wrap a `CourseGrid` on the home page and a form somewhere else without knowing or caring what it holds. The lesson is to favour composition over configuration: rather than growing a component endless boolean props to cover every case, let it wrap arbitrary children and stay generic. A `Section` that renders `{children}` is reusable everywhere; a `Section` with twenty props for every possible layout is reusable nowhere.

Interactivity comes from events. You attach handlers with camelCase props like `onClick` and `onChange`, and the critical detail is that you pass the function rather than call it: `onClick={handleEnroll}`, never `onClick={handleEnroll()}`, because the second runs it immediately during render. When you need to pass an argument you wrap the call in an arrow, exactly as `CategoryFilter` does — `onClick={() => onChange(cat.value)}` — so the function runs on click, not on render. React hands your handler a synthetic event, a cross-browser wrapper that smooths over the differences between browsers, a universal remote over everyone's slightly different hardware. From it you read `e.target.value` to see what the user typed, and you call `e.preventDefault()` to stop a form doing its default full-page reload.

Those two ideas combine in the controlled input, the standard way React handles forms. In a controlled input, React state is the single source of truth for the field: you bind `value={state}` and update that state in `onChange`. The real `SearchBar` is precisely this — it owns no state of its own, takes the value down as a prop and reports every keystroke back up through `onChange`. The input becomes a puppet on React's strings: state moves the field, and every keystroke reports back to pull the string. This is more setup than an uncontrolled field, but it is what unlocks the live-as-you-type search filter, validation, and disabling a submit button until a form is valid. If you set `value` without an

onChange, React warns you that you have made a read-only field, because you have given it a source of truth but no way to update it.

- children lets a component like Section wrap arbitrary content — favour composition over configuration.
- Pass a function to onClick/onChange; wrap it in an arrow to pass an argument (() => onChange(x)).
- Synthetic events give one consistent, cross-browser event object; read e.target.value from it.
- A controlled input binds value to state and updates it in onChange — SearchBar owns nothing.

```
// src/components/SearchBar.jsx (real code)

export default function SearchBar({ value, onChange }) {

  return (

    <input

      type='search'

      value={value}

      onChange={(e) => onChange(e.target.value)}

    />

  )

}
```

Deep Dive — State: a component's memory

Topic 3 introduces the single idea that turns a static page into an application: state. State is a component's memory — the data it owns and remembers from one render to the next. The reason a component needs a special mechanism for this, rather than an ordinary variable, is that a component function runs again from scratch on every render. Any plain variable you declare inside it is created fresh and thrown away each time, so changing it does nothing lasting and, crucially, cannot change what is on screen. Write `let query = ''` and then `query = 'sushi'` inside a component and the screen never moves, because the next render simply starts over at the empty string. State is the notepad React keeps for the component between renders; a local variable is scratch paper torn off and binned each time.

You create state with `useState`, which hands back the current value and a setter: `const [query, setQuery] = useState('')`. The setter is the important half, because calling it does two things at once. It stores the new value so the component will remember it on the next render, and it schedules a re-render so the screen catches up. That second step is precisely what a plain variable can never do. This is the loop at the heart of React: an

event calls the setter, the setter records the new state and asks React to re-render, the component runs again, and it returns fresh JSX computed from the new state. You never touch the DOM yourself — you change state, and the declarative render you met earlier redraws the UI to match. State and screen stay in step by construction, which is the exact opposite of the manual-DOM desync from the first dive in this topic.

Deciding whether a value should be state or a prop comes down to one question: does this component own the value, or was it given the value? Props come from the parent and are read-only; state is owned locally and can change over time. `SearchBar` owns nothing and is all props; `CoursesPage` owns the search query in state and passes it down. When several components need the same changing value, the answer is not to copy state into each of them — it is to lift the state up to their nearest common parent and pass it back down as props, so there is a single source of truth. Two rules from Topic 4 then govern how you change state safely, and they are worth previewing: within a render the state value is a fixed snapshot, and you replace it with a brand-new value rather than mutating it in place. But the foundational idea comes first and is simpler — state is the memory that lets a component change, and the setter is what makes the screen follow.

- State is data a component owns and remembers across renders; a local variable resets every render.
- A plain variable cannot update the screen — only a setter both stores a value and re-renders.
- The loop: event → setter → re-render → UI recomputed from the new state.
- Ask 'owned or given?' — owned is state, given is a prop; lift shared state to a common parent.

```
let query = ''          // reset every render – useless
query = 'sushi'         // the screen never moves
```

```
const [query, setQuery] = useState('')
setQuery('sushi')      // stored, and re-renders the UI
```

Lab 3.1 — The Real DOM, the Virtual DOM and Babel

Objective: explain what the browser actually runs — and why React is worth the layer.

Goal: Before improving the app, the learner finds out what React is doing on their behalf. They build the Cook & Bake course grid TWICE: once by hand with `document.getElementById` and `innerHTML` (the real DOM), and once in React. They then see, with their own eyes, that the browser never receives a single line of JSX — Babel has already compiled it into `React.createElement` calls that return plain JavaScript objects, and it is those objects that React diffs.

What you'll build

A hand-built vanilla DOM grid at /vanilla.html, sitting next to the React grid — and the compiled output of your own JSX (Tech & files: document.getElementById, innerHTML, the Virtual DOM, reconciliation, Babel, @vitejs/plugin-react, babeljs.io/repl.)

Step-by-step

43. REAL DOM — build the course grid the way you would WITHOUT React. Vite serves public/ at the site root, so this runs immediately at <http://localhost:5173/vanilla.html>

```
Prompt: "Create public/vanilla.html: plain HTML and JS, no React. Hardcode
an array of 6 Cook & Bake courses (title, level, fee, weeks, emoji). Render
them into <div id=\"grid\"> by building an HTML string and assigning it to
grid.innerHTML. Add a search <input id=\"q\"> whose oninput re-runs that
same render with the filtered array."
```

44. Feel why it hurts. Type in the vanilla search box and watch: every keystroke calls `grid.innerHTML = ...`, which DESTROYS and rebuilds all six cards. Scroll position resets, any focus inside the grid is lost, and a course titled `<img onerror=alert(1)>` would execute — innerHTML does not escape anything

```
document.getElementById('grid').innerHTML = courses.map(cardHtml).join('')
```

45. VIRTUAL DOM — now the React way. You never write a DOM instruction at all: you describe what the grid should BE for the current query, and React works out the difference

```
const visible = courses.filter((c) =>
c.title.toLowerCase().includes(q.toLowerCase()))

return <div className="grid">{visible.map((c) => <CourseCard key={c.id}
course={c} />)}</div>
```

46. Prove React is not rebuilding everything. Open DevTools → Settings → More tools → Rendering → tick 'Paint flashing', then filter the React grid: only the cards that actually left or arrived repaint. React diffed its new object tree against the previous one and patched ONLY the changed real-DOM nodes

47. BABEL — the browser has never seen JSX; it is not valid JavaScript. Paste one line of your own card JSX into <https://babeljs.io/repl> (tick the 'react' preset, set Runtime = classic) and read the right-hand pane: your tag has become a plain function call

```
<CourseCard fee={680} title="Artisan Sourdough Bread Baking" />
```

```
// Babel output:
```

```
React.createElement(CourseCard, { fee: 680, title: "Artisan Sourdough Bread
Baking" });
```

48. See Babel run inside YOUR project. Vite compiles every .jsx module with Babel (@vitejs/plugin-react) before the browser gets it. Ask the dev server for the module and read what it actually sends — there is no JSX in the response, only jsx(...) calls imported from react/jsx-runtime (React 19's automatic runtime — same idea as createElement, just auto-imported)

```
curl -s 'http://localhost:5173/src/components/CourseCard.jsx' | head -40
```

49. Close the loop: React.createElement returns a plain JS OBJECT, not a DOM node. Log an element in the browser console and expand it — { type, key, props } — and nothing is on the page yet. That object tree IS the Virtual DOM; createRoot(...).render() is the only thing that turns it into real DOM

```
console.log(<CourseCard course={courses[0]} />)
```

```
// { type: CourseCard, key: null, props: { course: {...} }, ... }
```

Test it

Both grids show the same six courses, but you can demonstrate that the vanilla one rebuilds every card on each keystroke while React repaints only what changed; you can show the Babel output of your own JSX as a React.createElement call; and you can state what the Virtual DOM object tree is and where the ONE real-DOM write in a React app lives (main.jsx).

Watch out for

``dangerouslySetInnerHTML`` turns off the escaping that protects you from XSS. JSX escapes values for you — let it.

Lab 3.2 — Components, Props and Composition

Objective: refactor the Topic 1 monolith into composable components with one-way data flow.

Goal: This lab pays off smell 1 from Lab 1.3. The learner moves the Cook & Bake catalogue into src/data/courses.js, splits App.jsx into real component files, builds a Section layout component that wraps arbitrary content with the children prop, and refactors CourseCard to take ONE course object — the same shape the Neon Postgres row will have in Topic 5, so the card will never need changing again.

What you'll build

Navbar, Hero, Section, CourseGrid, CourseCard and Footer as separate files, composed by App.jsx and driven by src/data/courses.js (Tech & files: Components, props, the children prop, composition, src/data/courses.js, src/components/*.jsx.)

Step-by-step

50. Move the catalogue out of the markup. Every course is now a data record with the exact fields the database will later return

Prompt: "Create src/data/courses.js exporting `courses`: an array of all 20 Cook & Bake courses (10 Bakery, 10 Cooking). Each has id, code (BAK-1xx / CUL-2xx), slug, title, category ('Bakery' | 'Cooking'), level, weeks, fee, campus ('Bakehouse' | 'Culinary'), summary, emoji, image. Also export `campuses` (Bakehouse / Culinary, each with emoji, name, address, area) and `categories` = All / 🍰 Bakery / 🔪 Cooking."

51. Extract the components the AI buried in App.jsx into their own files — one component, one file

```
src/components/{Navbar,Hero,Footer,Section,CourseGrid}.jsx
```

52. Rewrite App.jsx so it only COMPOSES. It should now define nothing but App itself

```
<Navbar />

<Hero onBrowse={scrollToCourses} />

<Section eyebrow="Our programmes" title="Popular courses">...</Section>

<Footer />
```

53. Build Section as a layout component. Anything nested between its tags arrives as the `children` prop — that is how a component wraps content it knows nothing about, instead of growing endless props

```
export default function Section({ id, title, eyebrow, alt, children }) {

  return <section id={id} className={alt ? 'section section--alt' : 'section'}>{children}</section>

}
```

54. Refactor CourseCard from six flat props to a single `course` object, and destructure inside. The call site collapses from six attributes to one

```
export default function CourseCard({ course }) {

  const { slug, title, category, level, weeks, fee, campus, image } = course

  ...

}
```

55. Swap the emoji placeholder for the real photo, and read the campus name out of the campuses map rather than printing the raw key

```
<div className="card__img" style={{ backgroundImage: `url('${courseImage(image)}')` }} />
```

```
<span>📍 {campuses[campus].area}</span>
```

```
<span className="card__price">$${fee}</span>
```

56. Confirm data flows ONE way. Hero takes an `onBrowse` function prop — the parent decides what the button does; the child only calls it. A child never writes to a prop and never reaches up to its parent

Test it

`App.jsx` defines no component other than `App`, the page renders exactly as before, `CourseCard` takes a single `course` prop, and adding a 21st course to `src/data/courses.js` requires no change to any component.

Watch out for

Props are read-only. If a child needs to change something, the parent passes a callback down; the child never writes to a prop.

Lab 3.3 — Lists, Keys and Conditional Rendering

Objective: render the whole catalogue from data with stable keys, and render UI conditionally without bugs.

Goal: This lab pays off smell 2 from Lab 1.3. Six copy-pasted cards become one `.map()` over all 20 courses. The learner then deliberately BREAKS it with `key={index}` to see state stick to the wrong course card, and meets the 0 && footgun that puts a stray zero on the page.

What you'll build

A `CourseGrid` rendering all 20 Cook & Bake courses from data, with a level badge and a real empty state (Tech & files: `Array.map`, the key prop, reconciliation, conditional rendering, `src/components/CourseGrid.jsx`.)

Step-by-step

57. Delete the six hardcoded `<CourseCard>` elements and render the array instead. One expression now draws the entire catalogue

```
{courses.map((course) => <CourseCard key={course.id} course={course} />)}
```

58. Import the data and render all 20 on the courses section, and just the first 6 as 'Popular courses' on the home page — same component, different slice

```
const popular = courses.slice(0, 6)
```

59. Understand `key`: it is React's IDENTITY for a row across renders. It tells the diff that the sushi card is still the sushi card even though it moved from position 9 to position 2
60. Reproduce the bug on purpose. Switch to `key={index}`, type in a card's quantity input, then filter out a card ABOVE it — the typed value stays behind on the wrong course, because position 3 is still 'key 3' to React

```
key={index} // 🛑 breaks on filter, reorder, insert and delete
```

61. Add a conditional badge with `&&` — only Advanced courses get the chef's-hat flag

```
{course.level === 'Advanced' && <span className="card__lvl">👨🍳  
Advanced</span>}
```

62. Meet the footgun: 0 is falsy but is NOT false, so React renders a literal 0 onto the page. The fix is to make the left side a real boolean

```
{course.seatsLeft && <Badge/>} // 🛑 renders a bare 0 when seatsLeft  
=== 0
```

```
{course.seatsLeft > 0 && <Badge/>} // ✅
```

63. Handle the empty list with an early return, not an `&&` chain — the filter in the next lab will hit it constantly

```
if (!courses.length) return <p className="muted">No courses match your  
search.</p>
```

Test it

All 20 courses render from `src/data/courses.js` with `key={course.id}`, the home page shows 6, Advanced courses show the badge, an empty array shows the empty state, no literal 0 appears anywhere, and you can demonstrate the `key={index}` bug and explain it.

Watch out for

`key={index}` looks fine until you delete or reorder a row, at which point React reuses the wrong DOM node and state sticks to the wrong card. And `{seats && ...}` renders a literal 0, because 0 is falsy but is not `false`.

Lab 3.4 — Events, the Bakery / Cooking Filter and a Controlled Search Box

Objective: handle React events and build controlled inputs without storing derived state.

Goal: The final piece of the phase: the mockup's All / 🍞 Bakery / 🔍 Cooking chips and a live search box. The learner meets React's synthetic events and the controlled input, and then the single most common mistake in AI-generated React — mirroring a value that can simply be COMPUTED into a second piece of state.

What you'll build

A working toolbar over the catalogue: a SearchBar and the CategoryFilter chips, filtering all 20 courses live (Tech & files: Synthetic events, onClick, onChange, onSubmit, controlled inputs, useState (preview), src/components/{SearchBar,CategoryFilter}.jsx.)

Step-by-step

64. Vibe-code the two toolbar components — and note the constraint in the prompt: neither owns state. The parent holds the truth; the chips and the box only report clicks and keystrokes up

```
Prompt: "Create src/components/CategoryFilter.jsx and
src/components/SearchBar.jsx. Both are CONTROLLED: CategoryFilter({ value,
onChange }) maps the exported `categories` (All / 🍰 Bakery / 🔍 Cooking) to
<button className='chip'> elements, marking the one matching `value` as is-
active. SearchBar({ value, onChange }) is a single controlled <input
type='search'>. Neither component may call useState."
```

65. Read the event React hands you. It is a SyntheticEvent — React's own wrapper, so the same handler behaves identically in every browser

```
<button onClick={(e) => console.log(e.type, e.target)}>
```

66. Pass an argument to a handler by wrapping it in an arrow function. Writing `onClick={onChange(cat.value)}` calls it during render instead — a classic AI slip

```
onClick={() => onChange(cat.value)} // ✅ a function React can call later
```

67. Make the input CONTROLLED: the value comes DOWN from state, and every keystroke sends it back UP. React, not the DOM node, owns what is in the box

```
<input type="search" value={value} onChange={(e) =>
onChange(e.target.value)} />
```

68. Hold the two filters in the parent with useState — the minimum you need today; Topic 4 goes deep on hooks

```
const [query, setQuery] = useState('')
const [category, setCategory] = useState('All')
```

69. Compute the visible list DURING RENDER. It is a plain local variable, derived from state — it is NOT state itself

```
const needle = query.toLowerCase()
const visible = courses.filter((c) =>
  (category === 'All' || c.category === category) &&
```

```
(c.title.toLowerCase().includes(needle) ||  
c.summary.toLowerCase().includes(needle)))
```

70. Now audit the anti-pattern the agent will offer you: a second `useState` for `'filtered'` kept in sync by a `useEffect`. It renders twice, it can go stale, and it can disagree with the source of truth. If you can compute it, do not store it

```
const [filtered, setFiltered] = useState(courses) // 🚫  
useEffect(() => setFiltered(courses.filter(...)), [query]) // 🚫 delete both
```

Test it

Typing in the search box filters the 20 courses live, the 🍰 Bakery and 🔍 Cooking chips narrow the grid further and combine with the search, an unmatched search shows the empty state, and the filtered array exists only as a local variable — never in `useState`.

Watch out for

Never mirror derived state into `'useState' + 'useEffect'`. If the filtered list can be computed from `'courses'` and `'query'` during render, compute it during render.

Topic 04 — React Hooks (The Vibe Way)

`useState` · `useEffect` · `useRef` · `useContext` · `useReducer` · Custom hooks

A component that cannot remember anything can only ever render. Hooks give components memory (`useState`), a way to reach the outside world (`useEffect`), an escape hatch to the real DOM (`useRef`), and a way to share both state (`useContext`) and logic (custom hooks).

Two ideas cause almost every hook bug. First, state is a snapshot: within one render, `'count'` never changes, which is why `setCount(count + 1)` twice only adds one. Second, an effect that starts something must be able to stop it: the function you return from `useEffect` is the cleanup, and React's StrictMode deliberately runs your effects twice in development to expose the ones that forgot it.

Key Concepts — Topic 04

- **The Rules of Hooks** — Call them at the top level of a component or another hook. React tracks hooks by call order.
- **State is a snapshot** — `count` does not change mid-render. Use the updater form `setCount(c => c + 1)` when the next state depends on the previous.
- **Immutable updates** — `cart.push(x)` does not re-render. Build a new array: `[...cart, x]`.

- **Every effect cleans up after itself** — Timers, subscriptions and listeners are torn down in the function `useEffect` returns. `StrictMode` double-invokes effects to expose the ones that don't.
- **useRef is an escape hatch to the real DOM** — Changing `.current` does NOT re-render. It is how you focus the search box, scroll to the course grid, or hold a timer id — the one sanctioned way to touch a DOM node directly.
- **ref vs state — the dividing line** — If changing it should redraw the screen, it is state. If it should not, it is a ref. Putting a timer id in state re-renders on every tick.
- **Context is dependency injection** — It removes prop drilling. `useReducer` names the actions that change state. Custom hooks share logic, never state.

Concepts Explained — Topic 04

The Rules of Hooks

Think of it like... Hooks are called in roll-call order — React counts 'hook one, hook two' every render, so you must never skip or reorder a name.

- Call hooks only at the top level of a component or hook — never in a loop, condition or nested function.
- React identifies each hook purely by its call order, which must be identical on every render.
- A hook inside an `if` runs some renders and not others, and the whole list of hooks shifts and breaks.
- Only call hooks from React function components or from your own custom hooks.
- The lint rules for hooks catch most violations — do not disable them to silence a warning.

```
function CoursesPage({ open }) {
  const [q, setQ] = useState('') // ok: top level
  if (open) {
    const [x] = useState(1)      // never: in an if
  }
}
```

Note: Call hooks at the top level, in the same order, every render.

What a hook actually is

Think of it like... A hook is a power outlet on the render: plug in with `use...()` and your function taps React's memory and lifecycle.

- A hook is a function whose name starts with 'use' that lets a function component use React features.
- Hooks let a stateless function remember values (useState) and sync with the outside (useEffect).
- React identifies each hook by call order, matching it to a memory slot by position, not by name.
- That is why hooks live at the top level — never inside an if, a loop, or after an early return.
- Only components and other hooks may call hooks; a plain helper function cannot.

```
function CoursesPage() {
  const { courses } = useCourses()    // custom hook
  const [q, setQ] = useState('')      // slot 2
  const searchInput = useRef(null)    // slot 3
  useEffect(() => {                    // slot 4
    searchInput.current?.focus()
  }, [])
}
```

Note: A hook is a use...() function; React tracks it by call order.

State is a snapshot

Think of it like... State is a photograph, not a live feed — count is frozen at the value it had when this render's photo was taken.

- useState returns the current value and a setter: `const [count, setCount] = useState(0)`.
- Within one render the state value never changes — it is a snapshot captured at render time.
- Calling `setCount(count + 1)` twice in a row adds only one, because both read the same stale count.
- When the next value depends on the previous, use the updater form: `setCount(c => c + 1)`.
- Setting state schedules a re-render; it does not change the current variable mid-function.

```
setCount(count + 1)    // stale: +1 total  
setCount(count + 1)
```

```
setCount((c) => c + 1) // updater: +1  
setCount((c) => c + 1) // updater: +2
```

Note: State is fixed per render; use `setX(prev => ...)` for dependent updates.

Immutable updates

Think of it like... Mutating state is editing the original photo — React only re-renders when you hand it a brand-new print.

- React decides to re-render by checking whether the state value is a NEW reference.
- `items.push(course)` mutates the same array, so the reference is unchanged and nothing re-renders.
- Build a new value with spread instead: `setItems([...items, course])` creates a fresh array.
- Update an object the same way: `setCourse({ ...course, fee: 720 })`.
- The real `CartContext` never mutates: every add, remove and clear returns a brand-new array.

```
// src/context/CartContext.jsx (real code)  
  
const addItem = (course) =>  
  setItems((prev) =>  
    prev.some((c) => c.id === course.id)  
      ? prev  
      : [...prev, course])    // NEW array -> re-renders  
  
  
const removeItem = (id) =>  
  setItems((prev) => prev.filter((c) => c.id !== id))
```

Note: Never mutate state — build a new array or object with spread.

useEffect and the dependency array

Think of it like... The dependency array is a guest list — the effect only re-runs when a name on the list actually changes.

- `useEffect` runs code after render for side effects: fetching, subscriptions, timers, the DOM.
- Its second argument, the dependency array, decides when it re-runs.
- An empty array `[]` runs the effect once after the first render, and never again.
- `Listing [slug]` re-runs the effect whenever `slug` changes — that is how a new course gets fetched.
- Omit the array entirely and it runs after every render — the classic infinite-fetch loop.

```
// src/context/ThemeContext.jsx (real code)

useEffect(() => {

  document.documentElement.dataset.theme = theme

}, [theme]) // re-runs only when the theme changes
```

Note: `[]` runs once; `[dep]` re-runs when `dep` changes; no array runs always.

Every effect cleans up after itself

Think of it like... `useEffect` cleanup is turning the tap off as you leave the room — `StrictMode` runs in and out twice to catch a tap left running.

- An effect that starts something ongoing must stop it, or you leak timers, listeners and requests.
- Return a cleanup function from the effect; React runs it before the next effect and on unmount.
- `clearTimeout`, `clearInterval`, `removeEventListener` and `unsubscribe` all belong in that function.
- In development, `StrictMode` mounts, unmounts and remounts once to expose effects with no cleanup.
- So a double log in dev is not a bug — it is React helping you find a missing cleanup.

```
// src/hooks/useDebounce.js (real code)

useEffect(() => {

  const id = setTimeout(() => setDebounced(value), delay)
```

```

    return () => clearTimeout(id)    // cleanup
  }, [value, delay])

// Every keystroke cancels the previous pending timer,
// so only the last one ever fires.

```

Note: Return a cleanup for anything ongoing; StrictMode double-runs to test it.

useRef: the escape hatch to the DOM

Think of it like... A ref is a sticky note the renderer never reads — write to `.current` all you like and React will not repaint the screen.

- `useRef` returns a mutable object, `{ current }`, whose value persists across renders like state does.
- But unlike state, writing to `.current` never triggers a re-render — that is its defining difference.
- Job one: a live handle on a real DOM node, for the things JSX cannot describe.
- Job two: a mutable box for a value that must survive renders but must not drive the UI.
- In React 19 `ref` is a normal prop, so you pass it straight to your own components — no `forwardRef`.

```

const searchInput = useRef(null) // job one: a DOM handle
const timer = useRef(null)       // job two: a mutable box

```

```

// .current is null on the first render – React has not
// created the node yet. It is set right after paint.

```

```

useEffect(() => {
  searchInput.current?.focus()
}, [])

```

```

<input ref={searchInput} /> // React 19: ref is a prop

```

Note: Two jobs — a DOM handle and a mutable box; neither triggers a re-render.

ref vs state — the dividing line

Think of it like... State is what the audience sees; a ref is the stagehand's clipboard — necessary, remembered, and never on stage.

- The whole decision is one question: should changing this value REDRAW the screen?
- If yes, it is STATE. The search text is state — every keystroke must repaint the results.
- If no, it is a REF. Focus, scroll position and a timer id change nothing that is rendered.
- Put a timer id in state and you re-render the entire component on every single tick. For nothing.
- 'Is focused' is browser state, not React state. There is no JSX you can write to express it.

```
// STATE – the screen must change:
const [query, setQuery] = useState('')

// REF – the screen must NOT change:
const searchInput = useRef(null)    // which DOM node
const timer = useRef(null)          // the interval id

// timer in useState => a re-render every tick, forever.
```

Note: Redraw the screen? State. Must not redraw? Ref. That is the whole rule.

Three real refs in Cook & Bake

Think of it like... Refs are the three things the declarative world cannot say: put the cursor here, scroll there, hold this receipt.

- CoursesPage focuses the search box on mount, and again after every category chip click.
- The ref is created in the PARENT and handed down to SearchBar as an ordinary prop.
- HomePage points a ref at the course grid, so the hero's 'Browse courses' button can scroll to it.

- Hero receives onBrowse as a function prop — it never knows a ref or a DOM node exists.
- A timer id lives in a ref because clearing it later must not cost the user a re-render.

```
// CoursesPage.jsx – focus the search input
const searchInput = useRef(null)
useEffect(() => { searchInput.current?.focus() }, [])
<SearchBar inputRef={searchInput} ... />

// HomePage.jsx – scroll the grid into view
const popularRef = useRef(null)
const scrollToCourses = () =>
  popularRef.current?.scrollIntoView({ behavior: 'smooth' })
<Hero onBrowse={scrollToCourses} />
<section ref={popularRef}>...</section>
```

Note: Focus, scroll and timer ids: the sanctioned reasons to hold a DOM node.

useContext ends prop drilling

Think of it like... Context is a building's PA system — announce once and any floor hears it, instead of passing a note desk to desk down every level.

- Prop drilling is threading a value through components that do not use it, just to reach a deep child.
- Context provides a value once high in the tree and lets any component below read it directly.
- Create a context, wrap a subtree in its Provider, and read it with useContext — or a custom hook.
- Cook & Bake has three: ThemeContext (dark mode), AuthContext (the user), CartContext (shortlist).
- Do not use context for fast-changing values like keystrokes — every consumer re-renders on change.

```
// src/context/CartContext.jsx (real code)
```

```
export function useCart() {
  const ctx = useContext(CartContext)
  if (!ctx) throw new Error('useCart needs <CartProvider>')
  return ctx
}
```

// Anywhere, at any depth, with no prop drilling:

```
const { count } = useCart()
```

Note: Provide once, read anywhere with useContext — no more prop drilling.

useReducer and custom hooks

Think of it like... A reducer is a vending machine — press a labelled button (an action) and get a predictable new state, every time.

- useReducer suits state with several related actions, like a shortlist's add, remove and clear.
- A reducer is a pure function (state, action) => newState — same inputs always give the same output.
- It must build and return new state, never mutate the state it was handed.
- A custom hook is a function starting with 'use' that bundles hook logic for reuse.
- Custom hooks share LOGIC, not state — each caller of useLocalStorage gets its own key and value.

```
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD':
      return state.some((c) => c.id === action.course.id)
        ? state
        : [...state, action.course]
    case 'REMOVE':
      return state.filter((c) => c.id !== action.id)
    default: return state
  }
}
```

```
}  
  
}
```

Note: Reducers are pure (state, action) => newState; custom hooks share logic, not state.

Deep Dive — State is a snapshot, and updates are immutable

The deepest source of confusion with `useState` is expecting the state variable to change the moment you set it. It does not. Within a single render, a state value is a snapshot — a photograph taken when that render began — and it stays frozen at that value for the whole render, no matter how many times you call the setter. This is why `setCount(count + 1)` called twice in a row only increments once: both calls read the same frozen count and both schedule it to become that same value plus one. When the next value depends on the previous, you must use the updater form, `setCount(c => c + 1)`, which hands React a function that receives the latest pending value rather than the stale snapshot. Two updater calls correctly increment by two, because each builds on the result of the last.

The second half of using state correctly is immutability. React decides whether to re-render by checking whether the new state is a different reference from the old one — not whether its contents differ. So `items.push(course)` is invisible to React: it mutates the same array, the reference is unchanged, and nothing re-renders. Mutating state is like editing the original photograph; React only reacts when you hand it a brand-new print. You create that new reference with the spread operator: the real `CartContext` adds to the shortlist with `setItems(prev => [...prev, course])` — a fresh array — and updates an object the same way with `{ ...course, fee: 720 }`. This is why immutable updates underpin every single state update you will write; a stray `.push()` is one of the quiet bugs to watch for in generated code, because the array really does change but the screen never does.

- A state value is a snapshot, frozen for the whole render.
- Use `setX(prev => ...)` whenever the next value depends on the previous one.
- React re-renders on a new reference, not on changed contents.
- Build new arrays/objects with spread; never push or mutate state in place.

```
// src/context/CartContext.jsx (real code)  
  
setItems((prev) =>  
  prev.some((c) => c.id === course.id)  
    ? prev  
    : [...prev, course]) // NEW array -> re-renders  
  
// items.push(course) // same array -> nothing renders
```

Deep Dive — useEffect: dependencies, cleanup and StrictMode

`useEffect` exists for side effects — the things that reach outside React's render, such as fetching data, setting up a subscription, starting a timer, or touching the DOM directly. Its behaviour is governed entirely by its second argument, the dependency array, which acts like a guest list: the effect re-runs only when a value on the list actually changes. An empty array means it runs once after the first render and never again — how `ThemeContext` applies the saved theme on boot. Listing `[theme]` means it re-runs whenever the theme changes, which is how the toggle updates `<html data-theme>`. Listing `[slug]` is how a detail page refetches when the user navigates to a different course. Omitting the array entirely means it runs after every render, which is almost always a mistake and a common cause of infinite fetch loops in generated code.

The other half of `useEffect` is cleanup, and it is the part AI code forgets most often. Any effect that starts something ongoing must also stop it, or you leak. A timer keeps ticking after the component is gone; an event listener keeps firing; a subscription keeps receiving. You prevent this by returning a cleanup function from the effect — React runs it before the next effect and when the component unmounts. The real `useDebounce` hook shows the pattern in miniature: it sets a timeout and returns `() => clearTimeout(id)`, so every keystroke cancels the previous pending timer and only the last one ever fires. The mental image is turning the tap off as you leave the room: whatever you turned on, turn off in the returned function.

This is where `StrictMode` enters, and why it confuses people. In development, React's `StrictMode` deliberately mounts each component, immediately unmounts it, and mounts it again. It does this to expose effects with no cleanup: if your effect sets a timer and never clears it, `StrictMode`'s double-invoke leaves two timers running and you notice. So a console message that logs twice in development is not a bug — it is React holding up a mirror to a missing cleanup. Write the cleanup, and the double-invoke becomes harmless. In production `StrictMode` does not double-invoke, but by then your effects are already correct.

- The dependency array controls when the effect re-runs: `[]` once, `[dep]` on change, none every render.
- Return a cleanup function to tear down timers, listeners and subscriptions.
- `StrictMode` double-invokes effects in development to expose missing cleanups.
- A double log in dev usually means an effect you forgot to clean up.

```
// src/hooks/useDebounce.js (real code)

useEffect(() => {

  const id = setTimeout(() => setDebounced(value), delay)

  return () => clearTimeout(id)    // cleanup

}, [value, delay])
```

Deep Dive — useRef: two jobs, and the dividing line against state

`useRef` returns a small object with a single property, `current`, and it has two defining characteristics. Like `state`, the value survives across renders — React hands you back the same ref object every render, so whatever you stored in `.current` is still there. Unlike `state`, changing `.current` does not trigger a re-render. That combination is the whole point: a ref is a sticky note the renderer never reads. It gives you exactly one clean decision to make, and learning to make it is a real mark of understanding React: should changing this value redraw the screen? If yes, it is `state`. If no, it is a ref. The search text in `CoursesPage` is `state`, because every keystroke must repaint the filtered results. Which DOM node the search box is, and whether it is focused, is a ref, because focusing it changes nothing that is rendered.

That dividing line has a sharp, practical edge, and it is worth stating as a warning: a value that changes often but should not repaint the screen is a ref, and putting it in `state` instead is a real performance bug. The classic example is the id returned by `setInterval` or `setTimeout`. You need to keep it so you can clear the timer later, but it should never cause a render — and if you stored it in `state`, the component would re-render every time the timer id changed, which for a per-second tick means re-rendering the whole component every second for nothing. A plain variable cannot do the job either, because it is reset on every render. A ref threads the needle: it remembers the value across renders, and touching it costs nothing.

The ref's second job is to be the sanctioned escape hatch to the real DOM. Most of the time you never touch the DOM in React; you describe UI as a function of `state` and let React do the rest. But a few things genuinely require a live DOM node, and 'is focused' or 'scroll to here' are browser state that no JSX can express. The Cook & Bake app uses this in three real places. `CoursesPage` creates a ref, hands it to `SearchBar` as an ordinary prop, and calls `searchInput.current?.focus()` in a mount effect so the cursor lands in the search box — and again after every category chip, so the user can keep typing. `HomePage` points a ref at the course grid and gives the hero an `onBrowse` function that calls `popularRef.current?.scrollIntoView`, so 'Browse courses' smoothly scrolls down; note that `Hero` receives only a function and never knows a DOM node exists. And a timer id, as above, lives in a ref. Two details matter: `.current` is null on the first render because React has not created the node yet, and is filled right after paint, which is why you read it inside an effect; and in React 19 ref is now an ordinary prop, so you pass it straight through to your own components with no `forwardRef` wrapper. The discipline to keep is that refs are the escape hatch, not the main road: reach for a DOM ref only for the handful of things `state` genuinely cannot express.

- A ref persists across renders like `state`, but changing it never triggers a re-render.
- The rule: should changing it redraw the screen? Yes = `state`; no = ref.
- Job one: a mutable box for values you must remember but must not repaint on — like a timer id.
- Job two: a live DOM handle for focus, scroll or measurement, read inside an effect after mount.

- A timer id in state re-renders on every tick; React 19 makes ref a normal prop — no forwardRef.

```
// src/pages/CoursesPage.jsx (focus) · HomePage.jsx (scroll)

const searchInput = useRef(null) // a DOM handle
useEffect(() => { searchInput.current?.focus() }, [])

<SearchBar inputRef={searchInput} ... /> // ref as a prop

const popularRef = useRef(null)

const scrollToCourses = () =>
  popularRef.current?.scrollIntoView({ behavior: 'smooth' })

<Hero onBrowse={scrollToCourses} />

<section ref={popularRef}>...</section>
```

Deep Dive — Context, reducers and custom hooks

Context solves prop drilling: the tedious threading of a value through layers of components that do not use it, purely to reach a deep child. Context is the building's PA system compared to passing a note desk to desk down every floor — you provide a value once, high in the tree, and any component below reads it directly with `useContext`, skipping every layer in between. It is dependency injection for React. Cook & Bake has three contexts, each wrapped around App in main.jsx: ThemeContext holds the light/dark choice, AuthContext holds the signed-in user, and CartContext holds the shortlist. Each exposes a custom hook — `useTheme`, `useAuth`, `useCart` — that wraps `useContext` and throws a clear error if used outside its provider, so Navbar can read the shortlist count with a single line and no prop drilling. The one caution is that every consumer re-renders when the context value changes, so context is wrong for high-frequency updates like keystrokes; it shines for relatively stable shared state like the current user, theme, or shortlist.

For state with several related operations, `useReducer` is often clearer than juggling many `useState` calls. A reducer is a pure function of the form `(state, action) => newState` — a vending machine where each labelled button (an action like `{ type: 'ADD', course }`) yields a predictable new state, and, being pure, it must return new state rather than mutating what it was given. The shortlist's add, remove and clear map naturally onto a reducer's cases, each returning a fresh array with the spread operator.

Finally, a custom hook is any function named with a leading 'use' that packages hook logic for reuse. The Cook & Bake app is full of them: `useLocalStorage` (a `useState` that persists), `useDebounce` (a value that lags behind by a delay), `useCourses` and `useCourse` and `useReviews` (each wrapping a fetch with loading and error state). The

vital thing to grasp is that custom hooks share logic, not state: ThemeContext and CartContext both call useLocalStorage, but each gets its own independent key and value. You are reusing the behaviour, not sharing a single value between them — which is exactly why a custom hook is safe to call from as many components as you like.

- Context provides a value once and reads it anywhere below, ending prop drilling.
- Cook & Bake has three: ThemeContext, AuthContext, CartContext — each with its own custom hook.
- Reducers are pure (state, action) => newState and must return new state, never mutate.
- Custom hooks share logic, not state — two callers of useLocalStorage get independent values.

```
// src/context/CartContext.jsx (real code)

export function useCart() {

  const ctx = useContext(CartContext)

  if (!ctx) throw new Error('useCart must be used inside <CartProvider>')

  return ctx
}
```

```
const { count } = useCart() // any depth, no prop drilling
```

Deep Dive — What a hook is, and why the rules exist

Before you can use hooks well, it helps to know what a hook actually is. A hook is simply a function whose name begins with 'use' and that lets a plain function component tap into React's built-in features: useState gives it memory, useEffect lets it synchronise with the outside world, useContext reads shared values, useRef holds a mutable box. Before hooks existed, only class components could do these things; hooks brought state and lifecycle to ordinary functions, which is why modern React — and every line of the Cook & Bake app — is written with them and never with class components. A hook is a power outlet on the render: call use...() at the top of your function and it plugs into machinery React maintains on the component's behalf.

Now the part that explains every rule you have been told to follow. React does not identify your hooks by name — it identifies them by the order in which they are called. Picture React keeping, for each component instance, an ordered list of memory slots. The first time the component renders, the first useState call claims slot one, the second useState claims slot two, the useRef claims slot three, the useEffect claims slot four, and so on down the list. On every subsequent render React walks that same list in the same order, handing slot one back to the first hook it meets, slot two to the second, and so on. It has no names to match on, only position. This is elegant and fast, but it has an

ironclad requirement: the sequence of hook calls must be exactly the same on every single render.

That single fact is the origin of the Rules of Hooks. If you call a hook inside an if, a loop, or after an early return, then on some renders that call happens and on others it does not — the slots shift, and slot two's state suddenly gets handed to what used to be slot three. The result is state attaching to the wrong hook, corrupt values, and crashes. So the rules are not arbitrary ceremony: call hooks only at the top level of a component or another hook, never conditionally, so the call order can never change; and call them only from React function components or your own custom hooks, because only those run inside React's render machinery. The `eslint-plugin-react-hooks` lint rules catch almost every violation automatically — treat a warning from them as a real bug, never as noise to silence. And note the consequence for custom hooks: because each component that calls a custom hook runs its hook calls in its own slot list, custom hooks share logic, not state — every caller gets its own independent copy.

- A hook is a `use...()` function that lets a function component tap React's memory and lifecycle.
- React tracks hooks by call order, matching each to a memory slot by position, not by name.
- So the call order must be identical every render — no hooks in ifs, loops or after an early return.
- Call hooks only from components or custom hooks; custom hooks share logic, not state.

```
function CoursesPage() {  
  const { courses } = useCourses()    // slot 1  
  const [query, setQuery] = useState('') // slot 2  
  const searchInput = useRef(null)    // slot 3  
  useEffect(() => { /* ... */ }, []) // slot 4  
  // order is identical every render, so slots line up  
}
```

Lab 4.1 — `useState` — State, Snapshots and Immutability

Objective: manage component state correctly with `useState`.

Goal: Cook & Bake Academy can be browsed but not USED. The learner adds a course shortlist owned by App, meets the Rules of Hooks, and learns why state is a snapshot, why the updater form exists, and why mutating an array never re-renders.

What you'll build

A course shortlist owned by App, with Add / Remove buttons on every course card and a running S\$ total (Tech & files: useState, the Rules of Hooks, immutable updates, lifting state up.)

Step-by-step

71. Prompt the agent for the shortlist, then read every line it writes

```
// Vibe prompt: 'In src/App.jsx add a shortlist of Cook & Bake courses with  
// useState. Add/remove a course by id, show the count and the total fee in  
// S$.'
```

```
// Update state immutably – never push into the existing array.'
```

72. Put the state in the closest common ancestor of everything that needs it — App

```
const [shortlist, setShortlist] = useState([])
```

73. Learn the Rules of Hooks: top level only, and only inside a component or another hook

```
// ⚠️ if (user) { useState(...) } – React tracks hooks by CALL ORDER
```

74. Add a course immutably — build a NEW array, never push into the old one

```
setShortlist((prev) => [...prev, course]) // ⚠️ shortlist.push(course)  
does not re-render
```

75. Remove a course with filter, and guard against adding the same course twice

```
setShortlist((prev) => prev.filter((c) => c.id !== course.id))
```

76. See the snapshot: calling setCount(count + 1) twice in one handler only increments once

```
setCount(count + 1); setCount(count + 1) // ⚠️ +1, not +2 – count is a  
snapshot
```

77. Fix it with the updater form, which always receives the latest value

```
setCount((c) => c + 1); setCount((c) => c + 1) // ✅ +2
```

78. Derive the S\$ total during render — never mirror it into a second useState

```
const total = shortlist.reduce((sum, c) => sum + Number(c.fee), 0)
```

79. Pass the shortlist and its handlers down to CourseCard as props — this is 'lifting state up'

```
<CourseCard course={c} onAdd={add} onRemove={remove} />
```

Test it

Clicking 'Add to shortlist' on Artisan Sourdough (S\$680) adds it and the button flips to Remove; the navbar count and the S\$ total both update; adding the same course twice does nothing.

Watch out for

``cart.push(id)`` mutates the array and does not re-render. Build a new one: ``[...cart, id]``. And when the next state depends on the previous, use the updater form ``setCart(prev => ...)``.

Lab 4.2 — `useEffect` — Side Effects and Cleanup

Objective: synchronise a component with the outside world and clean up after it.

Goal: The shortlist is wiped on every refresh. The learner writes three effects — a document title, a `localStorage` save, and an enrolment-deadline countdown — and learns the dependency array, the cleanup function, and why `StrictMode` runs effects twice.

What you'll build

A live browser-tab title, a shortlist persisted to `localStorage`, and a countdown to the next intake (Tech & files: `useEffect`, dependency arrays, cleanup functions, `StrictMode`, `localStorage`.)

Step-by-step

80. Sync the browser tab title with the shortlist count

```
useEffect(() => {  
  document.title = `Cook & Bake Academy (${shortlist.length})`  
}, [shortlist])
```

81. Learn the three dependency-array forms: none (every render), `[]` (once on mount), `[deps]` (on change)

```
useEffect(fn)           // every render  
useEffect(fn, [])       // once  
useEffect(fn, [q])      // when q changes
```

82. Persist the shortlist on every change, and restore it with LAZY initial state so `localStorage` is read once

```
const [shortlist, setShortlist] = useState(  
  () => JSON.parse(localStorage.getItem('cart')) ?? [],
```

)

83. Start the intake countdown with `setInterval` — and RETURN a cleanup function that clears it

```
useEffect(() => {  
  const id = setInterval(tick, 1000)  
  return () => clearInterval(id)  
}, [])
```

84. Delete the `clearInterval` and watch the countdown tick twice per second: `StrictMode` ran the effect twice

85. Understand why that double-invoke is a feature — it exposes exactly the effects that never clean up

86. Read `src/hooks/useDebounce.js` — the same shape: a timer created in the effect, cancelled in the cleanup

```
useEffect(() => {  
  const id = setTimeout(() => setDebounced(value), delay)  
  return () => clearTimeout(id)  
}, [value, delay])
```

87. Meet the anti-pattern: never use an effect to compute state you could derive during render

```
// ❌ useEffect(() => setTotal(sum(shortlist)), [shortlist])  
// ✅ const total = shortlist.reduce(...) — just compute it
```

Test it

The tab title tracks the shortlist, the shortlist survives a page refresh, the intake countdown ticks exactly once per second, and removing the `clearInterval` visibly doubles its speed.

Watch out for

An effect that starts a timer, a subscription or a listener must return a cleanup function that stops it. `StrictMode` runs effects twice in development on purpose, to make the missing cleanup obvious.

Lab 4.3 — `useRef` — The DOM Escape Hatch, and What Is NOT State

Objective: use refs for real DOM work and for values that must not trigger a render.

Goal: Two things the Cook & Bake catalogue needs are impossible to describe in JSX: putting the cursor in the search box, and scrolling the page to the course grid. Both are browser state, not UI description. The learner uses useRef for both, then draws the dividing line between a ref and state and proves it with a timer.

What you'll build

A search box that focuses itself on mount and after every filter chip (CoursesPage), and a hero 'Browse courses' button that smooth-scrolls to the grid (HomePage) (Tech & files: useRef, DOM refs, ref as an ordinary prop in React 19, scrollIntoView, focus.)

Step-by-step

88. Learn the dividing line BEFORE you write any code — say it out loud

```
// If changing it should REDRAW the screen -> it is STATE.  
// If changing it should NOT redraw the screen -> it is a REF.  
// Changing ref.current never re-renders. That is the whole difference.
```

89. Prompt the agent for the focus behaviour, and demand refs — not a document.getElementById

```
// Vibe prompt: 'In src/pages/CoursesPage.jsx focus the search input when the  
page mounts, and again after the user clicks a category chip. Use useRef +  
useEffect and pass the ref down to SearchBar as a prop. Do NOT use  
document.getElementById or document.querySelector anywhere.'
```

90. Create the ref in CoursesPage and focus the input on mount — the DOM node exists by the time effects run

```
const searchInput = useRef(null)
```

```
useEffect(() => {  
  searchInput.current?.focus()  
}, [])
```

91. In React 19 `ref` is an ordinary prop on a function component — forwardRef is no longer needed

```
// CoursesPage:  
  
<SearchBar value={query} onChange={setQuery} inputRef={searchInput} />
```

```
// SearchBar.jsx:
```

```
export default function SearchBar({ value, onChange, inputRef }) {  
  return <input ref={inputRef} value={value} ... />  
}
```

92. Put the cursor back in the box after a Bakery / Cooking chip click — an event handler is a legal place to touch a ref

```
const setCategory = (cat) => {  
  setSearchParams(/* ... */)   
  searchInput.current?.focus() // ✅ handler, not render  
}
```

93. Now the second real use: scroll the hero CTA to the course grid in HomePage

```
const popularRef = useRef(null)  
  
const scrollToCourses = () =>  
  popularRef.current?.scrollIntoView({ behavior: 'smooth', block:  
    'start' })
```

```
<Hero onBrowse={scrollToCourses} />
```

```
<section ref={popularRef} className="section" id="courses"> ... </section>
```

94. Ask why neither of these could be state: 'is focused' and 'is scrolled to' are BROWSER state, not UI description

95. Prove the dividing line with a timer id. Put it in state and it re-renders on every single tick

```
// 🚫 const [timerId, setTimerId] = useState(null) - a render per tick,  
for a number nobody displays
```

```
// ✅ const timerId = useRef(null)
```

```
// timerId.current = setInterval(tick, 1000)
```

96. Count renders with a ref to see it: renders.current++ changes the value and re-renders NOTHING

```
const renders = useRef(0)
```

```
useEffect(() => { renders.current++ }) // never read or write a ref
DURING render
```

Test it

The search box is focused the moment /courses loads and again after clicking the 🍪 Bakery chip; the hero button smooth-scrolls to the grid; and you can state the rule — redraw the screen means state, otherwise a ref — and point to a line in the app that follows it.

Watch out for

Changing `ref.current` does not re-render, and you must never read or write a ref during render. In React 19, `ref` is an ordinary prop — `forwardRef` is no longer needed, though most AI-generated code still uses it.

Lab 4.4 — useContext and useReducer — State Without Prop Drilling

Objective: share state across the tree with Context and manage it with a reducer.

Goal: The theme toggle in the navbar and the shortlist count are needed six levels apart. The learner feels prop drilling, then moves the theme and the shortlist into React Context, guards each context with a custom hook that fails loudly, and names the shortlist's actions with a reducer.

What you'll build

src/context/ThemeContext.jsx (dark mode) and src/context/CartContext.jsx (the shortlist), both consumed by Navbar (Tech & files: createContext, useContext, useReducer, provider composition, custom guard hooks.)

Step-by-step

97. Feel the problem first: thread `theme` from App → RootLayout → Navbar → the toggle button as a prop. That is prop drilling

98. Prompt for the theme context — and require the guard hook

```
// Vibe prompt: 'Create src/context/ThemeContext.jsx with a ThemeProvider
that
```

```
// stores light/dark in localStorage and mirrors it onto <html data-theme>.
```

```
// Export a useTheme() hook that THROWS a clear error if it is called
outside
```

```
// the provider.'
```

99. Create the context and provider, persisting the choice with the useLocalStorage hook

```
const ThemeContext = createContext(null)
```

```
export function ThemeProvider({ children }) {  
  const [theme, setTheme] = useLocalStorage('theme', 'light')  
  ...  
}
```

100. The one side effect: mirror the theme onto the <html> element, which index.css already keys its variables off

```
useEffect(() => {  
  document.documentElement.dataset.theme = theme  
}, [theme])
```

101. Guard the consumer so a missing provider fails loudly instead of silently returning undefined

```
export function useTheme() {  
  const ctx = useContext(ThemeContext)  
  if (!ctx) throw new Error('useTheme must be used inside <ThemeProvider>')  
  return ctx  
}
```

102. Move the shortlist into CartContext so Navbar can read the count without a single prop

```
const { count } = useCart() // in Navbar – no props threaded through  
RootLayout
```

103. Convert the shortlist to a reducer with NAMED actions: ADD, REMOVE, CLEAR

```
const [items, dispatch] = useReducer(cartReducer, [])  
dispatch({ type: 'ADD', course })
```

104. Keep the reducer PURE — same input, same output, no fetch, no localStorage, no Date.now()

```
case 'ADD':  
  return state.some((c) => c.id === action.course.id)  
  ? state
```

```
: [...state, action.course]
```

105. Wrap the app in main.jsx. Order matters: routing outside, app state inside

```
<BrowserRouter>

  <ThemeProvider>

    <AuthProvider>

      <CartProvider><App /></CartProvider>

    </AuthProvider>

  </ThemeProvider>

</BrowserRouter>
```

Test it

The 🌙 toggle in the navbar switches the whole site between light and dark and survives a refresh; the 🛒 shortlist count updates from any page with no props threaded through; and calling useCart() outside <CartProvider> throws a clear, named error.

Watch out for

Context is dependency injection, not a state manager. Guard the consumer hook so using it outside its provider throws a clear error rather than silently returning `undefined`.

Lab 4.5 — Custom Hooks — Sharing Logic, Not State

Objective: extract reusable stateful logic into custom hooks.

Goal: The same useState + useEffect pair now appears in three places. The learner extracts useLocalStorage and useDebounce, wires the debounce into the course search so it stops filtering on every keystroke, and kills the near-universal misconception that a custom hook shares state between components.

What you'll build

src/hooks/useLocalStorage.js and src/hooks/useDebounce.js, with the course search debounced to 300 ms (Tech & files: Custom hooks, the 'use' naming rule, useEffect cleanup, localStorage.)

Step-by-step

106. Spot the extractable logic: the theme and the shortlist both do 'read localStorage once, write it on change'

107. Prompt for the extraction, and insist the initial read is lazy

```
// Vibe prompt: 'Extract src/hooks/useLocalStorage.js – a useState that
also

// persists to localStorage. Read the stored value LAZILY (the function
form of

// useState) so we touch localStorage once, on mount. Support the updater
form.

// Then rewrite ThemeContext and CartContext to use it.'
```

108. Write useLocalStorage — a hook is just a function starting with 'use' that calls other hooks

```
export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const stored = window.localStorage.getItem(key)
    return stored !== null ? JSON.parse(stored) : initialValue
  })
  ...
}
```

109. Write useDebounce with a cleanup that cancels the pending timer on every keystroke

```
useEffect(() => {
  const id = setTimeout(() => setDebounced(value), delay)
  return () => clearTimeout(id)
}, [value, delay])
```

110. Wire it into CoursesPage: type instantly, filter 300 ms after typing stops

```
const [query, setQuery] = useState(q)
const debounced = useDebounce(query, 300)
```

111. Kill the misconception: two components calling useDebounce get two INDEPENDENT states. A hook shares LOGIC, not state

```
// Hooks share logic. Context (Lab 4.4) shares state. Different tools.
```

112. Check the AI's work against the rule: a hook may call other hooks, but a plain function must never

113. Line up the hooks folder you now own — every one of them is used by the real app

`src/hooks/useLocalStorage.js`

`src/hooks/useDebounce.js`

Test it

Typing 'sourdough' filters 300 ms after you stop, not on every keystroke; the shortlist and theme still survive a refresh; both hooks live in `src/hooks/`; and you can explain why two components using `useDebounce` do not share a value.

Watch out for

A custom hook shares logic, never state. Two components calling `useDebounce` each get their own independent state — this is the single most common misconception about hooks.

Topic 05 — Giving Your App a Backend API and a Database

Three tiers · Serverless API routes · Neon Postgres · SQL injection · Password hashing · JWT

Until now Cook & Bake Academy has been a pretend app: the 20-course catalogue lived in a JavaScript array and an enrolment vanished when you closed the tab. This topic gives it a real backend — a serverless API of your own over a Postgres database on Neon.

The shape is three tiers: the browser calls your `/api/*` functions, and only those functions talk to Postgres. The connection string lives in the server's environment and never reaches the browser — which is the whole reason it must never carry a `VITE_` prefix, since everything prefixed `VITE_` is compiled into the bundle every visitor downloads.

The network is not your computer. It is slow, it fails, and it answers out of order. Every remote read therefore has three states — loading, error and success — and AI-generated code reliably writes only two of them. And `fetch` does not reject on a 404 or 500; only a network failure rejects, so you must check `response.ok`.

Finally, you will learn where security actually lives. Queries are parameterised tagged templates, so a value can never become SQL. Passwords are stored as bcrypt hashes, never in the clear. And a user's identity comes from the signed JWT the API verifies — never from the request body — with ownership of a row enforced in the SQL itself (and `user_id = ${userId}`). A client-side check is a convenience; the check inside the API route is the boundary.

Key Concepts — Topic 05

- **Three tiers, one rule** — Browser → your API → the database. The browser NEVER talks to Postgres. The connection string lives only on the server.
- **A VITE_ variable is public** — Anything prefixed VITE_ is compiled into the bundle every visitor downloads. DATABASE_URL must never carry that prefix — it holds the password.
- **Every remote read has three states** — Loading, error, success. AI-generated code writes two of them and forgets the error branch.
- **fetch does not reject on 404** — Only a network failure rejects. Check response.ok, or you will parse an error page and render nonsense.
- **Parameterised queries, always** — `sql`... where slug = ${slug}`` sends the value separately from the SQL. String-concatenating user input is how you get dropped tables.
- **Never store a password** — Store a bcrypt hash. If the table leaks, the hashes are useless. Never select the hash into a response.
- **Trust the token, not the body** — The user id comes from the verified JWT — never from the request body. Enforce ownership in the SQL: ``and user_id = ${userId}``.

Concepts Explained — Topic 05

Three tiers, and one hard rule

Think of it like... The API is the counter in a bank — customers state what they want; only staff go into the vault.

- React (browser) --fetch--> /api/* (Vercel functions) --sql--> Neon Postgres. Three tiers.
- The BROWSER NEVER TALKS TO POSTGRES. It has no driver, no connection string and no SQL.
- It knows one thing: how to call our own /api/* URLs. The functions decide what is allowed.
- Every security rule lives in the middle tier, on a server, where the user cannot edit it.
- Anything you enforce in the browser is a suggestion. Anything you enforce in /api/ is a rule.

```
React (browser)          src/lib/api.js
    |  fetch('/api/courses')
    v
/api/* (Vercel function) api/courses/index.js
    |  sql`select ... from courses`
    v
```

Note: The browser asks the API. Only the API touches the database.

Why the browser cannot hold the password

Think of it like... Putting DATABASE_URL in a VITE_ variable is printing your house key on the flyer and posting it to every visitor.

- Vite compiles any VITE_-prefixed variable straight into the JavaScript bundle it ships.
- Every visitor downloads that bundle. They can read it. There is no exception and no hiding it.
- DATABASE_URL contains the database password and grants full READ AND WRITE to your data.
- So DATABASE_URL must NEVER carry the VITE_ prefix. It is read only in api/, via process.env.
- If you ever 'fix' a connection error by renaming it VITE_DATABASE_URL, you have published the vault key.

.env.local – the prefix is the whole security boundary

```
DATABASE_URL=postgresql://user:PASSWORD@ep-...neon.tech/db
```

```
JWT_SECRET=<32 random bytes>
```

NO VITE_ prefix on either. Both are server-only.

// api/_lib/db.js – the ONLY place it is ever read:

```
export const sql = neon(process.env.DATABASE_URL)
```

Note: A VITE_ variable is public. DATABASE_URL must never have that prefix.

A serverless API route is a function

Think of it like... Each file in api/ is a shopfront window: one URL, one handler, opened only when someone knocks.

- A file in api/ becomes a URL: api/courses/index.js serves GET /api/courses.
- It exports one default async handler(req, res) — no Express app, no server to keep running.

- Square brackets make it dynamic: `api/courses/[slug].js` matches `/api/courses/macaron-masterclass`.
- Vercel hands you the matched segment as `req.query.slug`, and the query string as `req.query` too.
- The status code IS the answer: 200 for a course, 404 for no such slug, 401 for no valid token.

```
// api/courses/[slug].js (real code)

import { sql } from '../_lib/db.js'

export default async function handler(req, res) {
  const { slug } = req.query ?? {}

  const rows = await sql`
    select id::int as id, code, slug, title, fee::float8 as fee
    from courses where slug = ${slug}
  `

  if (rows.length === 0) throw new HttpError(404, 'No course.')

  return res.status(200).json(rows[0])
}
```

Note: One file, one URL, one async handler — and it holds the only DB connection.

Tagged templates defeat SQL injection

Think of it like... A parameterised query is a form with boxes — Postgres reads the form first, then fills the boxes. Text in a box can never become an instruction.

- `sql`...`` is a TAGGED TEMPLATE. You call it with backticks, not brackets — and that is the whole story.
- It LOOKS like string interpolation but is not: the SQL text and the values travel SEPARATELY.
- The driver sends ``where slug = $1`` plus the value. Postgres parses the SQL, THEN binds the value.
- So a slug of ``x``; `drop table users; --`` is looked up as an absurd literal slug and finds nothing.

- The one thing that reintroduces the hole is building the string yourself. Never do `sql(...${x}...)`.

```
// SAFE – a tagged template. Value travels separately.
await sql`select * from courses where slug = ${slug}`
//           Postgres receives:  where slug = $1
//           and, separately:    ["macaron-masterclass"]

// CATASTROPHIC – never build SQL by concatenation:
await sql(`select * from courses where slug = '${slug}'`)
// slug = "x'; drop table users; --"  now EXECUTES.
```

Note: The value travels separately from the SQL, so it can never be parsed as SQL.

Never store a password

Think of it like... A hash is a fingerprint, not a photograph — you can check a match, but you can never reconstruct the face.

- Store a bcrypt HASH, never the password. Hashing is one-way: the hash cannot be turned back.
- bcrypt is deliberately SLOW (cost 10, ~100ms), which makes brute-forcing a stolen table painful.
- It salts every hash automatically, so two users with the same password get different hashes.
- Verify with `bcrypt.compare()`, which re-hashes with the stored salt and compares in constant time.
- NEVER select `password_hash` into a response. Whitelist the fields you send; do not blacklist.

```
// api/auth/signup.js (real code)
const passwordHash = await bcrypt.hash(password, 10)
await sql`
  insert into users (email, name, password_hash)
  values (${email}, ${name}, ${passwordHash})
```

```

    returning id::int as id, email, name, created_at
  `
  // note what is NOT in RETURNING: password_hash.

// api/auth/login.js
const ok = await bcrypt.compare(password, user.password_hash)

```

Note: Hash with bcrypt, compare with bcrypt, and never let the hash leave the server.

Trust the token, never the body

Think of it like... A JWT is a tamper-proof wristband — the payload is readable by anyone, but only the door staff can make a real one.

- A JWT is header.payload.signature. The payload is ENCODED, not encrypted — never put a secret in it.
- The signature is what makes it trustworthy: it is computed with JWT_SECRET, which only the server has.
- Edit one byte of the payload and the signature no longer matches, so jwt.verify() throws.
- So the user id comes from the VERIFIED token's `sub` claim — NEVER from req.body.userId.
- If POST /api/enrollments trusted req.body.userId, anyone with curl could enrol anyone.

```

// api/_lib/auth.js (real code)
export function requireAuth(req) {
  const header = req.headers?.authorization ?? ''
  if (!header.startsWith('Bearer ')) throw new HttpError(401, '...')
  const token = header.slice('Bearer '.length).trim()
  const payload = jwt.verify(token, JWT_SECRET) // throws if forged
  return Number(payload.sub) // verified, not claimed
}

```

Note: The id comes from a verified signature, not from something the caller typed.

Ownership is enforced in the SQL

Think of it like... An id in the URL is a request, not a permission — asking for locker 5 does not make locker 5 yours.

- An id in the URL comes from the address bar. Anyone can change a 4 to a 5 and ask to delete it.
- So the id alone is never enough. Every statement carries TWO conditions, and the second is ownership.
- `where id = \${id} and user\_id = \${userId}` — the userId comes from the verified JWT, not the caller.
- A row you do not own simply does not match, so Postgres changes nothing and we answer 404.
- Get this wrong and you have an Insecure Direct Object Reference — one of the commonest real bugs.

```
// api/enrollments/[id].js (real code)

const userId = requireAuth(req)           // from the JWT
const { id } = req.query ?? {}            // from the URL

const rows = await sql`
  delete from enrollments
  where id = ${id} and user_id = ${userId}
  returning id::int as id
`
// ^^^^^^^^^^^^^^^^^^^^^^^^^ the real check

if (rows.length === 0) throw new HttpError(404, 'Not found.')
```

Note: The URL says WHICH row; the token says WHOSE. Both go in the WHERE clause.

Every remote read has three states

Think of it like... Every fetch is a package delivery — it is either in transit, lost, or on your doorstep, and the UI must show all three.

- A remote read is never instant, so you must model loading, error and success as distinct states.
- Hold three pieces of state: the data, a loading flag, and an error value.

- Show a skeleton while loading, a message on error, and the data on success — never assume success.
- AI-generated code reliably writes the loading and success branches and forgets the error one.
- Always clear the flag in a finally, or one failed request leaves the UI spinning forever.

```
// src/hooks/useCourses.js (real code)

try {
  const data = await api.get('/courses')
  setCourses(data); setError(null)
} catch (err) {
  setError(err.message)          // the branch AI forgets
} finally {
  setLoading(false)              // or it spins forever
}
```

Note: Model loading, error and success — the error branch is the one AI skips.

fetch does not reject on 404

Think of it like... fetch only cries if the road is out — a 404 is a signed 'not found' slip delivered successfully, so you must read the slip.

- fetch rejects its Promise only on a network failure, not on a 404 or a 500 HTTP response.
- A 404 is a successful delivery of a failure message, so the await resolves perfectly normally.
- Always check response.ok (true only for 200-299) before you read the body.
- Skip it and an error payload like {error: '...'} sails into your UI and renders as if it were a course.
- Carry the status on the thrown error so a 404 (no such course) is distinguishable from a 500.

```
// src/lib/api.js (real code)

const res = await fetch(`${BASE}${path}`, { ... })
```

```

const data = res.status === 204 ? null : await res.json()

if (!res.ok) {

  const err = new Error(data?.error ?? `Failed (${res.status})`)

  err.status = res.status    // 404 -> not found page

  throw err                  // 500 -> error message
}

```

Note: Check `response.ok` before reading the body — `fetch` won't reject on 404.

Deep Dive — Three tiers: why the browser never touches Postgres

Everything in Topic 5 rests on one diagram: React (in the browser) calls `/api/*` (serverless functions on Vercel), and only those functions run SQL against Neon Postgres. Three tiers, and the rule that gives them meaning is that the browser NEVER talks to Postgres. It has no database driver, no connection string, and no SQL anywhere in it. Look at `src/lib/api.js` and you will find no password and no query — only `fetch` calls to our own `/api/*` URLs. Think of the API as the counter in a bank: customers state what they want, but only staff go into the vault, and the customer never sees the combination. This is not architectural fashion; it is the only place security can actually live. Anything you check in the browser, a determined user can bypass by editing the JavaScript or calling the API directly with `curl`. Anything you enforce inside an `/api/` function runs on a server the user cannot touch. So the browser decides WHAT it wants; the server decides WHO is allowed and WHAT is true.

The concrete reason the browser cannot be trusted with a connection is the environment-variable rule from Topic 2, now with teeth. Vite compiles any variable prefixed `VITE_` straight into the JavaScript bundle every visitor downloads — it is public, always, with no exception. `DATABASE_URL` contains the database password and grants full read AND write access to all your data, so it must never carry the `VITE_` prefix. It is read in exactly one place, `api/_lib/db.js`, as `process.env.DATABASE_URL`, which only Node code running on Vercel's servers can see. The single most dangerous 'fix' a beginner or an over-eager agent can make is to rename it `VITE_DATABASE_URL` to make a connection error go away — that one edit publishes full control of your database to every visitor of the site. The same rule protects `JWT_SECRET`: anyone who learns it can mint a valid login token for any user, so it too is server-only.

- React → `/api/*` → Neon. The browser never talks to Postgres; it only calls our own `/api/*` URLs.
- Security can only live on the server: a client check is bypassable with devtools or `curl`.

- A `VITE_` variable is compiled into the public bundle, so `DATABASE_URL` must never carry that prefix.
- `DATABASE_URL` and `JWT_SECRET` are read only in `api/` via `process.env` — never in React code.

```
// api/_lib/db.js (real code)

import { neon } from '@neondatabase/serverless'

// No VITE_ prefix, so Vite never bundles it. Server-only.

export const sql = neon(process.env.DATABASE_URL)

// src/lib/api.js (the browser) holds NO url, NO sql:

await fetch('/api/courses') // that is all the browser knows
```

Deep Dive — Serverless API routes and parameterised SQL

A serverless API route is far simpler than a traditional backend: a file in the `api/` folder becomes a URL, and it exports one default async handler(`req`, `res`). There is no Express app to configure and no server process to keep alive — Vercel runs the function on demand when a request arrives. `api/courses/index.js` serves `GET /api/courses`; putting the filename in square brackets makes the route dynamic, so `api/courses/[slug].js` matches `/api/courses/macaron-masterclass` and hands you the matched segment as `req.query.slug`. The handler's job is to read the request, run a query, and answer with a status code and JSON — and the status code IS the answer: 200 with a course, 404 when no slug matches, 401 when the token is missing or forged. The front-end hooks translate those codes into screens, which is why `useCourse` renders a real 'Course not found' page from a 404 rather than a red error.

The security heart of every route is how it puts values into SQL, and it rests on a JavaScript feature that is easy to miss: the tagged template. When you write `sql`select ... where slug = ${slug}``, you are not calling `sql` with an interpolated string; you are calling it as a tagged template, and that distinction is the whole defence against SQL injection. The driver does not paste the value into the query text. It sends Postgres the query with a numbered placeholder — `where slug = $1` — and sends the value separately, as data. Postgres parses the SQL first, decides what is a keyword and what is a table, and only THEN binds the value into the placeholder. Because the value arrives after parsing is finished, it can never be interpreted as SQL. A slug of `x'; drop table users; --` is simply looked up as an absurd literal string that matches no course; it cannot execute. The one thing that would reintroduce the vulnerability is building the query string yourself with ordinary interpolation — `sql(`... where slug = '${slug}'`)` — which pastes the attacker's text straight into the SQL. The Cook & Bake codebase never does this: every query is a tagged template, there is no string-built SQL and no

sql.unsafe anywhere, deliberately, so there is no door to walk user input through. One more practical note that bites everyone once: Postgres returns bigint and numeric columns as strings, so the queries cast explicitly — `id::int`, `fee::float8` — and the browser receives real numbers, which keeps `course.id === enrollment.course_id` from silently comparing 1 with '1'.

- A file in `api/` is a URL exporting one async handler; `[slug]` makes it a dynamic route.
- The status code is the answer: 200 a course, 404 no such slug, 401 no valid token.
- `sql`... ${slug}`` is a tagged template — the value travels separately and is never parsed as SQL.
- Never build SQL by string concatenation; cast ids with `::int` so numbers compare correctly.

```
// api/courses/[slug].js (real code)

export default async function handler(req, res) {

  const { slug } = req.query ?? {}

  const rows = await sql`

    select id::int as id, code, slug, title, fee::float8 as fee

    from courses where slug = ${slug}

  `

  if (rows.length === 0) throw new HttpError(404, 'No course.')

  return res.status(200).json(rows[0])

}

// Postgres receives: where slug = $1 + ["..."]
```

Deep Dive — Passwords, tokens, and enforcing ownership in SQL

Authentication and authorisation are where a hobby app becomes a real one, and the Cook & Bake backend does both in a handful of small, careful functions. Start with passwords, and the iron rule: never store one. When a user signs up, `api/auth/signup.js` runs `bcrypt.hash(password, 10)` and stores only the resulting hash. Hashing is one-way — you cannot turn a hash back into the password — and `bcrypt` is deliberately slow, so brute-forcing a stolen table is painfully expensive; it also salts every hash automatically, so two users who happen to pick the same password still get different hashes. Just as important is what leaves the function: the INSERT's RETURNING list is `id`, `email`, `name`, `created_at`, and `password_hash` is deliberately not in it. The hash never appears in a response body or a log line. Login mirrors this: `api/auth/login.js` selects `password_hash` into a local variable — the only place it is ever read — and verifies the attempt with `bcrypt.compare()`, which re-hashes with the stored salt and compares in constant time. It

returns one identical error for both 'no such email' and 'wrong password', so an attacker cannot use the response to discover which emails have accounts.

On success, login and signup issue a JSON Web Token. A JWT is three base64 chunks — header.payload.signature — and the crucial thing to understand is that the payload is only ENCODED, not encrypted; anyone can read it, so it must never contain a secret. What makes it trustworthy is the signature, computed with JWT_SECRET, which only the server knows. Change a single byte of the payload — say, the sub claim, which holds the user's id, from your id to someone else's — and the signature no longer matches, so jwt.verify throws. This is the entire basis of the most important rule in the backend: the user id ALWAYS comes from the verified token, and NEVER from the request body. requireAuth(req) in api/_lib/auth.js reads the Authorization: Bearer header, verifies the signature, and returns Number(payload.sub) — an id the caller cannot forge. If POST /api/enrollments trusted req.body.userId instead, anyone with curl could enrol anyone; because it reads the id from the token, a request can only ever act as the person who holds that token.

Verifying WHO is calling is only half of authorisation; the other half is making sure they can only touch their OWN rows, and in this app that check lives in the SQL itself. Consider DELETE /api/enrollments/:id. The id comes from the address bar, so a caller can change a 4 to a 5 and ask to delete enrollment 5 — which may belong to another student. The id alone must therefore never be enough. Every statement carries two conditions: where id = \${id} and user_id = \${userId}, where userId is the verified value from the token. A row the caller does not own simply does not match, so Postgres deletes nothing and the route answers 404. There is no Row Level Security behind this doing the work invisibly — this WHERE clause IS the access control, and forgetting the second condition is the classic Insecure Direct Object Reference, one of the most common real-world API vulnerabilities. The same pattern secures enrolling (the row is inserted with user_id = \${userId}, never a client-supplied id) and reviews (the UPSERT can only ever touch the caller's own review). Read as one sentence: the URL says WHICH row, the token says WHOSE, and both go into the WHERE clause.

- Never store a password: bcrypt.hash on signup, bcrypt.compare on login; the hash never leaves the server.
- Whitelist response fields (RETURNING id, email, name...) so password_hash cannot leak.
- A JWT's payload is only encoded — the signature (JWT_SECRET) is what makes it trustworthy.
- The user id comes from the verified token, never req.body; requireAuth returns an id you cannot forge.
- Enforce ownership in SQL: `where id = \${id} and user\_id = \${userId}` — the WHERE clause IS the access control.

```
// api/enrollments/[id].js (real code)
```

```
const userId = requireAuth(req) // verified JWT 'sub'
```

```
const { id } = req.query ?? {} // from the URL

const rows = await sql`
  delete from enrollments
  where id = ${id} and user_id = ${userId}
  returning id::int as id
`

if (rows.length === 0) throw new HttpError(404, 'Not found.')
```

Deep Dive — The three states of a fetch, and response.ok

Once the data lives behind a network call, the front end has to cope with the fact that a read is never instant and can fail. Any remote read is in one of three states — loading, error, or success — and you must model all three explicitly, usually as the data itself, a loading flag and an error value. While loading, show a skeleton; on error, show a message; on success, render the data. This is precisely where AI-generated code fails in a recognisable way: it reliably writes the loading and success branches and forgets the error branch entirely, which is exactly why a real app spins forever on a failed request instead of telling the user something went wrong. The real useCourses hook models all three, and — just as important — clears the loading flag in a finally block, so a failure stops the spinner instead of leaving it turning forever. Treat a fetch as a package delivery that is either in transit, lost, or on your doorstep, and make sure the UI can render all three.

There is a specific browser trap that magnifies this, and the Cook & Bake api wrapper exists to handle it once for everyone. fetch only rejects its Promise on a genuine network failure — the road being out. A 404 or a 500 is a response that arrived successfully; it is a signed 'not found' slip, delivered to your door. So the await resolves normally, and if you go straight to response.json() you will happily parse an error page and render nonsense. The guard is to check response.ok, which is true only for status codes in the 200s, before reading the body — and to throw if it is false, so the catch block runs and the error state is set. src/lib/api.js does this in one place and, cleverly, carries the status code onto the thrown error (err.status = res.status). That lets each hook tell the difference between failures that look the same to fetch but mean different things: useCourse turns a 404 into 'course not found' with no error screen, but shows the red error for a 500. Two different failures, two different screens — collapsing them is how you end up showing 'Something went wrong' to a user who simply mistyped a URL.

- Model loading, error and success as three explicit states, and clear the flag in a finally.
- AI code reliably writes loading and success and forgets error — the branch that stops the spinner.

- fetch rejects only on network failure — a 404 resolves normally, so check response.ok yourself.
- Carry the status on the thrown error so a 404 (not found) is distinguishable from a 500 (broke).

```
// src/lib/api.js (real code)

const data = res.status === 204 ? null : await res.json()

if (!res.ok) {
    // fetch won't do this
    const err = new Error(data?.error ?? `Failed (${res.status})`)
    err.status = res.status // 404 vs 500
    throw err
}

return data
```

Lab 5.1 — Three Tiers — A Neon Database and Your First API Route

Objective: stand up Neon Postgres and put a serverless API tier in front of it.

Goal: The 20 courses live in a JavaScript array. The learner creates a Neon Postgres project, runs the real schema, and writes the first Vercel serverless function — GET /api/courses — reading it. The architectural rule is drilled here: the browser NEVER talks to Postgres, and DATABASE_URL must never carry a VITE_ prefix, because a VITE_ variable is compiled into the bundle every visitor downloads.

What you'll build

A live Neon database with users, courses (20 seeded), enrollments and reviews — plus api/_lib/db.js and api/courses/index.js (Tech & files: Neon Postgres, @neondatabase/serverless, Vercel Functions, vercel dev, .env.local.)

Step-by-step

114. Draw the three tiers before you write a line — this is the whole topic in one diagram

```
React (browser) --fetch--> /api/* (Vercel serverless) --sql--> Neon
Postgres

no password holds DATABASE_URL the data
```

115. Create a free Neon project, copy the connection string, and run the real schema against it

```
psql "$DATABASE_URL" -f neon/schema.sql
```

```
# users, courses (20 seeded: BAK-1xx bakery, CUL-2xx cooking), enrollments, reviews
```

116. Put the connection string in `.env.local` with NO `VITE_` prefix — and understand exactly why

```
# ✅ server-only: Vite never touches it, only process.env in api/ can read it
```

```
DATABASE_URL=postgresql://USER:PASSWORD@ep-xxx.aws.neon.tech/neondb?sslmode=require
```

```
# ❌ VITE_DATABASE_URL=... would publish your database password – read AND write – to every visitor of your site. There is no exception to this.
```

117. Install the driver and the Vercel CLI, then run the API and the front end together

```
npm install @neondatabase/serverless
```

```
npm i -g vercel && vercel dev # serves /api/* AND the Vite app
```

118. Write `api/_lib/db.js` — the ONE place in the codebase that holds a connection to Postgres

```
import { neon } from '@neondatabase/serverless'
```

```
if (!process.env.DATABASE_URL) {  
  throw new Error('DATABASE_URL is not set. See .env.example.')  
}
```

```
export const sql = neon(process.env.DATABASE_URL)
```

119. Prompt the agent for the first route — a Vercel function is a file that default-exports `(req, res)`

```
// Vibe prompt: 'Create api/courses/index.js – a Vercel serverless function.'
```

```
// GET only. Read all courses from Neon with the tagged-template sql from
```

```
// api/_lib/db.js and return them as JSON. Support optional ?category= and ?q=
```

```
// filters. Cast id::int and fee::float8. Reject any other HTTP method with 405.'
```

120. Read what it wrote: a Vercel function is just a default-exported handler, and the FILE PATH is the URL

```
export default async function handler(req, res) {  
  const courses = await sql`select id::int as id, code, title, fee::float8  
as fee ... from courses order by id`  
  
  return res.status(200).json(courses)  
}  
  
// api/courses/index.js -> GET /api/courses
```

121. Cast your types. Postgres bigint and numeric arrive in JavaScript as STRINGS, and "1" !== 1 silently

```
id::int as id, fee::float8 as fee -- skip these and course.id ===  
enrollment.course_id is always false
```

122. Hit the API directly with curl — no browser, no React. The API is a product in its own right

```
curl http://localhost:3000/api/courses | head -c 400  
  
curl 'http://localhost:3000/api/courses?category=Bakery'
```

123. Prove the boundary: build the front end and grep the bundle for the password. Nothing may come back

```
npm run build && grep -r "postgresql://" dist/ # must return NOTHING
```

Test it

curl /api/courses returns all 20 Cook & Bake courses as JSON with numeric ids and fees; ?category=Bakery returns 10; a POST returns 405; and grepping dist/ for 'postgresql://' returns nothing.

Watch out for

The browser never talks to Postgres. `DATABASE\_URL` lives only in the serverless function's environment — and it must NEVER carry a `VITE\_` prefix, because every `VITE\_` variable is compiled into the bundle each visitor downloads. Prove it to yourself: `grep` the built `dist/` for the value.

Lab 5.2 — Parameterised SQL — Dynamic Routes and an Injection Probe

Objective: write dynamic API routes whose SQL cannot be injected.

Goal: Every course now needs its own URL. The learner adds the dynamic route `api/courses/[slug].js`, learns that Neon's `sql`` tagged template sends the VALUE separately from the SQL, and then attacks their own API with a real injection payload to watch it bounce — before deliberately writing the unsafe version and seeing the difference.

What you'll build

`api/courses/[slug].js` — one course by slug, a real 404 when there is none — and a security probe you run yourself (Tech & files: Vercel dynamic routes, tagged templates, parameterised queries, HTTP status codes.)

Step-by-step

124. Add the dynamic route. The `[slug]` in the FILENAME is what makes it dynamic; Vercel hands you `req.query.slug`

```
// api/courses/[slug].js -> GET /api/courses/artisan-sourdough-bread-baking

const { slug } = req.query ?? {}
```

125. Query with the tagged template — note there are NO parentheses. `sql`...``, not `sql("...")`

```
const rows = await sql`
  select id::int as id, code, slug, title, category, level,
         fee::float8 as fee, weeks, campus, summary, emoji, image
  from courses
  where slug = ${slug}
`
```

126. Understand WHY that is safe. It looks like string interpolation and is not

```
// The driver sends the query TEXT and the VALUES to Postgres separately,
as a

// parameterised query ($1). Postgres parses the SQL FIRST and only then
binds

// the value. A value can therefore never become SQL. Injection is
impossible.
```

127. PROBE IT. Attack your own API with a classic payload and watch it do nothing at all

```
curl "http://localhost:3000/api/courses/x'%20or%20'1'='1"
```

```
# -> 404 {"error":"No course found at /courses/x' or 'l'='l."}

# Postgres looked for a course whose slug is literally x' or 'l'='l - and
there isn't one.
```

128. Now write the version that is wrong, and see the difference in ONE line of code

```
// 🛑 NEVER. This builds a STRING, so the payload becomes part of the SQL:
// await sql(`select * from courses where slug = '${slug}'`)
// slug = "x'; drop table users; --" -> you no longer have a users
// table.
```

129. Delete the unsafe line. The rule: user input is always a `{}` placeholder, never part of the query string

130. Return a real 404 when no row matches — the STATUS CODE is the API's answer, not a 200 with null

```
if (rows.length === 0) {
  throw new HttpError(404, `No course found at /courses/${slug}.`)
}
```

131. Prompt the agent for the shared error plumbing so every route answers the same way

```
// Vibe prompt: 'In api/_lib/auth.js add an HttpError class (status +
message),

// a sendError(res, err) that maps HttpError to its status and ANY other
error to

// a generic 500 (never leak a stack trace or a SQL error naming our
columns),

// and requireMethod(req, ...allowed) that throws 405.'
```

132. Audit the whole api/ folder for the one thing that reopens the door

```
grep -rn "sql(\\`\\`|sql.unsafe" api/      # must return NOTHING – every query
is a tagged template
```

Test it

GET /api/courses/artisan-sourdough-bread-baking returns one course; GET /api/courses/not-a-course returns 404 with a clear message; the injection payload returns a harmless 404 and the users table is still there; and grep finds no string-built SQL anywhere in api/.

Watch out for

``sql`... where slug = ${slug}`` is a tagged template, not string interpolation: the driver sends the SQL text and the value to Postgres separately, so Postgres parses the query before it ever sees the value. A value can therefore never become SQL. The moment you write ``sql(`... `${slug}`)`` with parentheses, you have reopened the door — grep your ``api/`` folder for it.

Lab 5.3 — Calling the API from React — Loading, Error and Success

Objective: consume your own API from React with all three states handled.

Goal: The API works; the app still renders a hardcoded array. The learner writes the `src/lib/api.js` fetch wrapper — including the check every AI forgets, `response.ok`, because `fetch` does NOT reject on a 404 — then replaces the array with the `useCourses` and `useCourse` hooks and renders loading, error and success for real.

What you'll build

`src/lib/api.js` and `src/hooks/useCourses.js` + `useCourse.js` — the catalogue now comes from Postgres (Tech & files: `fetch`, `response.ok`, HTTP status, `useState` + `useEffect`, skeleton and error UI.)

Step-by-step

133. Write the wrapper — the browser's ONLY way to reach the server. Note what is NOT in it: no SQL, no password

```
const BASE = '/api'
```

```
async function request(method, path, body) {  
  const res = await fetch(`${BASE}${path}`, { method, ... })  
  ...  
}
```

134. THE line AI-generated `fetch` code always omits: `fetch` does NOT reject on 404 or 500

```
if (!res.ok) {  
  const err = new Error(data?.error ?? `Request failed (${res.status})`)  
  err.status = res.status // carry the status so the caller can tell 404  
  from 500  
  throw err
```

```
}
```

// Without this, an error payload {error: "..."} sails into the UI and is rendered as a course.

135. Prompt for the hook, and name all three states in the prompt so the agent cannot skip one

```
// Vibe prompt: 'Write src/hooks/useCourses.js. Load GET /api/courses through
```

```
// src/lib/api.js inside a useEffect. Return { courses, loading, error }. Clear
```

```
// loading in a finally so a failed request does not leave the skeleton forever.
```

```
// Guard against setting state after unmount with an `active` flag in the cleanup.'
```

136. Read the result against the three states — loading, error, success. Miss one and the UI feels broken

```
const [courses, setCourses] = useState([])
```

```
const [loading, setLoading] = useState(true)
```

```
const [error, setError] = useState(null)
```

137. Check the finally and the cleanup — the two things a generated hook usually lacks

```
} finally {
```

```
  if (active) setLoading(false) // no finally = a stuck skeleton forever
```

```
}
```

```
return () => { active = false } // no guard = setState on an unmounted component
```

138. Render all three in CoursesPage and HomePage — skeletons, a red error line, then the grid

```
{loading && <div className="grid">[[1,2,3,4,5,6].map(n => <div key={n} className="skeleton" />)}</div>}
```

```
{error && <p className="error">{error}</p>}
```

```
{!loading && !error && <CourseGrid courses={visible} />}
```

139. Write useCourse(slug) and make it distinguish the two failures — a 404 is NOT a crash

```

if (err.status === 404) {
  setCourse(null)    // -> the 'Course not found' page
  setError(null)
} else {
  setError(err.message)    // -> the red 'Something went wrong'
}

```

140. Delete the import of `src/data/courses.js` from the pages. The catalogue is now Postgres

141. Prove it end to end: change a fee in the Neon table editor and refresh the browser

```
update courses set fee = 720 where code = 'BAK-101';
```

142. Test the error branch on purpose — throttle the network, then point BASE at a bad path

```
// DevTools -> Network -> Slow 3G (see the skeletons)
```

```
// const BASE = '/apix' (see the error line, not a blank page)
```

Test it

The catalogue renders from Postgres after a skeleton; editing a fee in Neon changes the page on refresh; `/courses/not-a-course` shows 'Course not found' (not a red error); and breaking the API base URL shows the error message instead of an endless skeleton.

Watch out for

``fetch`` does not reject on a 404 or a 500; only a network failure rejects. Check ``response.ok``. Every remote read has three states — loading, error, success — and AI-generated code reliably writes two of them and forgets the error branch. Never make the ``useEffect`` callback itself ``async``: it would return a Promise where React expects a cleanup function.

Lab 5.4 — Auth — bcrypt, JWT, and Ownership Enforced in SQL

Objective: hash passwords, issue and verify JWTs, and let only the owner touch a row.

Goal: Students must be able to sign up, sign in, enrol and review. The learner builds `api/auth/*` with bcrypt hashing and JWT signing, then the enrolments API — where the security rule of the whole course lands: the user id comes from the VERIFIED TOKEN, never from the request body, and ownership is enforced in the WHERE clause, not in an if statement.

What you'll build

api/auth/signup.js, login.js, me.js, api/_lib/auth.js, the enrollments + reviews routes, and src/context/AuthContext.jsx (Tech & files: bcryptjs, jsonwebtoken, JWT_SECRET, Authorization: Bearer, insecure direct object references.)

Step-by-step

143. Add the second server-side secret — again, NO VITE_ prefix. Anyone holding it can mint a token for any user

```
JWT_SECRET=$(node -e
"console.log(require('crypto').randomBytes(32).toString('hex'))")

npm install bcryptjs jsonwebtoken
```

144. NEVER store the password. bcrypt is a deliberately slow, salted, one-way hash

```
const passwordHash = await bcrypt.hash(password, 10) // ~100ms by design:
brute force is expensive

// bcrypt salts every hash, so two users with the same password get
different hashes.
```

145. Insert the user and look hard at the RETURNING list — password_hash is NOT in it, and never will be

```
const rows = await sql`
  insert into users (email, name, password_hash)
  values (${email.trim().toLowerCase()}, ${name}, ${passwordHash})
  on conflict (email) do nothing
  returning id::int as id, email, name, created_at
`

// zero rows = the email is taken -> 409. The hash never leaves this
function.
```

146. Log in with bcrypt.compare — never re-hash and compare with ===, and give ONE message for both failures

```
const ok = user ? await bcrypt.compare(password, user.password_hash) :
false

if (!ok) throw new HttpError(401, 'Invalid email or password.')

// Separate messages for 'no such email' and 'wrong password' hand an
attacker a
```

// free tool for discovering which emails have accounts.

147. Sign a JWT. The payload is base64, NOT encrypted — anyone can read it, so put no secret in it

```
export function signToken(user) {  
  return jwt.sign({ sub: String(user.id), email: user.email, name:  
    user.name },  
    JWT_SECRET, { expiresIn: '7d' })  
}  
  
// What makes it trustworthy is the SIGNATURE. Edit one byte of the payload  
and  
  
// verify() throws – which is why we may trust `sub` and must not trust  
req.body.
```

148. Write requireAuth — the single source of identity for the whole API

```
export function requireAuth(req) {  
  const header = req.headers?.authorization ?? ''  
  
  if (!header.startsWith('Bearer ')) throw new HttpError(401, 'Missing  
  Authorization header.')  
  
  const payload = jwt.verify(header.slice(7).trim(), JWT_SECRET) //  
  throws if tampered or expired  
  
  return Number(payload.sub) // the user's id – VERIFIED, not claimed  
}
```

149. THE RULE. In POST /api/enrollments the caller chooses the COURSE. The caller does NOT choose the USER

```
const userId = requireAuth(req) // from the token  
const { courseId } = readBody(req) // from the caller  
  
await sql`insert into enrollments (user_id, course_id, status)  
  values (${userId}, ${courseId}, 'active')  
  on conflict (user_id, course_id) do nothing  
  returning id::int as id`  
  
// If this trusted req.body.userId, anyone could enrol anyone. And they  
would.
```

150. Enforce OWNERSHIP in the SQL, not in an if. The id in the URL is a REQUEST, not a PERMISSION

```
// api/enrollments/[id].js

await sql`delete from enrollments

    where id = ${id} and user_id = ${userId}

    returning id::int as id`

//                      ^^^^^^^^^^^^^^^^^^^^^ from the verified JWT

// A row you do not own simply does not match -> 0 rows -> 404. Get this
wrong and

// you have shipped an Insecure Direct Object Reference, the classic API
hole.
```

151. Scope every read the same way — this WHERE clause IS the access control. There is no RLS behind it

```
select ... from enrollments e join courses c on c.id = e.course_id
where e.user_id = ${userId}
order by e.created_at desc
```

152. Build AuthContext on top of it: store the token, send it on every request, verify it on boot with /api/auth/me

```
// src/lib/api.js

...(token ? { Authorization: `Bearer ${token}` } : {}),
```

```
// src/context/AuthContext.jsx – on boot, ask the SERVER who we are

const { user } = await api.get('/auth/me') // 401 -> clearToken()
```

153. ATTACK YOUR OWN API. Sign in as student A, then try to delete student B's enrolment by guessing the id

```
curl -X DELETE http://localhost:3000/api/enrollments/1 \

-H "Authorization: Bearer <STUDENT-A-TOKEN>"
```

-> 404 Enrollment not found. The row exists – it is just not yours.

154. Tamper with the token in devtools (change one character) and refresh. The signature no longer matches

```
# localStorage['cookbake.token'] -> edit a char -> refresh -> 401 -> signed out.
```

```
# The browser cannot lie about who it is.
```

Test it

You can sign up, sign in, refresh and stay signed in; enrolling writes a row you can see in Neon; password_hash appears in NO API response; deleting another student's enrolment by id returns 404; and editing the JWT in localStorage signs you straight out.

Watch out for

Store a bcrypt hash, never the password, and never select `password\_hash` into a response. The user id comes from the VERIFIED JWT (`sub`), never from `req.body` — if the body could name the user, anyone could enrol anyone. Enforce ownership in the SQL itself (`and user\_id = \${userId}`), not in an `if`: an id in the URL is a request, not a permission.

Topic 06 — React Router for Real App Navigation

Routes · Layouts · Dynamic params · Protected routes · Deploy full stack

A real application has more than one page. React Router turns your single HTML document into a multi-page experience: the URL changes, the view swaps, and the server is never asked for a new document.

Two lessons matter most here. Put filter state in the URL, not in a component, so a filtered view is shareable, bookmarkable and survives the back button. And when you guard a route behind authentication, wait for the session to finish loading before you redirect — otherwise a signed-in user is bounced to /login on every single refresh. That guard is user experience, not security; the database policies you wrote in Topic 5 are what actually protect the data.

Key Concepts — Topic 06

- **A Single Page Application** — One HTML document. JavaScript swaps the view and rewrites the URL with no server round-trip.
- **<Link> not <a>** — A plain anchor reloads the page and throws away every piece of React state you were holding.
- **Layout routes and <Outlet/>** — A pathless parent route renders the shared chrome; <Outlet/> is where the matched child appears.
- **URL state beats component state** — Filters in ?category=Frontend are shareable, bookmarkable, and survive the back button and a refresh.
- **Wait for auth before you redirect** — The session resolves asynchronously. Guard on loading first, or a signed-in user is bounced to /login on every refresh.

- **A client guard is UX, not security** — Anyone can edit the JavaScript. The check inside the API route — user id from the verified token, ownership in the SQL — is what actually protects the data.

Concepts Explained — Topic 06

A Single Page Application

Think of it like... A single-page app is a theatre with one stage — JavaScript changes the scenery and the marquee while the audience never leaves their seats.

- A traditional site fetches a fresh HTML page from the server on every navigation.
- A single-page app loads one HTML document, then JavaScript swaps the view in place.
- The router also rewrites the URL bar so back, forward and bookmarks still work.
- No server round-trip means navigation is instant and React state is preserved across views.
- React Router maps each URL path to the component that should render for it.

```
// src/App.jsx (real code)
```

```
<Routes>

  <Route path="/" element={<RootLayout />}>

    <Route index element={<HomePage />} />

    <Route path="courses" element={<CoursesPage />} />

    <Route path="courses/:slug" element={<CourseDetailPage />} />

    <Route path="*" element={<NotFoundPage />} />

  </Route>

</Routes>
```

Note: One document; JS swaps views and rewrites the URL, no server reload.

Use <Link>, not <a>

Think of it like... An <a> tag rebuilds the whole theatre between scenes; <Link> just swaps the set, keeping every actor in place.

- A plain <a href> triggers a full-page reload, refetching the whole app from scratch.
- That reload throws away every piece of React state — the shortlist, the search box, the scroll.

- `<Link to>` navigates within the SPA, swapping the view without any reload.
- The whole `CourseCard` is a `<Link>`, so clicking anywhere on a card routes to its detail page.
- Use `<a>` only for links that truly leave your app; use `<Link>` for every internal route.

```
// src/components/CourseCard.jsx (real code)

<Link to={` /courses/${slug}`} className="card">

  ...the entire card...

</Link>
```

```
<a href="/courses">Browse</a>    // full reload: state gone
```

Note: `<Link>` navigates in-app and keeps state; `<a>` reloads and loses it.

Layout routes and `<Outlet/>`

Think of it like... A layout route is a picture frame with a window — `<Outlet/>` is the window where each page's picture slots into the same frame.

- A layout route is a parent route that renders shared chrome like the navbar and footer.
- Its child routes render inside it, so the shared frame is written once, not per page.
- The layout component renders `<Outlet/>` to mark where the matched child should appear.
- Layouts nest: `DashboardPage` is itself a layout, with `My Courses` and `Profile` inside its own `Outlet`.
- A `PATHLESS` parent route adds no URL segment — which is exactly how `ProtectedRoute` gates a subtree.

```
// src/layouts/RootLayout.jsx (real code)

export default function RootLayout() {

  return (

    <>

      <Navbar />

      <main><Outlet /></main>    {/* the matched child */}

    </>

  )
}
```

```

    <Footer />

  </>
)
}

```

Note: A layout route holds shared chrome; <Outlet/> is where children render.

Dynamic params and useParams

Think of it like... A dynamic segment is a mail slot labelled :slug — useParams reaches in and reads whichever course the URL dropped through.

- A path segment starting with a colon, like :slug, is a placeholder that matches any value.
- One route definition serves all 20 courses: /courses/macaron-masterclass, /courses/knife-skills...
- Inside the component, useParams() returns an object of the matched segment values.
- Read it with const { slug } = useParams(), then fetch that one course from /api/courses/:slug.
- useNavigate() moves programmatically: navigate(-1) goes back with no hardcoded path.

```
<Route path="courses/:slug" element={<CourseDetailPage />} />
```

```

// src/pages/CourseDetailPage.jsx (real code)

const { slug } = useParams()

const { course, loading, error } = useCourse(slug)

// null course = no such slug -> render a real 404 page

```

Note: A :param matches any value; useParams() reads it inside the component.

URL state beats component state

Think of it like... Putting filters in the URL is writing them on a shareable ticket — bookmark it, send it, hit back, and the same view returns.

- Filters, search terms and pagination are state, but they belong in the URL, not just `useState`.
- `/courses?category=Bakery&q=sourdough` is shareable, bookmarkable and survives a refresh.
- It also makes the browser back button restore the previous filter instead of leaving the app.
- `useSearchParams` reads and writes the query string exactly like a piece of state.
- Debounce before you write: one history entry per keystroke would wreck the back button.

```
// src/pages/CoursesPage.jsx (real code)

const [searchParams, setSearchParams] = useSearchParams()

const category = searchParams.get('category') ?? 'All'

const q = searchParams.get('q') ?? ''

// { replace: true } updates the URL without a new
// history entry – the debounced value is what we write.
```

Note: Put shareable view state in the URL with `useSearchParams`, not `useState`.

Protected routes wait for auth

Think of it like... Guarding before auth loads is checking a wristband in the dark — wait for the lights (loading) or you eject your own paying guest.

- A protected route should render only for a signed-in user and redirect everyone else.
- The catch is that the session resolves asynchronously: on boot we ask `/api/auth/me` to verify the token.
- During that moment ``user`` is null but nobody is actually signed out. You know nothing yet.
- Guard on loading FIRST: while loading, show a message and redirect nobody.
- Skip that check and a signed-in student is bounced to `/login` on every single refresh.

```
// src/components/ProtectedRoute.jsx (real code)

const { user, loading } = useAuth()
```

```

if (loading) return <p>Checking your session...</p> // FIRST
if (!user) {
  return <Navigate to="/login" replace
    state={{ from: location }} /> // remember where
}
return <Outlet />

```

Note: Wait for loading before redirecting, or you eject real users on refresh.

A client guard is UX, not security

Think of it like... A client-side guard is a velvet rope, not a vault — anyone can step over it in devtools; only the API can lock the door.

- Hiding a link or redirecting from a route improves the experience for honest users.
- It is not security: anyone can edit the JavaScript, or skip the app entirely and call `/api/` with curl.
- The client cannot be trusted, because it runs entirely on the user's own machine.
- Real protection is in the API: `requireAuth(req)` for identity, and ``and user_id = ${userId}`` for ownership.
- Design so that a completely bypassed client guard exposes nothing the server would not already allow.

// Hiding the link is NOT security:

```
{user && <NavLink to="/dashboard">Dashboard</NavLink>}
```

// THIS is security – and it runs on the server:

```

const userId = requireAuth(req) // 401 if forged
await sql`select * from enrollments
  where user_id = ${userId}` // your rows only

```

Note: Client guards are UX only; the API's auth checks are what protect the data.

Deep Dive — Single-page apps and client-side routing

A traditional website fetches a fresh HTML page from the server every time you navigate — you physically move to a new document. A single-page application works differently: it loads one HTML document once, and from then on JavaScript swaps the visible view in place and rewrites the URL bar, with no server round-trip. It is a theatre with a single stage where the scenery and the marquee change while the audience never leaves their seats. The benefit is that navigation is instant and, crucially, your React state survives it — the shortlist, the search box, the scroll position are all still there because the page never actually reloaded.

React itself has no router; routing is a separate concern solved by a library, and this course uses React Router in its declarative mode. You declare a set of routes that map URL paths to the components that should render for them, and the router watches the URL and renders the match. This is why the choice between `<Link>` and a plain `<a>` tag is not cosmetic. An `<a href>` triggers a full-page reload: it rebuilds the entire theatre between scenes and throws away every piece of React state you were holding. `<Link to>` navigates within the SPA — it swaps the set and updates the URL while keeping every actor in place. In Cook & Bake the whole `CourseCard` is a `<Link>`, so clicking anywhere on a card routes to its detail page with no reload. Use `<a>` only for links that truly leave your app, and `<Link>` for every internal route.

- An SPA loads one document; JavaScript swaps views and rewrites the URL.
- Navigation is instant and React state survives because nothing reloads.
- React has no built-in router; React Router maps paths to components.
- `<Link>` navigates in-app and preserves state; `<a>` reloads and discards it.

```
// src/components/CourseCard.jsx (real code)

<Link to={` /courses/${slug}`} className='card'>

  ...the entire card...

</Link>
```

```
<a href='/courses'>Browse</a>    // full reload: state gone
```

Deep Dive — Layouts, dynamic params and URL state

Real apps share chrome — a navbar, a footer, a sidebar — across many pages, and you do not want to repeat it in every component. A layout route solves this: it is a parent route that renders the shared structure once, and its child routes render inside it. The layout marks where children go by rendering `<Outlet/>` — a picture frame with a window, where each page's picture slots into the same surrounding frame. `RootLayout` is exactly this: it renders the `Navbar` and `Footer` once and an `<Outlet/>` in between for the matched page. Layouts nest, too: `DashboardPage` is itself a layout with its own `<Outlet/>`, inside which `My Courses` and `Profile` render. And a `PATHLESS` parent route — one with an

element but no path — adds no URL segment at all, which is precisely how `ProtectedRoute` wraps and gates a whole subtree without changing any address.

Most real routes are not fixed strings but patterns. A path segment written with a leading colon, like `/courses/:slug`, is a placeholder that matches any value, so one route definition serves all twenty courses, from `/courses/macaron-masterclass` to `/courses/knife-skills-and-kitchen-essentials`. Inside the component, `useParams()` reads the matched value out — `const { slug } = useParams()` — like reaching into a labelled mail slot and pulling out whichever course the URL dropped through; `CourseDetailPage` then feeds that slug to `useCourse` to fetch the one course. When you need to navigate in code rather than from a click — after saving a form, or a back button — `useNavigate()` does it, and `navigate(-1)` simply returns to wherever the user came from with no hardcoded path.

Finally, an important habit: some state belongs in the URL, not in `useState`. Filters, search terms and pagination are the classic cases. `CoursesPage` keeps its category and search query in the URL with `useSearchParams`, so `/courses?`

`category=Bakery&q=sourdough` is a shareable ticket — bookmark it, send it to a colleague, press the back button, refresh the page, and the exact same filtered view returns every time. If that state lived only in component state it would vanish on refresh and could not be shared. One subtlety the real page handles: it writes the debounced query to the URL with `{ replace: true }`, so a burst of keystrokes does not stuff the history with an entry per character and wreck the back button. The rule of thumb is: if a view should be reproducible from its link alone, its state belongs in the URL.

- A layout route renders shared chrome once; `<Outlet/>` is where children appear, and layouts nest.
- A `:param` matches any value; `useParams()` reads it, and a pathless route gates a subtree.
- `useNavigate()` moves programmatically; `navigate(-1)` goes back with no hardcoded path.
- Put shareable view state (filters, search) in the URL with `useSearchParams`, debounced with `replace`.

```
<Route path='courses/:slug' element={<CourseDetailPage />} />
```

```
// src/pages/CourseDetailPage.jsx (real code)
```

```
const { slug } = useParams()
```

```
const { course, loading, error } = useCourse(slug)
```

Deep Dive — Protected routes, and why a client guard is not security

Some pages should only render for a signed-in user — the private dashboard, the profile page. A protected route wraps such a page and redirects everyone else to the login

screen. The subtle part is timing. When your app first loads, the authentication session resolves asynchronously: AuthContext holds a JWT from a previous visit in localStorage, but this page load has never had it checked, so on boot it calls /api/auth/me to have the server verify the signature and hand back the current user. For that brief moment you do not yet know whether anyone is signed in. If your guard treats that unknown moment as 'not signed in' and redirects, then a genuinely signed-in user gets bounced to /login on every single refresh, because the redirect fires before the verification returns. It is checking a wristband in the dark and ejecting your own paying guest. The fix, which ProtectedRoute implements, is to guard on the loading state first: while loading, render a 'Checking your session...' message and redirect nobody; only once loading is false do you decide between rendering the page and navigating to login.

The most important idea in the whole topic, though, is that a client-side guard is user experience, not security. Hiding the Dashboard link from signed-out users and redirecting them away from the route makes the app pleasant and coherent — but it protects nothing, because all of that code runs on the user's own machine. Anyone can open dev tools, edit the JavaScript, or ignore the app entirely and call your API directly with curl, stepping straight over the velvet rope. Real protection lives on the server, in the /api/ functions from Topic 5: requireAuth(req) rejects any request without a validly signed token, and every query that reads or writes a user's data carries `and user\_id = \${userId}` with the id taken from that verified token. So even if someone completely bypasses ProtectedRoute and hits GET /api/enrollments by hand, the server still only ever returns their own rows, and a request with no valid token gets a 401. Design so that a bypassed client guard exposes nothing the server would not already allow: the client makes the app nice to use; the API is what makes it safe.

- A protected route renders only for signed-in users and redirects the rest.
- The session loads asynchronously (a /api/auth/me check) — guard on loading before you redirect.
- Skip that check and real users get bounced to login on every refresh.
- A client guard is UX only; the API's requireAuth plus `user\_id = \${userId}` is what protects data.

```
// src/components/ProtectedRoute.jsx (real code)

const { user, loading } = useAuth()

if (loading) return <p>Checking your session...</p> // FIRST

if (!user) return <Navigate to='/login' replace

  state={{ from: location }} />

return <Outlet />
```

```
// The REAL guard, server-side: requireAuth(req) + user_id = ${userId}
```

Lab 6.1 — Routes, Links and Layout Routes

Objective: turn a single page into a multi-page single-page application.

Goal: Cook & Bake Academy is one long scroll. The learner installs React Router, splits it into Home, Courses and About, and builds a layout route whose `<Outlet/>` renders the matched page inside the shared navbar and footer.

What you'll build

`src/layouts/RootLayout.jsx` with an `<Outlet/>`, plus `HomePage`, `CoursesPage` and `AboutPage` (Tech & files: `react-router-dom`, `BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink`, `Outlet`.)

Step-by-step

155. Install the router and wrap the app — `BrowserRouter` goes OUTSIDE the providers so every page can route

```
npm install react-router-dom
```

```
<BrowserRouter>

  <ThemeProvider><AuthProvider><CartProvider><App
/></CartProvider></AuthProvider></ThemeProvider>

</BrowserRouter>
```

156. Prompt the agent for the split, and name the files you want

```
// Vibe prompt: 'Split the Cook & Bake app into routes with react-router-
dom v7:

// / (HomePage), /courses (CoursesPage), /about (AboutPage). Put Navbar and
Footer

// in src/layouts/RootLayout.jsx and render the matched page in its
<Outlet/>.

// Declare the route map in src/App.jsx. Use <Link>, never a plain <a>.'
```

157. Declare the route map — the nesting in `App.jsx` mirrors the nesting of the URL

```
<Routes>

  <Route path="/" element={<RootLayout />}>

    <Route index element={<HomePage />} />

    <Route path="courses" element={<CoursesPage />} />

    <Route path="about" element={<AboutPage />} />
```

```
</Route>
```

```
</Routes>
```

158. Build `RootLayout`: the chrome that never re-mounts, and the hole the page renders into

```
<Navbar />
```

```
<main style={{ minHeight: '70vh' }}><Outlet /></main>
```

```
<Footer />
```

159. Navigate with `<Link>`. A plain `<a>` reloads the document and throws away your shortlist, theme and session

```
<Link to="/courses">Courses</Link> // 🛑 <a href="/courses"> – full reload, state gone
```

160. Style the current tab with `NavLink`'s `isActive` render prop — the component never tracks the URL by hand

```
const navClass = ({ isActive }) => (isActive ? 'nav__link is-active' : 'nav__link')
```

```
<NavLink to="/courses" className={navClass}>Courses</NavLink>
```

161. Watch the network tab while you click `Courses`: no document is requested. That is the SPA

162. Connect it back to Topic 2 — this is exactly why a static host needs the SPA rewrite

```
// vercel.json – every path that is not /api/* falls through to index.html
{ "rewrites": [{ "source": "/((?!api/.*)", "destination": "/index.html" }]
}
```

Test it

Clicking `Courses` and `About` swaps the page with no full reload, the navbar and footer never re-mount (the 🍪 count and the theme survive), and the active tab is highlighted.

Watch out for

Use `<Link>`, never a plain `<a>`. An anchor triggers a full page reload, which throws away all your React state and refetches the whole bundle.

Lab 6.2 — Dynamic Routes, URL State and 404s

Objective: read route parameters, navigate imperatively, and keep filter state in the URL.

Goal: Each of the 20 courses needs its own shareable URL. The learner adds `/courses/:slug` backed by the `useCourse` hook and the `[slug]` API route from Topic 5, moves the search and category filters into the query string, and adds a real catch-all 404.

What you'll build

`CourseDetailPage` at `/courses/:slug`, `NotFoundPage`, and shareable filter URLs like `/courses?category=Bakery&q=sourdough` (Tech & files: `useParams`, `useNavigate`, `useSearchParams`, catch-all routes, HTTP 404.)

Step-by-step

163. Add the dynamic segment and read it with `useParams` — the URL is the input to the page

```
<Route path="courses/:slug" element={<CourseDetailPage />} />
```

```
const { slug } = useParams()
```

```
const { course, loading, error } = useCourse(slug)
```

164. The route param goes straight to the API route you built in Lab 5.2 — and is bound as a SQL parameter there

```
GET /api/courses/artisan-sourdough-bread-baking
```

165. Render THREE outcomes, not two: loading, a real error, and a real not-found

```
if (loading) return <div className="skeleton skeleton--tall" />
```

```
if (error) return <p className="error">{error}</p>
```

```
if (!course) return <Section title="Course not found">...</Section> // the API said 404
```

166. Add the catch-all route LAST — any URL that matched nothing above lands here

```
<Route path="*" element={<NotFoundPage />} />
```

167. Navigate imperatively after an action with `useNavigate`; `-1` goes back with no hardcoded path

```
const navigate = useNavigate()
```

```
<button onClick={() => navigate(-1)}>← Back</button>
```

168. Move the filters into the URL so a filtered view is shareable, bookmarkable and survives a refresh

```
const [searchParams, setSearchParams] = useSearchParams()

const category = searchParams.get('category') ?? 'All'

const q = searchParams.get('q') ?? ''
```

169. Write the DEBOUNCED value to the URL, with replace, or you push a history entry per keystroke

```
useEffect(() => {

  setSearchParams((prev) => { /* set or delete q */ }, { replace: true })

}, [debounced, setSearchParams])
```

170. Paste the URL into a fresh tab and confirm the exact filtered grid comes back

<http://localhost:3000/courses?category=Bakery&q=sourdough>

171. Learn the rule: <Link> for navigation the user initiates, useNavigate for navigation an action causes

Test it

Clicking a course card opens /courses/<slug> with its fee, weeks and campus; /courses/does-not-exist renders 'Course not found' and /nonsense renders the 404 page; and pasting /courses?category=Bakery&q=sourdough into a new tab restores the filtered grid.

Watch out for

Have the API return a real 404 when the slug matches nothing, and let the hook surface it — that is what lets you render a proper 'course not found' page instead of a spinner that never stops.

Lab 6.3 — Protected Routes and the Student Dashboard

Objective: guard routes behind authentication and nest a dashboard layout.

Goal: Enrolments are private. The learner builds a ProtectedRoute layout route over the AuthContext from Topic 5, nests My Courses and Profile inside a dashboard layout, returns the student to the page they were trying to reach after login — and learns exactly why a client guard is not security.

What you'll build

src/components/ProtectedRoute.jsx and a nested /dashboard with /dashboard/my-courses and /dashboard/profile (Tech & files: Navigate, useLocation, nested layout routes, Outlet, redirect-after-login.)

Step-by-step

172. Write the guard as a pathless LAYOUT route — everything nested inside it is gated by one component

```
<Route element={<ProtectedRoute />}>
  <Route path="dashboard" element={<DashboardPage />}>
    <Route index element={<Navigate to="my-courses" replace /> } />
    <Route path="my-courses" element={<MyCoursesPage /> } />
    <Route path="profile" element={<ProfilePage /> } />
  </Route>
</Route>
```

173. CRITICAL — wait for the session to resolve BEFORE you redirect

```
const { user, loading } = useAuth()

if (loading) return <p className="section muted">Checking your session...</p>

if (!user) return <Navigate to="/login" replace state={{ from:
location }} />

return <Outlet />
```

174. Delete that loading line and refresh /dashboard while signed in — you are bounced to /login every time

```
// On boot AuthContext is still asking /api/auth/me to verify the JWT, so
`user`

// is momentarily null. Redirecting on !user alone punishes every signed-in
user.
```

175. Use replace so the back button does not trap the student in a redirect loop
176. Capture the attempted URL in location state and send them back there after they sign in

```
// LoginPage

const from = location.state?.from?.pathname ?? '/dashboard'

useEffect(() => { if (user) navigate(from, { replace: true }) }, [user,
from, navigate])
```

177. Render the dashboard's own nested layout — relative NavLinks and a second <Outlet/>

```
<NavLink to="my-courses" className={tabClass}>My courses</NavLink>
```

```
<NavLink to="profile" className={tabClass}>Profile</NavLink>
```

```
<Outlet />
```

178. Feed My Courses from useEnrollments — and notice the hook never sends a user id

```
const { enrollments, loading, error, updateStatus, remove } =  
useEnrollments()
```

```
// The browser says WHAT it wants; the server decides WHO is asking (Lab  
5.4).
```

179. Now prove the boundary. Delete the ProtectedRoute guard in devtools and open /dashboard/my-courses

```
// The page renders. The LIST IS EMPTY: GET /api/enrollments answered 401.  
// The guard is UX. `where user_id = ${userId}` in the API is the security.
```

Test it

Visiting /dashboard/my-courses signed out redirects to /login; after signing in you land back on /dashboard/my-courses; refreshing while signed in keeps you there; and bypassing the client guard shows an empty, 401'd page — no data leaks.

Watch out for

Wait for `useAuth().loading` to become false before redirecting, or a signed-in user is bounced to `/login` on every refresh. And remember: anyone can edit your JavaScript, so this guard is UX only. The check inside the API route — user id from the token, ownership in the SQL — is the actual security boundary.

Lab 6.4 — Capstone — Assemble the Full-Stack App

Objective: assemble every concept in the course into one working application.

Goal: Everything is now built. The learner walks the finished Cook & Bake Academy against a rubric that maps each concept in the course to the file in their own project that proves it, runs the AI React Bug Checklist over the generated code, and builds it clean.

What you'll build

The complete Cook & Bake Academy: catalogue, search, shortlist, theme, API, database, auth, dashboard and routing (Tech & files: React 19, React Router 7, Vercel Functions, Neon Postgres, bcrypt + JWT.)

Step-by-step

180. Confirm the provider order in main.jsx — routing outermost, app state inside, <App/> last

```
<BrowserRouter><ThemeProvider><AuthProvider><CartProvider><App  
/></CartProvider></AuthProvider></ThemeProvider></BrowserRouter>
```

181. Walk the rubric and POINT AT THE FILE for each concept — props, keys, effects, refs, context, reducer

components/CourseCard.jsx · components/CourseGrid.jsx (key={c.id})

context/ThemeContext.jsx · context/CartContext.jsx

182. Point at the ref, and say the rule again

pages/CoursesPage.jsx -> useRef -> focus the search input

pages/HomePage.jsx -> useRef -> scrollIntoView the course grid

// Redraw the screen -> state. Don't redraw -> ref.

183. Point at every custom hook you own

hooks/useCourses.js · useCourse.js · useEnrollments.js · useReviews.js

hooks/useDebounce.js · useLocalStorage.js

184. Point at the three states — loading, error, success — and the finally, in every data hook

185. Point at the security boundary, and note that it is NOT in any line of the front end

api/_lib/auth.js -> requireAuth(req) - identity comes from the verified JWT

api/enrollments/[id].js -> where id = \${id} and user_id = \${userId}

186. Run the AI React Bug Checklist over the agent's code and fix what it catches

// key={index}? · a missing effect cleanup? · {count && ...} rendering a 0?

// a fetch with no res.ok check? · state mutated with push? · a stored derived value?

187. Build it clean before you ship — a warning today is a broken deploy tomorrow

npm run lint && npm run build

Test it

The app builds with no errors or warnings, every rubric row maps to a real file in your project, and you can defend each architectural choice out loud — especially why the browser has no database credentials.

Watch out for

Run the AI React Bug Checklist over your own code before you call the capstone done.

Lab 6.5 — Deploy the Full-Stack App to Vercel and Neon

Objective: deploy a React front end, a serverless API and a live database to production.

Goal: The learner ships the whole three-tier app. The front end builds to static files, the `api/` folder becomes real serverless functions, and `DATABASE_URL` and `JWT_SECRET` are set as ENCRYPTED, SERVER-SIDE Vercel environment variables — never with a `VITE_` prefix. The result is the live app at `cookbake-academy.vercel.app`.

What you'll build

A live, public, full-stack Cook & Bake Academy — real Postgres, real accounts, real enrolments (Tech & files: Vercel, Vercel Functions, environment variables, `vercel.json` rewrites, Neon.)

Step-by-step

188. Run the schema against the database the PRODUCTION deployment will use

```
psql "$DATABASE_URL" -f neon/schema.sql # tables + the 20 seeded courses
```

189. Import the repo in Vercel: framework Vite, build ``npm run build``, output ``dist``. The `api/` folder is detected automatically

190. Set the two SERVER-SIDE secrets in Vercel → Settings → Environment Variables (Production AND Preview)

```
DATABASE_URL = postgresql://...@ep-....aws.neon.tech/neondb?sslmode=require
```

```
JWT_SECRET = <32 random bytes>
```

```
// Neither has a VITE_ prefix. Vercel stores them ENCRYPTED and injects them into
```

```
// the serverless functions only – process.env.DATABASE_URL. They are never in the bundle.
```

191. Say why one more time. A `VITE_` variable is COMPILED INTO the JS every visitor downloads

```
// 🚫 VITE_DATABASE_URL would publish your database password, with write access,
```

```
// on the public internet. 'Fixing' a connection error by adding VITE_ is the
```

```
// single most expensive mistake in this course.
```

192. Ship the SPA rewrite that excludes /api — or every API call falls through to index.html and returns HTML

```
{ "rewrites": [{ "source": "/((?!api/).*)", "destination": "/index.html" }] }
```

193. Deploy by pushing. Vercel builds the front end AND deploys api/* as functions

```
git add -A && git commit -m 'feat: full-stack Cook & Bake Academy' && git push
```

194. Smoke-test production in this order: catalogue → detail page → sign up → enrol → refresh → dashboard

```
curl https://cookbake-academy.vercel.app/api/courses | head -c 200
```

195. Deep-link test: paste a course URL into a fresh tab and refresh it. The rewrite is what makes this work

```
https://cookbake-academy.vercel.app/courses/artisan-sourdough-bread-baking
```

196. Security review — prove the secrets never reached the browser

```
npm run build
```

```
grep -r "postgresql://" dist/ # nothing
```

```
grep -ri "jwt_secret" dist/ # nothing
```

```
# Then: DevTools → Sources → read the bundle yourself. No password. No SQL.
```

Test it

The live URL loads the 20 courses signed out; a deep link survives a refresh; you can sign up, enrol and see it on the dashboard in production; and grepping the built bundle for the connection string or the JWT secret returns nothing.

Watch out for

Set `DATABASE\_URL` and `JWT\_SECRET` as Vercel environment variables (Production, encrypted). They are server-side only, so they never reach the browser. Confirm the SPA rewrite excludes `/api/\*` — a catch-all to `index.html` will happily swallow every API route and hand your fetch an HTML page to parse.

Lab 6.6 — Mini-Capstone — Add Course Reviews, On Your Own

Objective: extend the deployed app with a new full-stack feature, mostly by vibe-coding it yourself.

Goal: The app is built and live. Now you add one brand-new END-TO-END feature — star-rated course reviews — mostly on your own. You write the prompts, read the generated code against everything this course taught you, correct it, and redeploy. This is the exam of the whole course: a new API route with parameterised SQL, ownership from the JWT, a custom hook, a controlled form, a derived average, and a git push that puts it in production.

What you'll build

Reviews on the course detail page: a public list, a form only signed-in students see, a live average rating, one review per student per course — deployed (Tech & files: `api/reviews/*`, an UPSERT, a custom hook, controlled forms, derived state, Vercel redeploy.)

Step-by-step

197. The reviews table is already in `neon/schema.sql`. Read it — the unique constraint is the feature spec

```
create table public.reviews (  
  id          bigint generated always as identity primary key,  
  user_id     bigint not null references public.users (id) on delete  
cascade,  
  course_id   bigint not null references public.courses (id) on delete  
cascade,  
  rating      int      not null check (rating between 1 and 5),  
  body        text     not null check (length(body) between 1 and 2000),  
  created_at  timestamptz not null default now(),  
  unique (user_id, course_id)      -- one review per student per course  
);
```

198. Prompt for the API route yourself — and put the security rule INTO the prompt

```
// Vibe prompt: 'Create api/reviews/index.js (a Vercel function).  
  
// GET ?courseId= : PUBLIC, no token. Join users for the reviewer NAME  
only –  
  
//      never email, never password_hash.  
  
// POST {courseId, rating, body} : requireAuth(req) for the user id – NEVER  
take  
  
//      a user id from the body. UPSERT with on conflict (user_id,  
course_id).
```

```
//      Validate rating 1-5 and body length server-side. Tagged-template SQL only.'
```

199. Read what it wrote. Is `requireAuth` INSIDE the POST branch, so signed-out visitors can still READ?

```
if (req.method === 'GET') { /* no token needed */ }
```

```
const userId = requireAuth(req) // only for the write
```

200. Check the UPSERT — one statement, race-free, and it can only ever touch YOUR row

```
insert into reviews (user_id, course_id, rating, body)
values (${userId}, ${courseId}, ${rating}, ${body.trim()})
on conflict (user_id, course_id) do update
    set rating = excluded.rating, body = excluded.body, created_at = now()
returning id::int as id, user_id::int as user_id, rating, body, created_at
```

201. Write `api/reviews/[id].js` for DELETE. Same rule as enrolments — the id says WHICH, the token says WHOSE

```
delete from reviews
where id = ${id} and user_id = ${userId}
returning id::int as id

// 0 rows -> 404. You cannot delete somebody else's review by guessing its id.
```

202. Prompt for the hook, and DERIVE the average — never store it on the course

```
// Vibe prompt: 'Write src/hooks/useReviews.js like useEnrollments: load
// GET /api/reviews?courseId=, expose submit() and remove(), find myReview,
// and DERIVE count and average at render. Return { data } / { error },
never throw.'
```

203. Check the derivation and the id comparison — a string/number mismatch here breaks 'my review' silently

```
const myReview = reviews.find((r) => r.user_id === user?.id) ?? null //
both ints (::int in SQL)

const average = count
```

```
    ? Math.round((reviews.reduce((s, r) => s + r.rating, 0) / count) * 10) /  
    10
```

```
    : null    // derived at render, not stored
```

204. Build the star rating as real BUTTONS (keyboard + screen-reader accessible), and a controlled form

```
<button type="button" onClick={() => onChange(n)} aria-label={`Rate ${n}  
stars`} >★</button>
```

205. Gate writing behind auth in the UI; show the list to everyone

```
{user ? <ReviewForm existing={myReview} onSubmit={submit} />  
  : <p className="muted"><Link to="/login">Sign in</Link> to write a  
review.</p>}
```

206. Attack it before you ship it — the two probes from Topic 5, against your own new endpoint

```
curl "http://localhost:3000/api/reviews?courseId=1'%20or%20'1'='1"      #  
harmless
```

```
curl -X DELETE http://localhost:3000/api/reviews/1 -H "Authorization:  
Bearer <OTHER-USER>" # 404
```

207. Commit and push. Vercel redeploys the front end and the new function, and your feature is live

```
git add -A && git commit -m 'feat: course reviews' && git push
```

Test it

Signed out you can read reviews but not write; signed in you can post exactly one review per course, edit it and delete it; the average on the detail page updates live; another account cannot delete your review (404 from the SQL, not from an if); and after git push the feature is live on your Vercel URL.

Watch out for

This is the mini-capstone — you drive it, mostly by prompting. The traps are the ones the course already taught: derive the average rating at render (never store it), handle the 23505 unique-violation on a second review, take the user id from the token and never from the body, gate writing behind auth, and enforce ownership in the SQL so one student cannot edit another's review. Probe your own new endpoint for injection before you call it done, then push so Vercel redeploys.

Appendix A — The AI React Bug Checklist

This is the course in one page. Every item is a mistake AI coding agents reliably make, and every one of them appears in a lab. Run this list against generated code before you run the app.

Rendering

- `key={index}` on a list — breaks on reorder, insert and delete. Use a stable id. (Lab 3.3)
- `{count && <Badge/>}` renders a literal 0, because 0 is falsy but not false. Use `count > 0 && ...` (Lab 3.3)
- Mutating props or state — `cart.push(x)` does not re-render. Build a new array. (Labs 3.2, 4.1)
- `dangerouslySetInnerHTML` offered casually — it turns off the escaping that prevents XSS. (Lab 3.1)

State and effects

- Derived state stored in `useState` + `useEffect` — compute it during render instead. The most common AI React mistake. (Labs 3.4, 4.2)
- `setCount(count + 1)` called twice only increments once. State is a snapshot; use `setCount(c => c + 1)`. (Lab 4.1)
- `useEffect` with no cleanup — timers, subscriptions and listeners leak. `StrictMode` double-invokes effects to expose this. (Lab 4.2)
- Missing or wrong dependency array — `useEffect(fn)` with no array runs after every render. (Lab 4.2)
- `forwardRef` in React 19 — no longer needed; `ref` is an ordinary prop. (Lab 4.3)
- Reading or writing `ref.current` during render. (Lab 4.3)
- Assuming a custom hook shares state between components. It shares logic only. (Lab 4.5)
- Calling a data hook in two components — you get two divergent copies. Call it once, pass the pieces down. (Lab 5.3)

Data and the backend

- No error state — every remote read has three states; agents write two. (Lab 5.1)
- `response.ok` unchecked — `fetch` does not reject on a 404 or a 500. (Lab 5.1)
- An async `useEffect` callback — it returns a Promise where React expects a cleanup function. (Lab 5.1)
- No `AbortController` — a stale response overwrites a fresh one. (Lab 5.1)
- String-concatenating a value into SQL — `sql(`... ${x}`)` invites injection. Always use the `sql`` tagged template so the value travels separately from the query. (Lab 5.2)
- The browser talking to Postgres directly — only the `/api` functions hold the connection; the browser calls `/api`. (Lab 5.1)

- DATABASE_URL or JWT_SECRET in client code, or any secret in a VITE_ variable — VITE_ values are compiled into the public bundle. (Labs 2.2, 5.1)
- Storing a password instead of a bcrypt hash, or selecting password_hash into a response. (Lab 5.3)
- Trusting req.body for the user id — it comes from the verified JWT, never the request body. (Lab 5.4)
- Ownership checked in an if instead of the SQL — enforce `and user\_id = \${userId}` in the WHERE clause. (Lab 5.4)
- No loading gate on the session — the first paint shows 'signed out' to a signed-in user. (Labs 5.3, 6.3)

Routing

- <a href> instead of <Link> — a full page reload throws away all your React state. (Lab 6.1)
- A client-side guard mistaken for security — the API's token + ownership check is the real boundary. (Lab 6.3)
- Redirecting before auth resolves. (Lab 6.3)
- Deep links 404 on a static host — you need an SPA rewrite or a 404.html fallback. (Labs 2.2, 2.3, 6.5)
- Changing a VITE_ env var and expecting a live change — they are baked in at build time. (Labs 2.2, 6.5)

Appendix B — Troubleshooting

Symptom	Likely cause	Fix
A fetch to /api returns HTML, not JSON	The SPA rewrite is swallowing /api	Exclude /api from the rewrite: "/((?!api/).*)" (Labs 2.2, 6.5)
Env var changes have no effect	VITE_ vars are baked in at build time	Restart the dev server locally; redeploy on Vercel
500 from an /api route, logs say 'DATABASE_URL undefined'	The function has no env var	Set DATABASE_URL (and JWT_SECRET) in .env.local locally and in Vercel for production
Loading skeleton never stops	A rejected fetch with no catch/finally, or response.ok unchecked	Check response.ok; handle the error branch; clear loading in a finally (Lab 5.2)
'Objects are not valid as a React child'	Rendering an error object	Render error.message
401 from a protected route while signed in	The Bearer token is missing or stale	api.js must attach the JWT from localStorage; sign in again if it was tampered with (Lab 5.4)
A delete/patch returns 404 for a row you can see	Ownership check: it is not yours	The WHERE clause is `and user_id = \${userId}` — this is the guard working (Lab 5.4)
Insert fails with code 23505	The unique (user_id, course_id) constraint	Already enrolled/reviewed — handle it gracefully (Labs 5.4, 6.6)
Sign-up returns 409	The email is already registered	Log in instead, or use another email (Lab 5.3)
Signed-in user bounced to /login	Guard redirects before loading is	Wait for loading (Lab 6.3)

on refresh	false	
Deep link 404s in production	No SPA fallback on the static host	Add the vercel.json rewrite (Labs 2.2, 6.5)
Effect runs twice in development	StrictMode, on purpose	Add the missing cleanup

Appendix C — Glossary

- **API route (serverless function)** — A file in `api/` that runs on the server, holds the database connection and answers a browser fetch. The backend tier.
- **Babel** — The compiler that turns the JSX you write into plain JavaScript (`React.createElement` / `jsx` calls) the browser can run. Vite runs it for you.
- **bcrypt** — A deliberately slow, salted, one-way password hash. You store the hash; you never store or return the password itself.
- **Cleanup function** — The function returned from `useEffect`. React runs it before the next effect and on unmount.
- **Component** — A JavaScript function that returns JSX. The unit of reuse in React.
- **Connection string (DATABASE_URL)** — The credential that opens the database. It lives only on the server, never in client code, and never in a `VITE_` variable.
- **Controlled input** — An input whose value comes from state and whose `onChange` updates that state.
- **Custom hook** — A function whose name begins with 'use' and which calls other hooks. Shares logic; each caller gets its own state.
- **Declarative** — Describing what the UI should be for the current state, and letting React work out how to change the DOM.
- **Dependency array** — The second argument to `useEffect`. `None` = every render; `[]` = once; `[deps]` = when deps change.
- **Derived state** — A value computable from existing state. Compute it during render; do not store it.
- **DOM** — The browser's live object tree of the page. The page you see IS the DOM; changing it is what makes the screen update.
- **JSX** — The HTML-like syntax in `.jsx` files. Babel compiles it to function calls returning plain JavaScript objects.
- **JWT** — A signed token proving who a user is. The browser sends it as a Bearer header; the API verifies the signature and reads the user id from it. Trust the token, not the request body.
- **key** — The prop that gives React a stable identity for a list row across renders. Not merely 'an id React needs'.
- **Lifting state up** — Moving state to the closest common ancestor of the components that need it.
- **Parameterised query** — `sql`... where slug = ${slug}`` — the value is sent to Postgres separately from the SQL, so it can never become SQL. The defence against injection.
- **Props** — The arguments to a component. Read-only, and they flow one way: parent to child.

- **Reconciliation** — React diffing the new Virtual DOM tree against the previous one and patching only the difference.
- **Ref** — A mutable { current } box that survives re-renders and does not cause one when changed.
- **SPA** — Single Page Application — one HTML document; JavaScript swaps the view and rewrites the URL.
- **Three-tier architecture** — Browser → your /api functions → the database. The browser never talks to Postgres directly.
- **State** — Data a component remembers between renders, and which triggers a re-render when it changes.
- **StrictMode** — A development-only wrapper that double-invokes components and effects to surface missing cleanup.
- **Updater form** — `setCount(c => c + 1)`. Reads the latest value rather than the render's snapshot.
- **Virtual DOM** — React's in-memory object tree describing the UI. Cheap to build and diff; the real DOM is what is expensive.